Principles of Discrete Applied Mathematics

Mihir Rao

Contents

1 Pigeonhole Principle **4**
1.1 The principle . . . 4
1.2 Two equal degrees . . . 4
1.3 Equal-sum subsets . . . 4
1.4 Monotone subsequences . . . 5
1.5 Family trees . . . 6
1.6 Twenty questions . . . 6
1.7 Sample spaces and events . . . 6

2 Independence and Conditioning **8**
2.1 Combining events and conditioning . . . 8
2.2 Independence . . . 8
2.3 The law of total probability . . . 9
2.4 Bayes' rule . . . 9
2.5 Random variables and expectation . . . 10
2.6 Conditional expectation . . . 11

3 Inclusion-Exclusion **12**
3.1 The matching problem . . . 12
3.2 The inclusion-exclusion formula . . . 12
3.3 Linearity of expectation in action . . . 13
3.4 Independent random variables . . . 14
3.5 Variance . . . 14

4 Counting **16**
4.1 Binomial and multinomial coefficients . . . 16
4.2 Counting in two ways . . . 16
4.3 Bijections . . . 17
4.4 Catalan families . . . 18
4.5 Stirling's formula . . . 20

5 More Counting and Generating Functions **21**
5.1 Cayley's formula . . . 21
5.2 Generating functions . . . 22
5.3 Sums and products . . . 23
5.4 Making change . . . 23
5.5 Differentiation . . . 24

6 More on Generating Functions **25**
6.1 Sequences . . . 25
6.2 Dots and dashes . . . 25
6.3 Diagonals of Pascal's triangle . . . 27

7 Generating Functions for Catalan Numbers **28**
7.1 A quadratic equation . . . 28
7.2 A second derivation from Dyck paths . . . 29
7.3 Tiling a strip . . . 30
7.4 Rational generating functions and linear recurrences . . . 31

8 Tail Bounds **32**
8.1 Markov's inequality . . . 32
8.2 Chebyshev's inequality . . . 33
8.3 The weak law of large numbers . . . 33
8.4 Sums of independent random variables . . . 34
8.5 Boosting a randomized algorithm . . . 34
8.6 The probabilistic method . . . 35

9 Chernoff Bounds **37**
9.1 Statement . . . 37
9.2 Example: boosting a randomized algorithm . . . 37
9.3 The moment generating function . . . 38
9.4 Proof of the Chernoff bound . . . 38

10 Modular Arithmetic **41**
10.1 Congruences and residues . . . 41
10.2 Multiplicative inverses and the gcd . . . 42
10.3 The Chinese remainder theorem . . . 44

11 Basic Group Theory **47**
11.1 Groups . . . 47
11.2 Multiplicative groups modulo n . . . 47
11.3 Subgroups and Lagrange's theorem . . . 48
11.4 Fermat's little theorem and Euler's theorem . . . 49
11.5 Fields and polynomial roots . . . 50

12 Introduction to Linear Programming **52**
12.1 Formulations . . . 52
12.2 Standard and canonical forms . . . 53
12.3 Basic feasible solutions . . . 54
12.4 Duality . . . 55

13 Duality in Linear Programming **57**
13.1 Weak duality and its consequences . . . 57
13.2 Taking the dual of a general linear program . . . 57
13.3 Complementary slackness . . . 58
13.4 A mechanical justification of strong duality . . . 59
13.5 König's theorem . . . 60

14 Zero-Sum Games **62**
14.1 Two-player zero-sum games . . . 62
14.2 Pure and mixed Nash equilibria . . . 62
14.3 The minimax theorem . . . 63
14.4 Solving the 3×3 example . . . 64

15 Max-Flow Min-Cut Theorem **66**
15.1 Flows and cuts . . . 66
15.2 Proof by linear programming duality . . . 67
15.3 Augmenting paths . . . 68

15.4 Bipartite matching . . . 69

16 Data Compression and Shannon's Noiseless Coding Theorem **71**
16.1 Sources and coding functions . . . 71
16.2 Entropy and Shannon's theorem . . . 71
16.3 Coding a uniform distribution . . . 72
16.4 The lower bound . . . 72
16.5 An optimal code . . . 73

17 Huffman Coding **75**
17.1 Prefix codes and binary trees . . . 75
17.2 Huffman's algorithm . . . 76
17.3 Bounding the expected length . . . 78
17.4 Coding blocks of letters . . . 79

18 Transmitting Information Reliably over a Noisy Channel & Shannon's Noisy Coding Theorem **80**
18.1 The binary symmetric channel and repetition codes . . . 80
18.2 Encoding schemes and the capacity . . . 81
18.3 Counting: Hamming balls and entropy . . . 81
18.4 Rates below capacity: random codes . . . 82
18.5 Rates above capacity (sketch) . . . 83

19 Error-Correcting Codes: Hamming Codes **84**
19.1 The $(7, 4)$ Hamming code . . . 84
19.2 Linear codes . . . 85
19.3 Hamming codes . . . 86
19.4 Parity-check matrices and syndrome decoding . . . 86
19.5 Perfect codes . . . 87

20 Reed-Solomon Codes **89**
20.1 Reed–Solomon codes . . . 89
20.2 Decoding by linear algebra . . . 90
20.3 Faster decoding with finite differences . . . 91
20.4 Concatenated codes . . . 93

Lecture 1: Pigeonhole Principle

1.1 The principle

If there are only two colors of socks in a drawer, three socks guarantee a matching pair; with five colors, six socks suffice. The general statement is simple but surprisingly powerful. The work in each application is deciding what the pigeons and the holes are.

Theorem 1.1 [Pigeonhole principle]. *If m objects (pigeons) are placed in n boxes (pigeonholes) and $m > n$, then some box contains more than one object. More strongly, some box contains at least $\lceil m/n \rceil$ objects.*

Proof. Suppose, for contradiction, that every box h contains $N_h \leq 1$ objects. Counting the objects box by box, $m = \sum_h N_h \leq n$, contradicting $m > n$. For the stronger form, if every box had $N_h \leq \lceil m/n \rceil - 1 < m/n$ objects, then $m = \sum_h N_h < n \cdot \frac{m}{n} = m$, a contradiction. □

1.2 Two equal degrees

A graph $G = (V, E)$ consists of a set V of vertices and a set E of pairs of vertices, the edges. The degree of a vertex is the number of edges containing it.

Theorem 1.2. *Every finite graph with at least two vertices has two vertices of equal degree. Equivalently, at a party of $n \geq 2$ people, two people know the same number of other guests.*

Proof. Let G have n vertices; these are the pigeons. Each degree lies in $\{0, 1, \ldots, n-1\}$, which gives n holes, one too many. But a vertex of degree 0 and a vertex of degree $n-1$ cannot coexist: a vertex of degree $n-1$ is adjacent to every other vertex, so no vertex has degree 0. Hence the degrees take values in $\{0, \ldots, n-2\}$ or in $\{1, \ldots, n-1\}$, a set of $n-1$ holes, and two of the n vertices share a degree. □

1.3 Equal-sum subsets

Theorem 1.3. *Any 30 positive integers below 10^7 contain two disjoint nonempty subsets with the same sum.*

Proof. Index the numbers $x_1, \dots, x_{30}$. The pigeons are the $2^{30} - 1 \approx 1.07 \cdot 10^9$ nonempty subsets $A \subseteq \{1, \dots, 30\}$, and each goes into the hole $\sum_{i \in A} x_i$. Every sum is at most $30 \cdot 10^7 = 3 \cdot 10^8$, so there are fewer holes than pigeons and two distinct nonempty subsets $A \neq B$ have equal sums. They need not be disjoint, so set $I = A \setminus B$ and $J = B \setminus A$. Splitting each sum over $A \cap B$,

$$\sum_{i \in A \cap B} x_i + \sum_{i \in I} x_i = \sum_{i \in A} x_i = \sum_{i \in B} x_i = \sum_{i \in A \cap B} x_i + \sum_{i \in J} x_i,$$

so I and J are disjoint with equal sums. Since $A \neq B$, at least one of I, J is nonempty; as the x_i are positive and the sums are equal, both are. □

The same argument gives the version with 20 four-digit numbers: $2^{20} - 1 > 10^6 > 20 \cdot 10^4$. The proof shows the subsets exist, but no known method finds them substantially faster than exhaustive search once the numbers grow, a hardness related to problems in cryptography.

1.4 Monotone subsequences

A SUBSEQUENCE of $a_1, \dots, a_N$ is $a_{i_1}, a_{i_2}, \dots$ with $i_1 < i_2 < \cdots$; the positions need not be contiguous. The sequence $4, 3, 2, 1, 8, 7, 6, 5, 12, 11, 10, 9$ has length 12, longest increasing subsequence of length 3 (for example $4, 8, 11$), and longest decreasing subsequence of length 4.

Theorem 1.4 [ERDŐS–SZEKERES]. *Every sequence of $mn + 1$ distinct real numbers $a_1, \dots, a_{mn+1}$ has an increasing subsequence of length $m + 1$ or a decreasing subsequence of length $n + 1$.*

Proof. For each position s let i_s be the length of the longest increasing subsequence ending at a_s, and d_s the length of the longest decreasing subsequence ending at a_s. The pigeons are the $mn + 1$ positions, and position s goes into the hole (i_s, d_s).
First, distinct positions land in distinct holes. Let $t < s$. If $a_s > a_t$, appending a_s to a longest increasing subsequence ending at a_t gives $i_s \geq i_t + 1$. If $a_s < a_t$, appending a_s to a longest decreasing subsequence ending at a_t gives $d_s \geq d_t + 1$. Either way $(i_s, d_s) \neq (i_t, d_t)$.
Now suppose the theorem fails. Then $i_s \leq m$ and $d_s \leq n$ for all s, so there are at most mn holes for $mn + 1$ pigeons, and two positions share a hole, a contradiction. □

The bound is tight: the concatenation of m decreasing blocks of length n, each block lying entirely above the previous one (as in the example above with $m = 3$, $n = 4$), has mn terms, no increasing subsequence of length $m + 1$, and no decreasing subsequence of length $n + 1$. The numbers i_s are exactly what a dynamic program for the longest increasing subsequence computes.

1.5 Family trees

Proposition 1.5. *Assume that everyone has two biological parents, nobody has children after age* 100*, the human race is at least* 4,000 *years old, and at most* 10^{12} *humans have ever lived. Then everyone's family tree contains a person whose parents are blood relatives.*

Proof. Trace a person's ancestry back 40 generations, which the age bounds allow. The ancestor *roles* (mother's father's mother, and so on) in generation 40 number $2^{40} = (2^{10})^4 > (10^3)^4 = 10^{12}$. These roles are the pigeons and actual humans are the holes, so two roles are filled by the same person X. Follow the two roles down from the original person: the paths agree up to some ancestor Y and then pass through Y's mother and Y's father respectively. Each of these two parents is X or a descendant of X, and they are different people, so they are blood relatives. □

1.6 Twenty questions

In twenty questions one player thinks of an object from an agreed set and the other asks yes/no questions. Questions may depend on previous answers, but the strategy is deterministic. Each object determines a sequence of 20 answers, and two objects with the same answer sequence receive the same questions and cannot be told apart. With objects as pigeons and the 2^{20} answer sequences as holes, at most $2^{20} \approx 10^6$ objects can be identified, and repeated halving achieves this.

Suppose now the answerer may lie at most once. A given object is then compatible with 21 answer sequences: no lie, or a lie at one of the 20 questions, after which the answers are forced. Two of these sequences differ at the earlier possible lie, so each object has 21 distinct sequences. The $21t$ sequences for t objects are the pigeons, and sequences of different objects must also differ, since otherwise the guesser could not decide between them. Hence

$$21t \le 2^{20}, \qquad \text{so} \qquad t \le \left\lfloor \frac{2^{20}}{21} \right\rfloor = 49{,}932.$$

Whether this bound can be achieved is a question about error-correcting codes.

1.7 Sample spaces and events

Probability will be used throughout, so we fix its basic language. We work with discrete probability.

Definition 1.6 [Sample space and events]. A sample space S is a finite or countable set of outcomes, with a probability $p(x) \ge 0$ for each $x \in S$ such that $\sum_{x \in S} p(x) = 1$. An event is a subset $A \subseteq S$, with probability $\mathbb{P}(A) = \sum_{x \in A} p(x)$. The uniform distribution on a finite S has $p(x) = 1/|S|$ for every x.

A fair die has $S = \{1, \dots, 6\}$ with the uniform distribution, and rolling an even number is the event

$A = \{2, 4, 6\}$ with $\mathbb{P}(A) = \frac{1}{2}$. The number of tosses of a coin until the first heads has an infinite sample space. The complement $\neg A = S \setminus A$ satisfies

$$\mathbb{P}(A) + \mathbb{P}(\neg A) = \sum_{x \in A} p(x) + \sum_{x \notin A} p(x) = 1, \qquad \text{so} \qquad \mathbb{P}(\neg A) = 1 - \mathbb{P}(A).$$

Source. Ankur Moitra, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 1, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 2: Independence and Conditioning

2.1 Combining events and conditioning

Events combine as sets. For events $A, B \subseteq S$, the event $A \cup B$ (also written $A \vee B$) is that at least one of them happens, and $A \cap B$ (also $A \wedge B$) is that both happen. Two events split S into four disjoint parts, so

$$\mathbb{P}(A \cap B) + \mathbb{P}(A \cap \neg B) + \mathbb{P}(\neg A \cap B) + \mathbb{P}(\neg A \cap \neg B) = 1.$$

Definition 2.1 [Conditional probability]. If $\mathbb{P}(B) > 0$, the probability of A conditioned on B is

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}.$$

Think of the probability measure as throwing a dart at S. Learning that the dart landed in B restricts the outcomes to B and rescales each of their probabilities by $1/\mathbb{P}(B)$; what matters is the share of B occupied by $A \cap B$. For a fair die, the probability of an odd roll given a roll of at most 3 is $\mathbb{P}(\{1,3\})/\mathbb{P}(\{1,2,3\}) = \frac{2/6}{1/2} = \frac{2}{3}$.

2.2 Independence

Definition 2.2 [Independence]. Events A and B are independent if $\mathbb{P}(A \cap B) = \mathbb{P}(A)\mathbb{P}(B)$. Events $A_1, \ldots, A_k$ are mutually independent if

$$\mathbb{P}(A_{i_1} \cap \cdots \cap A_{i_s}) = \mathbb{P}(A_{i_1}) \cdots \mathbb{P}(A_{i_s}) \qquad \text{for all } 1 \le i_1 < \cdots < i_s \le k,$$

and pairwise independent if every two of them are independent.

When $\mathbb{P}(B) > 0$, dividing by $\mathbb{P}(B)$ shows that independence is equivalent to $\mathbb{P}(A \mid B) = \mathbb{P}(A)$: learning that B happened does not change the probability of A. Pairwise independence does not imply mutual independence.

Independence is sometimes intuitive and sometimes not. Roll a fair die twice, so $S = \{1, \ldots, 6\}^2$ with all 36 outcomes equally likely, and let

$$A = \{x_1 \text{ odd}\}, \quad B = \{x_2 \le 3\}, \quad C = \{x_1 - x_2 = 3\}, \quad D = \{x_1 - x_2 = 4\}.$$

- $A \cap B$ has $3 \cdot 3 = 9$ outcomes, so $\mathbb{P}(A \cap B) = \frac{1}{4} = \mathbb{P}(A)\mathbb{P}(B)$. The rolls are separate experiments, and the events are independent.
- $C = \{(4,1), (5,2), (6,3)\}$ and $A \cap C = \{(5,2)\}$, so $\mathbb{P}(A \cap C) = \frac{1}{36} \neq \frac{1}{2} \cdot \frac{3}{36}$. These are dependent.

- $D = \{(5,1),(6,2)\}$ and $A \cap D = \{(5,1)\}$, so $\mathbb{P}(A \cap D) = \frac{1}{36} = \frac{1}{2} \cdot \frac{2}{36}$. Although D depends on x_1, the events A and D are independent.

Given distributions on $S_1, \ldots, S_k$, the PRODUCT DISTRIBUTION on $S_1 \times \cdots \times S_k$ is $p(s_1, \ldots, s_k) = \prod_{i=1}^{k} p(s_i)$; it models running the k experiments independently. For two fair dice, the probability of rolling one 1 and one 2 is $p(1,2) + p(2,1) = \frac{1}{18}$, while two 1s have probability $\frac{1}{36}$.

2.3 The law of total probability

Theorem 2.3 [LAW OF TOTAL PROBABILITY]. *If $B_1, B_2, \ldots$ are pairwise disjoint events whose union is S, then for every event A*

$$\mathbb{P}(A) = \sum_i \mathbb{P}(A \cap B_i) = \sum_i \mathbb{P}(A \mid B_i)\mathbb{P}(B_i),$$

where terms with $\mathbb{P}(B_i) = 0$ are omitted from the second sum. In particular $\mathbb{P}(A) = \mathbb{P}(A \mid B)\mathbb{P}(B) + \mathbb{P}(A \mid \neg B)\mathbb{P}(\neg B)$.

Proof. Each outcome of A lies in exactly one B_i, so $\sum_{x \in A} p(x) = \sum_i \sum_{x \in A \cap B_i} p(x)$. The second form is the definition of conditional probability. □

This organizes a random process as a tree: the first branching reveals which B_i happened, with probability $\mathbb{P}(B_i)$, and the second reveals whether A happened, with probability $\mathbb{P}(A \mid B_i)$. Multiplying along paths and adding over the paths ending in A gives $\mathbb{P}(A)$.

Example: *Monty Hall.* A prize is behind one of three doors, uniformly at random. You pick a door; the host, who knows where the prize is, opens a different door without the prize and offers a switch. Under the strategy "always switch", the first branch is whether your door hides the prize. With probability $\frac{2}{3}$ it does not, the host must open the only other empty door, and switching wins. With probability $\frac{1}{3}$ it does, and switching loses. So switching wins with probability $\frac{2}{3}$.

2.4 Bayes' rule

Theorem 2.4 [BAYES' RULE]. *If $\mathbb{P}(A), \mathbb{P}(B) > 0$, then*

$$\mathbb{P}(B \mid A) = \mathbb{P}(A \mid B)\,\frac{\mathbb{P}(B)}{\mathbb{P}(A)}.$$

Proof. The right side is $\frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)} \cdot \frac{\mathbb{P}(B)}{\mathbb{P}(A)} = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(A)} = \mathbb{P}(B \mid A)$. □

Example: *Trick coin.* One coin has two heads and the other is fair. A random coin is tossed and shows heads. Label the faces of the trick coin H_1 and H_2; the four equally likely outcomes are H_1, H_2 (trick coin) and H, T (fair coin). Heads leaves three of them, and the other side is tails only for

the fair coin's H, so the probability is $\frac{1}{3}$. Similarly, if a family with two children has at least one girl, both are girls with probability $\frac{1/4}{3/4} = \frac{1}{3}$, ignoring twins.

Example: *Screening tests.* About 1% of women aged 40 to 50 develop breast cancer, and a mammogram has a false positive rate $\mathbb{P}(A \mid \neg B) = 0.1$ and a false negative rate $\mathbb{P}(\neg A \mid B) = 0.1$, where A is a positive test and B is having the disease. By total probability, $\mathbb{P}(A) = 0.9 \cdot 0.01 + 0.1 \cdot 0.99 = 0.108$, and by Bayes' rule

$$\mathbb{P}(B \mid A) = 0.9 \cdot \frac{0.01}{0.108} \approx 0.083.$$

A positive result means only about an 8% chance of disease, far below the 75% that surveyed physicians estimate on average. Because the disease is rare, most positives come from the large healthy population. With incidence $1/1000$, false positive rate $1/30$, and false negative rate $1/10$, the same computation gives $\mathbb{P}(A) \approx 0.0342$ and $\mathbb{P}(B \mid A) \approx 0.026$.

2.5 Random variables and expectation

Definition 2.5 [Random variable and expectation]. A random variable is a function $f : S \to \mathbb{R}$. Its expectation is

$$\mathbb{E}(f) = \sum_{x \in S} p(x) f(x) = \sum_{\alpha \in \text{range}(f)} \alpha \, \mathbb{P}(f = \alpha),$$

when the sum converges absolutely.

The two forms agree because $\mathbb{P}(f = \alpha) = \sum_{x : f(x) = \alpha} p(x)$.

Example: *Tossing until heads.* Toss a fair coin until the first heads. The outcomes are $T^i H$ for $i \geq 0$, with $p(T^i H) = 2^{-(i+1)}$, and the number of tosses N has $\mathbb{E}(N) = \sum_{k \geq 1} k \, 2^{-k} = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \cdots$. Write the term $k 2^{-k}$ as k copies of 2^{-k} and sum the copies in rows instead:

$$\mathbb{E}(N) = \left(\tfrac{1}{2} + \tfrac{1}{4} + \cdots\right) + \left(\tfrac{1}{4} + \tfrac{1}{8} + \cdots\right) + \left(\tfrac{1}{8} + \cdots\right) + \cdots = 1 + \tfrac{1}{2} + \tfrac{1}{4} + \cdots = 2.$$

Proposition 2.6 [Mean time to failure]. *If a machine fails independently each day with probability $p > 0$, the day T of its first failure has $\mathbb{E}(T) = 1/p$.*

Proof. With probability p it fails on day 1. Otherwise one day has passed and, since the process has no memory, the remaining time has the same distribution as T. Hence $\mathbb{E}(T) = p \cdot 1 + (1 - p)(1 + \mathbb{E}(T))$, and $\mathbb{E}(T)$ is finite because $\mathbb{P}(T = k) = (1 - p)^{k-1} p$ decays geometrically. Solving gives $\mathbb{E}(T) = 1/p$. □

The indicator of an event A is the random variable $\mathbf{1}_A(x) = 1$ if $x \in A$ and 0 otherwise. It converts probabilities into expectations:

$$\mathbb{E}(\mathbf{1}_A) = \sum_{x \in S} p(x) \mathbf{1}_A(x) = \sum_{x \in A} p(x) = \mathbb{P}(A).$$

Theorem 2.7 [Linearity of expectation]. *For random variables f, g and $\alpha \in \mathbb{R}$, $\mathbb{E}(f+g) = \mathbb{E}(f) + \mathbb{E}(g)$ and $\mathbb{E}(\alpha f) = \alpha \mathbb{E}(f)$.*

Proof. $\mathbb{E}(f+g) = \sum_x p(x)(f(x)+g(x)) = \sum_x p(x)f(x) + \sum_x p(x)g(x)$, and similarly for αf. □

No independence is needed. However, $\mathbf{1}_A + \mathbf{1}_B \neq \mathbf{1}_{A \cup B}$ unless A and B are disjoint, since the sum equals 2 on $A \cap B$.

2.6 Conditional expectation

Definition 2.8 [Conditional expectation]. If $\mathbb{P}(A) > 0$, the conditional expectation of f given A is the expectation in the conditioned sample space:

$$\mathbb{E}(f \mid A) = \sum_{x \in A} \frac{p(x)}{\mathbb{P}(A)} f(x) = \sum_{\alpha} \alpha \, \mathbb{P}(f = \alpha \mid A) = \frac{\mathbb{E}(f \mathbf{1}_A)}{\mathbb{P}(A)}.$$

Proposition 2.9 [Mossel's dice paradox]. *Roll a fair die until the first 6, and let T be the number of rolls. Conditioned on the event E that every roll was even, $\mathbb{E}(T \mid E) = \frac{3}{2}$.*

The tempting answer 3 argues that, given E, each roll is uniform on $\{2,4,6\}$ and so the waiting time for a 6 is a mean time to failure with $p = \frac{1}{3}$. That is false: conditioning on E strongly favors short sequences, since every extra roll is another chance to see an odd number.

Proof. Let τ be the first roll showing one of $1,3,5,6$. Then E is the event that this roll is a 6, and on E we have $T = \tau$. For every $k \geq 1$,

$$\mathbb{P}(\tau = k, \text{ roll } k \text{ is } 6) = \left(\tfrac{1}{3}\right)^{k-1} \tfrac{1}{6} = \mathbb{P}(\tau = k) \cdot \tfrac{1}{4},$$

since $\mathbb{P}(\tau = k) = (\frac{1}{3})^{k-1}\frac{2}{3}$. So the value of the stopping roll is independent of τ, and $\mathbb{P}(E) = \frac{1}{4}$. Conditioning on E leaves the distribution of τ unchanged, so by Proposition 2.6 with $p = \frac{2}{3}$, $\mathbb{E}(T \mid E) = \mathbb{E}(\tau) = \frac{3}{2}$. □

Source. Ankur Moitra, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 2, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 3: Inclusion-Exclusion

3.1 The matching problem

Graded problem sets for n students are handed back in a uniformly random order. What is the probability that no student receives their own? Let A_i be the event that student i gets their own problem set. We want

$$\mathbb{P}(\neg A_1 \cap \cdots \cap \neg A_n) = 1 - \mathbb{P}(A_1 \cup \cdots \cup A_n).$$

The events are not independent: for $n = 2$, $\mathbb{P}(A_1 \cap A_2) = \frac{1}{2}$, not $\frac{1}{4}$, because once student 1 has the right set, so does student 2.

For two and three events, adding the probabilities overcounts the overlaps, subtracting the pairwise intersections then removes the triple intersection once too often, and adding it back gives

$$\begin{aligned}\mathbb{P}(A \cup B) &= \mathbb{P}(A) + \mathbb{P}(B) - \mathbb{P}(A \cap B),\\ \mathbb{P}(A \cup B \cup C) &= \mathbb{P}(A) + \mathbb{P}(B) + \mathbb{P}(C) - \mathbb{P}(A \cap B) - \mathbb{P}(A \cap C) - \mathbb{P}(B \cap C) + \mathbb{P}(A \cap B \cap C).\end{aligned}$$

For three students each $\mathbb{P}(A_i) = \frac{1}{3}$, and each intersection of two or three events is the single outcome in which everyone is correct, with probability $\frac{1}{6}$. So $\mathbb{P}(A_1 \cup A_2 \cup A_3) = 1 - \frac{1}{2} + \frac{1}{6} = \frac{2}{3}$.

3.2 The inclusion-exclusion formula

Theorem 3.1 [Inclusion-exclusion]. *For events $A_1, \ldots, A_n$,*

$$\mathbb{P}\Big(\bigcup_{i=1}^{n} A_i\Big) = \sum_i \mathbb{P}(A_i) - \sum_{i<j} \mathbb{P}(A_i \cap A_j) + \sum_{i<j<k} \mathbb{P}(A_i \cap A_j \cap A_k) - \cdots + (-1)^{n+1}\mathbb{P}(A_1 \cap \cdots \cap A_n). \quad (3.1)$$

Proof. Induct on n; the case $n = 1$ is trivial and $n = 2$ is the formula above. Assume (3.1) for n events and take $n + 1$. Split the right-hand side for $A_1, \ldots, A_{n+1}$ into three groups: the terms not involving A_{n+1}, the single term $\mathbb{P}(A_{n+1})$, and the terms involving A_{n+1} together with at least one other event.
By induction the first group equals $\mathbb{P}(C)$, where $C = A_1 \cup \cdots \cup A_n$. For the third group, set $B_i = A_i \cap A_{n+1}$. Since $B_i \cap B_j = A_i \cap A_j \cap A_{n+1}$ and so on, the third group is exactly the negative of the right-hand side of (3.1) for $B_1, \ldots, B_n$, so by induction it equals

$$-\mathbb{P}(B_1 \cup \cdots \cup B_n) = -\mathbb{P}(C \cap A_{n+1}).$$

The total is $\mathbb{P}(C) + \mathbb{P}(A_{n+1}) - \mathbb{P}(C \cap A_{n+1}) = \mathbb{P}(C \cup A_{n+1})$ by the case $n = 2$. □

Counting the terms requires the BINOMIAL COEFFICIENT

$$\binom{n}{r} = \frac{n(n-1)\cdots(n-r+1)}{r!},$$

the number of r-element subsets of an n-element set: there are $n(n-1)\cdots(n-r+1)$ ordered choices of r distinct elements, and each subset arises in $r!$ orders.

Theorem 3.2. *The probability that none of the n students receives their own problem set is*

$$\sum_{k=0}^{n} \frac{(-1)^k}{k!} = 1 - 1 + \frac{1}{2!} - \frac{1}{3!} + \cdots + \frac{(-1)^n}{n!} \longrightarrow \frac{1}{e} \qquad (n \to \infty).$$

Proof. Hand back the sets of students $i_1 < \cdots < i_t$ first; each succeeds with probability $\frac{1}{n}, \frac{1}{n-1}, \ldots$ given the earlier successes, so

$$\mathbb{P}(A_{i_1} \cap \cdots \cap A_{i_t}) = \frac{1}{n(n-1)\cdots(n-t+1)}.$$

There are $\binom{n}{t}$ such terms, so the t-th sum in (3.1) is $(-1)^{t+1}\binom{n}{t}\frac{1}{n(n-1)\cdots(n-t+1)} = \frac{(-1)^{t+1}}{t!}$. Hence $\mathbb{P}(\bigcup_i A_i) = \sum_{t=1}^{n}(-1)^{t+1}/t!$, and subtracting from 1 gives the claim. The limit is the Taylor series of e^x at $x = -1$. □

A heuristic explains the answer. For large n the events are nearly independent, since one student's luck barely changes the remaining pool, and the significant terms in (3.1) come from small intersections. Pretending independence gives $(1 - \frac{1}{n})^n \to \frac{1}{e}$. Inclusion-exclusion makes this kind of near-independence precise.

3.3 Linearity of expectation in action

The expected number of students who get their own problem set is much easier. The count is $\sum_{i=1}^{n} \mathbf{1}_{A_i}$, so by linearity

$$\mathbb{E}\Big(\sum_{i=1}^{n} \mathbf{1}_{A_i}\Big) = \sum_{i=1}^{n} \mathbb{P}(A_i) = n \cdot \frac{1}{n} = 1,$$

with no need to understand the dependence between the events.

Proposition 3.3 [COUPON COLLECTOR]. *Draw independently and uniformly from n types until every type*

has appeared. The expected number of draws is

$$\mathbb{E}(T) = \frac{n}{n} + \frac{n}{n-1} + \cdots + \frac{n}{1} = nH_n, \qquad H_n = \sum_{k=1}^{n} \frac{1}{k} = \ln n + O(1).$$

Proof. Write $T = T_0 + T_1 + \cdots + T_{n-1}$, where T_i is the number of draws after the i-th distinct type appears, up to and including the draw of the $(i+1)$-st. With i types seen, each draw is new with probability $\frac{n-i}{n}$, independently, so T_i is a waiting time with $\mathbb{E}(T_i) = \frac{n}{n-i}$ by the mean time to failure. Linearity gives the sum. □

Asking students their birthdays until all 365 days appear (ignoring leap years) takes $365H_{365} \approx 6.48 \cdot 365 \approx 2365$ students on average. In general $H_n = \ln n + \gamma + o(1)$ with $\gamma \approx 0.577$, so the answer is about $n \ln n$.

3.4 Independent random variables

Definition 3.4 [Independent random variables]. Random variables f and g are INDEPENDENT if the events $\{f = \alpha\}$ and $\{g = \beta\}$ are independent for every α and β.

Indicators $\mathbf{1}_A$ and $\mathbf{1}_B$ are independent exactly when A and B are, so this extends independence of events.

Theorem 3.5. *If f and g are independent, then $\mathbb{E}(fg) = \mathbb{E}(f)\mathbb{E}(g)$.*

Proof. Group the outcomes by the values of f and g:

$$\begin{aligned}\mathbb{E}(fg) &= \sum_x p(x) f(x) g(x) = \sum_{\alpha,\beta} \alpha\beta\, \mathbb{P}(f = \alpha \wedge g = \beta) \\ &= \sum_{\alpha,\beta} \alpha\beta\, \mathbb{P}(f = \alpha)\mathbb{P}(g = \beta) = \Big(\sum_\alpha \alpha \mathbb{P}(f = \alpha)\Big)\Big(\sum_\beta \beta \mathbb{P}(g = \beta)\Big) = \mathbb{E}(f)\mathbb{E}(g).\end{aligned}$$ □

For indicators the theorem says $\mathbb{P}(A \cap B) = \mathbb{P}(A)\mathbb{P}(B)$, since $\mathbf{1}_A \mathbf{1}_B = \mathbf{1}_{A \cap B}$. Independence is essential: if $0 < \mathbb{P}(A) < 1$, then $\mathbf{1}_A \mathbf{1}_{\neg A} = 0$ while $\mathbb{E}(\mathbf{1}_A)\mathbb{E}(\mathbf{1}_{\neg A}) = \mathbb{P}(A)(1 - \mathbb{P}(A)) > 0$. The converse fails; $\mathbb{E}(fg) = \mathbb{E}(f)\mathbb{E}(g)$ can hold for dependent f, g.

3.5 Variance

Definition 3.6 [Variance]. The VARIANCE of f is $\mathrm{Var}(f) = \mathbb{E}\big[(f - \mathbb{E}(f))^2\big]$, and its STANDARD DEVIATION is $\sigma = \sqrt{\mathrm{Var}(f)}$.

The variance measures how spread out f is around its mean. It is a weighted average of squares, so $\operatorname{Var}(f) \geq 0$, with equality exactly when f is constant on the outcomes of positive probability. Expanding the square and using linearity, with $\mathbb{E}(f)$ a constant,

$$\operatorname{Var}(f) = \mathbb{E}(f^2) - 2\mathbb{E}(f)\mathbb{E}(f) + \mathbb{E}(f)^2 = \mathbb{E}(f^2) - \mathbb{E}(f)^2.$$

Also $\operatorname{Var}(cf) = c^2 \operatorname{Var}(f)$ for constants c.

Example: For a fair die roll X, $\mathbb{E}(X) = \frac{21}{6} = \frac{7}{2}$ and $\mathbb{E}(X^2) = \frac{91}{6}$, so $\operatorname{Var}(X) = \frac{91}{6} - \frac{49}{4} = \frac{35}{12}$ and $\sigma \approx 1.708$. The mean absolute deviation $\mathbb{E}|X - \frac{7}{2}| = \frac{3}{2}$ is slightly smaller. In general $\mathbb{E}|f - \mathbb{E}(f)| \leq \sigma$, by Cauchy–Schwarz applied to the vectors with entries $\sqrt{p(x)}\,|f(x) - \mathbb{E}(f)|$ and $\sqrt{p(x)}$.

Theorem 3.7. *For random variables f, g,*

$$\operatorname{Var}(f + g) = \operatorname{Var}(f) + \operatorname{Var}(g) + 2\operatorname{Cov}(f, g), \qquad \operatorname{Cov}(f, g) = \mathbb{E}(fg) - \mathbb{E}(f)\mathbb{E}(g).$$

If f and g are independent, then $\operatorname{Var}(f + g) = \operatorname{Var}(f) + \operatorname{Var}(g)$. More generally, if $f_1, \ldots, f_k$ are pairwise independent, then $\operatorname{Var}(f_1 + \cdots + f_k) = \sum_i \operatorname{Var}(f_i)$.

Proof. Write $\bar{f} = f - \mathbb{E}(f)$ and $\bar{g} = g - \mathbb{E}(g)$, which have mean 0. By linearity $f + g - \mathbb{E}(f + g) = \bar{f} + \bar{g}$, so

$$\operatorname{Var}(f + g) = \mathbb{E}(\bar{f}^2) + \mathbb{E}(\bar{g}^2) + 2\mathbb{E}(\bar{f}\,\bar{g}),$$

and $\mathbb{E}(\bar{f}\,\bar{g}) = \mathbb{E}(fg) - \mathbb{E}(f)\mathbb{E}(g)$. If f, g are independent, so are $\bar{f}, \bar{g}$, and Theorem 3.5 gives $\mathbb{E}(\bar{f}\,\bar{g}) = \mathbb{E}(\bar{f})\mathbb{E}(\bar{g}) = 0$. For k variables, expanding $\mathbb{E}\left[(\bar{f}_1 + \cdots + \bar{f}_k)^2\right]$ gives the variances plus cross terms $2\mathbb{E}(\bar{f}_i \bar{f}_j)$, each zero by pairwise independence. □

Unlike linearity of expectation, additivity of variance needs independence (pairwise suffices). A coin with heads probability p has indicator f with $\operatorname{Var}(f) = p - p^2 = p(1 - p)$, so the number of heads in n independent tosses has variance $np(1 - p)$. Thus the variance of an average of n independent samples is $\frac{1}{n}$ times that of one sample, which is why polling a random sample works and the starting point for the tail bounds to come.

Source. Ankur Moitra, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 3, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 4: Counting

4.1 Binomial and multinomial coefficients

Choosing k of n objects one at a time gives $n(n-1)\cdots(n-k+1)$ ordered selections, and each k-element set arises from $k!$ orders. Hence

$$\binom{n}{k} = \frac{n(n-1)\cdots(n-k+1)}{k!} = \frac{n!}{k!\,(n-k)!}.$$

The second form is symmetric in k and $n-k$, so $\binom{n}{k} = \binom{n}{n-k}$: choosing the k objects to keep is the same as choosing the $n-k$ to discard.

Definition 4.1 [Multinomial coefficient]. For $k_1 + \cdots + k_j = n$, the multinomial coefficient

$$\binom{n}{k_1, k_2, \ldots, k_j} = \frac{n!}{k_1!\,k_2!\cdots k_j!}$$

counts the ways to color n distinct objects so that exactly k_i receive color i.

To see this, choose the k_1 objects of color 1, then color the rest:

$$\binom{n}{k_1, \ldots, k_j} = \binom{n}{k_1}\binom{n-k_1}{k_2, \ldots, k_j} = \frac{n!}{k_1!\,(n-k_1)!} \cdot \frac{(n-k_1)!}{k_2!\cdots k_j!},$$

and induct on j. Alternatively, condition on the color of the last object, as in Pascal's identity below. The binomial coefficient is the case $j = 2$, with $\binom{n}{k,n-k} = \binom{n}{k}$.

4.2 Counting in two ways

Counting one set in two ways yields an identity.

Proposition 4.2 [Pascal's identity]. *For* $1 \le k \le n$, $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$.

Proof. Count k-subsets of $\{1, \ldots, n\}$ by whether they contain n. Those containing n are completed by $k-1$ of the other $n-1$ elements, and those avoiding n choose all k elements from the other $n-1$. □

Pascal's identity generates Pascal's triangle, each entry being the sum of the two above it; each row is a palindrome, reflecting $\binom{n}{k} = \binom{n}{n-k}$. Similarly, $\sum_{k=0}^{n}\binom{n}{k} = 2^n$: both sides count the subsets of an

n-element set, the right side by deciding for each element whether it is included. This identifies subsets of $\{1, \ldots, n\}$ with vectors in $\{0,1\}^n$, so a uniformly random subset contains each element independently with probability $\frac{1}{2}$.

4.3 Bijections

Definition 4.3 [Bijection]. A map $f : A \to B$ is injective if $f(a_1) = f(a_2)$ implies $a_1 = a_2$, surjective if every $b \in B$ equals $f(a)$ for some a, and a bijection if it is both.

If there is a bijection $A \to B$, then $|A| = |B|$, so a hard set can be counted through an easier one. The usual way to verify a bijection is to exhibit an inverse.

Proposition 4.4. *If $g : B \to A$ satisfies $g(f(a)) = a$ for all $a \in A$ and $f(g(b)) = b$ for all $b \in B$, then f is a bijection.*

Proof. If $f(a_1) = f(a_2)$, then $a_1 = g(f(a_1)) = g(f(a_2)) = a_2$, so f is injective. Every $b \in B$ equals $f(g(b))$, so f is surjective. □

The converse also holds: a bijection has an inverse.

Theorem 4.5 [Balls and dividers]. *The number of ways to color n identical balls with j colors, where only the number of balls of each color matters, is $\binom{n+j-1}{j-1}$.*

Proof. List the balls by color in a fixed order of the colors and insert a divider between consecutive colors, for example RRRR | G | BB for $n = 7$, $j = 3$. Forgetting the colors leaves a row of $n + j - 1$ symbols, $j - 1$ of which are dividers. The map is invertible: color everything before the first divider with the first color, everything between the first and second dividers with the second, and so on, with empty stretches for unused colors. So the colorings are in bijection with the choices of $j - 1$ divider positions among $n + j - 1$. □

Example: For $n = 4$ balls and $j = 3$ colors: all one color (3 ways), a $3 + 1$ split ($3 \cdot 2 = 6$), a $2 + 2$ split (3, by choosing the unused color), and a $2 + 1 + 1$ split (3, by choosing the doubled color). The total $15 = \binom{6}{2}$ agrees with the theorem.

Dividing 30 distinct objects into three *unlabeled* groups of 10 gives $\binom{30}{10,10,10}/3!$ ways, since each division arises from 3! colorings. In general, splitting $n = \sum_i k_i n_i$ objects into k_i unlabeled groups of size n_i, for distinct sizes n_i, can be done in

$$\frac{1}{\prod_i k_i!} \cdot \frac{n!}{\prod_i (n_i!)^{k_i}}$$

ways. If the objects are identical as well, the count is the number of partitions of n; for example 5

has 7 partitions $(5, 4+1, 3+2, 3+1+1, 2+2+1, 2+1+1+1, 1+1+1+1+1)$. There is no simple closed formula for it.

4.4 Catalan families

Definition 4.6 [Dyck paths and Catalan numbers]. A Dyck path of length $2n$ is a sequence of $2n$ steps $+1$ (up) and -1 (down) from height 0 to height 0 that never goes below 0. Equivalently, it is a lattice path from $(0,0)$ to $(2n,0)$ with steps $(1,\pm 1)$ staying on or above the x-axis. Let D_n be the set of them. The Catalan number is $C_n = |D_n|$.

A Dyck path begins with an up step and ends with a down step, and its reversal is again a Dyck path. The sequence begins $C_1 = 1$, $C_2 = 2$, $C_3 = 5$, $C_4 = 14$, $C_5 = 42$, $C_6 = 132$, $C_7 = 429$; Figure 1 shows D_3. Catalan numbers count hundreds of families of objects. Two more:

- A plane tree is a rooted tree in which the left-to-right order of the children matters, so mirror images are different. Let T_n be the set of plane trees with n edges, hence $n+1$ vertices.
- A binary tree is a plane tree in which every vertex has 0 children (a leaf) or 2 children (an internal node). Let B_n be the set of binary trees with n nodes. A binary tree with n nodes has $n+1$ leaves: this holds for the single node with two leaves, and replacing a leaf by a node with two leaves adds one of each.

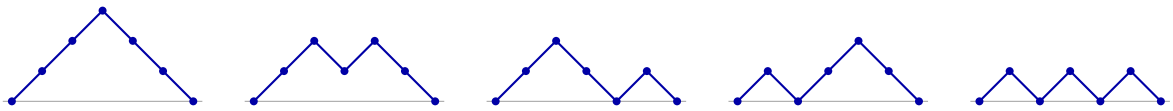


Figure 1: The $C_3 = 5$ Dyck paths of length 6.

Theorem 4.7. *For all* $n \geq 1$, $|T_n| = |B_n| = |D_n| = C_n = \dfrac{1}{n+1}\binom{2n}{n}$.

Proof that $|D_n| = \frac{1}{n+1}\binom{2n}{n}$. Let $P_n^{(0)}$ be the set of all ± 1 paths of length $2n$ from height 0 to height 0. Such a path has n up and n down steps in any order, so $|P_n^{(0)}| = \binom{2n}{n}$. Let $\overline{D}_n = P_n^{(0)} \setminus D_n$ be the paths that go below 0, and $P_n^{(-2)}$ the paths of length $2n$ from 0 to -2, which have $n-1$ up and $n+1$ down steps, so $|P_n^{(-2)}| = \binom{2n}{n-1}$.
For $D \in \overline{D}_n$, let t be the first time D reaches -1, and let $f(D)$ be D with every step after time t reflected in the line $y = -1$ (Figure 2). Reflection sends height h to $-2-h$, so the final height 0 becomes -2 and $f(D) \in P_n^{(-2)}$. Conversely, every path in $P_n^{(-2)}$ reaches -1, and reflecting it after its first visit to -1 gives a path ending at 0 that goes below 0. The first visit to -1 is the same before and after reflection, so this map g satisfies $g \circ f = \mathrm{id}$ and $f \circ g = \mathrm{id}$, and f is a bijection

by Proposition 4.4. Therefore

$$|D_n| = \binom{2n}{n} - \binom{2n}{n-1} = \frac{(2n)!}{n!\,n!} - \frac{(2n)!}{(n+1)!\,(n-1)!} = \binom{2n}{n}\left(1 - \frac{n}{n+1}\right) = \frac{1}{n+1}\binom{2n}{n}. \qquad \square$$

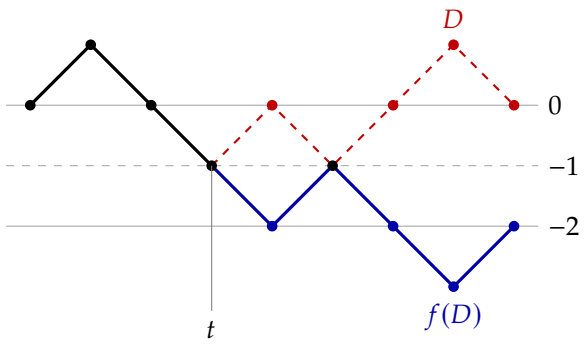


Figure 2: The reflection bijection for $n = 4$. The path D (black, then red dashed) first reaches -1 at time $t = 3$; reflecting the rest in $y = -1$ gives $f(D)$ (black, then blue), which ends at -2.

For $n = 3$, $\frac{1}{4}\binom{6}{3} = 5$. The formula has a probabilistic reading, the BALLOT PROBLEM: if an election between Alice and Bob ends in an n to n tie and the votes are counted in uniformly random order, Alice is never behind with probability $C_n / \binom{2n}{n} = \frac{1}{n+1}$; with 200 voters this is $\frac{1}{101}$.

Proof that $|B_n| = |T_n| = |D_n|$. Traverse a binary tree in depth-first order, visiting the left subtree before the right, and record $+1$ the first time a node is visited and -1 at each leaf. This gives n up steps and $n + 1$ down steps, and the last step is a leaf; dropping it leaves a path $\Psi(B)$ with n up and n down steps (Figure 3). Track the unfilled child positions: there is 1 at the start, each node fills one and opens two, and each leaf fills one. After u nodes and ℓ leaves there are $1 + u - \ell$ open positions, which is 1 plus the current height. At least one stays open until the final leaf, so the height never drops below 0. Conversely, a Dyck path rebuilds the tree in the same order, filling the next open position with a node for each $+1$ and a leaf for each -1. One position is open throughout, and one remains at the end for the final leaf. These maps are inverse, so $|B_n| = |D_n|$.

For plane trees, walk around the tree from the root in depth-first order and record $+1$ when descending an edge and -1 when returning along it. Each of the n edges contributes one of each, the height is the current depth and so never negative, and the walk can be reversed. This bijection Φ gives $|T_n| = |D_n|$. $\square$

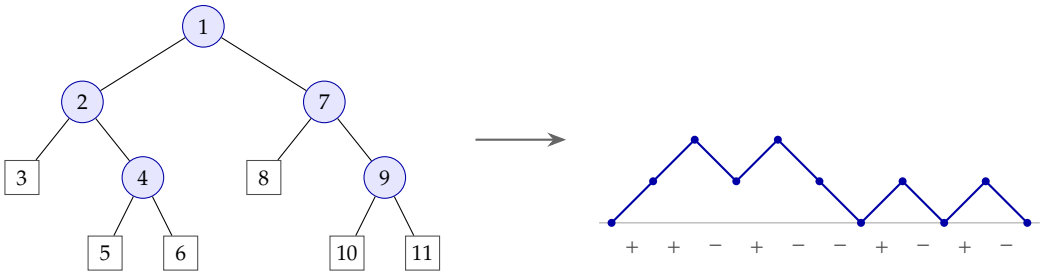


Figure 3: A binary tree with 5 nodes (circles) and 6 leaves (squares), numbered in depth-first order, and its Dyck path $\Psi(B)$. The signs $+, +, -, +, -, -, +, -, +, -$ record nodes and leaves; the final leaf 11 is dropped.

4.5 Stirling's formula

Theorem 4.8 [Stirling's formula]. *$n! \sim \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$, where $a_n \sim b_n$ means $a_n/b_n \to 1$. In particular $\ln(n!) = n\ln n - n + O(\log n)$.*

The cruder bounds $(n/2)^{n/2} \le n! \le n^n$ already show that $\log(n!)$ grows like $n\log n$. The only delicate part of Stirling's formula is the constant $\sqrt{2\pi}$. Applying it to the middle binomial coefficient,

$$\binom{2n}{n} = \frac{(2n)!}{n!\,n!} \sim \frac{\sqrt{4\pi n}\,(2n/e)^{2n}}{2\pi n\,(n/e)^{2n}} = \frac{4^n}{\sqrt{\pi n}}, \qquad \text{so} \qquad C_n \sim \frac{4^n}{\sqrt{\pi}\,n^{3/2}}.$$

Thus $2n$ fair coin tosses give exactly n heads with probability $\binom{2n}{n}4^{-n} \sim 1/\sqrt{\pi n}$, consistent with the fraction of heads concentrating in a window of width of order $1/\sqrt{n}$ around $\frac{1}{2}$. Without Stirling, induction on n gives

$$\frac{4^n}{\sqrt{4n}} \le \binom{2n}{n} \le \frac{4^n}{\sqrt{3n+1}},$$

with equality at $n = 1$. The step uses $\binom{2n+2}{n+1} = \frac{2(2n+1)}{n+1}\binom{2n}{n}$ and reduces to $4n(n+1) \le (2n+1)^2$ and $(2n+1)^2(3n+4) \le 4(n+1)^2(3n+1)$.

Source. Peter Shor, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 4, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 5: More Counting and Generating Functions

5.1 Cayley's formula

A labeled tree on n vertices is a tree whose vertices are labeled $1, \ldots, n$; two labeled trees are the same when they have the same edge set. Let T_n be the number of labeled trees on n vertices. Grouping by the shape of the tree gives the first values:

n	Shapes	T_n
2	one edge	1
3	path (3 choices of middle vertex)	3
4	path ($\binom{4}{2} \cdot 2 = 12$), star (4)	16
5	path ($5!/2 = 60$), spider with legs $2, 1, 1$ ($5!/2 = 60$), star (5)	125

The pattern $1, 3, 4^2, 5^3$ suggests the formula.

Theorem 5.1 [Cayley's formula]. *For $n \geq 2$, the number of labeled trees on n vertices is $T_n = n^{n-2}$.*

A bijection between labeled trees and sequences in $\{1, \ldots, n\}^{n-2}$ exists but is not transparent. Instead we follow Jim Pitman's proof, which counts one family of objects in two ways. A rooted tree is a tree with a distinguished vertex, the root, with each edge joining a child to its parent, the endpoint nearer the root; a rooted forest is a collection of vertex-disjoint rooted trees.

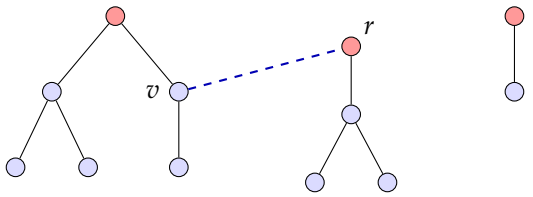


Figure 4: One step of the construction in a rooted forest of three trees (roots in red). The root r of one tree becomes a child of a vertex v in a different tree via the dashed edge.

Proof. Count labeled rooted trees on $\{1, \ldots, n\}$ whose $n-1$ edges are ordered (labeled first, second, and so on).
First count. Choose a labeled tree (T_n ways), a root (n ways), and an order of the edges ($(n-1)!$ ways), for a total of $T_n \cdot n \cdot (n-1)!$.
Second count. Build the tree one edge at a time. Start from the rooted forest of n one-vertex trees. At step $k = 0, 1, \ldots, n-2$ the forest has $n-k$ trees. Choose any vertex v (n ways) and any tree not containing v ($n-k-1$ ways), and make the root of that tree a child of v (Figure 4);

the new edge is the $(k+1)$st. The new edge must join two different trees, or it would close a cycle, and it merges exactly two of them, so the number of trees drops by one and after $n-1$ steps we have a single labeled rooted tree with ordered edges.
Every such tree arises from exactly one sequence of choices. Deleting the last edge of a forest built this way splits one tree into the part containing its root and the part hanging below the edge; rooting the latter at the lower endpoint recovers the previous forest, the vertex v, and the attached tree. Repeating this from any labeled rooted tree with ordered edges undoes the construction. Hence the number of these trees is

$$\prod_{k=0}^{n-2} n(n-k-1) = n^{n-1}(n-1)!.$$

Equating the two counts, $T_n \cdot n \cdot (n-1)! = n^{n-1}(n-1)!$, so $T_n = n^{n-2}$. □

5.2 Generating functions

Definition 5.2 [Generating function]. The generating function of a sequence $(a_n)_{n\geq 0}$ is the formal power series

$$A(x) = \sum_{n\geq 0} a_n x^n.$$

If $\mathcal{A}$ is a class of objects, each with a nonnegative integer size, and a_n is the number of objects of size n, then $A(x)$ is the generating function of $\mathcal{A}$.

The variable x stands for nothing; it only keeps the coefficients a_n apart. Generating functions are useful because natural operations on classes become algebra on series, and the series is often much simpler than the sequence.

Example:

- Binary sequences, with size the length: $a_n = 2^n$ and $A(x) = \sum_{n\geq 0} 2^n x^n$.
- Subsets of $\{1, \ldots, k\}$, with size the number of elements: $a_n = \binom{k}{n}$, and by the binomial theorem

$$A(x) = \sum_{n=0}^{k} \binom{k}{n} x^n = (1+x)^k.$$

  Expanding $(1+x)(1+x)\cdots(1+x)$, the coefficient of x^n counts the ways to take x from n of the k factors and 1 from the rest.

- Toss a coin with $\mathbb{P}(\text{heads}) = p$ and $q = 1-p$ a total of k times, and let a_n be the probability of

exactly n heads. Then $a_n = \binom{k}{n} p^n q^{k-n}$ and

$$A(x) = \sum_{n=0}^{k} \binom{k}{n} p^n q^{k-n} x^n = (q + px)^k,$$

the kth power of the generating function $q + px$ of a single toss.

5.3 Sums and products

Proposition 5.3. *If $C = \mathcal{A} \sqcup \mathcal{B}$ is a disjoint union, then $C(x) = A(x) + B(x)$.*

Proof. An object of size n in C lies in exactly one of $\mathcal{A}$ and $\mathcal{B}$, so $c_n = a_n + b_n$. □

Products are where generating functions become powerful. Roll a six-sided die and an eight-sided die, numbered from 1, and let c_n be the number of outcomes with sum n. Then

$$\begin{aligned} C(x) &= (x + x^2 + \cdots + x^6)(x + x^2 + \cdots + x^8) \\ &= x^2 + 2x^3 + 3x^4 + 4x^5 + 5x^6 + 6x^7 + 6x^8 + 6x^9 + 5x^{10} + 4x^{11} + 3x^{12} + 2x^{13} + x^{14}. \end{aligned}$$

Each way of reaching x^n picks x^j from the first factor and x^{n-j} from the second, that is, a roll of j and a roll of $n - j$. For instance, the five ways to roll 10 are $(2, 8), (3, 7), (4, 6), (5, 5), (6, 4)$.

Theorem 5.4 [Product rule]. *Let $\mathcal{A}$ and $\mathcal{B}$ be classes with generating functions $A(x)$ and $B(x)$, and give the pair $(a, b) \in \mathcal{A} \times \mathcal{B}$ the size $|a| + |b|$. Then $C = \mathcal{A} \times \mathcal{B}$ has generating function $C(x) = A(x)B(x)$.*

Proof. A pair of size n consists of an object of size k in $\mathcal{A}$ (a_k choices) and one of size $n - k$ in $\mathcal{B}$ (b_{n-k} choices), for some $0 \le k \le n$. Hence $c_n = \sum_{k=0}^{n} a_k b_{n-k}$. This is exactly the coefficient of x^n in $\left(\sum_k a_k x^k\right)\left(\sum_j b_j x^j\right)$, which collects the products $a_k x^k \cdot b_j x^j$ with $k + j = n$. □

The additivity of size is essential. By induction, the class $\mathcal{A}^k$ of k-tuples has generating function $A(x)^k$; for example, the sum of four six-sided dice has generating function $(x + x^2 + \cdots + x^6)^4$.

5.4 Making change

With 6 pennies, 1 nickel, and 2 dimes, let g_n be the number of ways to pay exactly n cents. A payment is a triple (pennies, nickels, dimes) and its size is its value, so by Theorem 5.4

$$G(x) = (1 + x + x^2 + \cdots + x^6)(1 + x^5)(1 + x^{10} + x^{20}).$$

For example, x^{21} arises only as $x^6 \cdot x^5 \cdot x^{10}$ and as $x \cdot 1 \cdot x^{20}$, so there are two ways to pay 21 cents.

With unlimited pennies, nickels, and dimes, each factor becomes a geometric series:

$$G(x) = \Big(\sum_{k\geq 0} x^k\Big)\Big(\sum_{k\geq 0} x^{5k}\Big)\Big(\sum_{k\geq 0} x^{10k}\Big) = \frac{1}{(1-x)(1-x^5)(1-x^{10})}.$$

5.5 Differentiation

Differentiating a generating function and multiplying by x multiplies the kth coefficient by k:

$$x\frac{d}{dx}\sum_k a_k x^k = \sum_k k a_k x^k.$$

Proposition 5.5. *For $n \geq 1$, $\displaystyle\sum_{k=0}^{n} k\binom{n}{k} = n2^{n-1}$.*

Proof. Generating functions. Let $F(x) = \sum_{k=0}^{n}\binom{n}{k}x^k = (1+x)^n$. Then

$$xF'(x) = \sum_{k=0}^{n} k\binom{n}{k}x^k = nx(1+x)^{n-1},$$

and setting $x = 1$ (both sides are polynomials) gives the identity.
Counting in two ways. Count pairs (S, s) with $s \in S \subseteq \{1, \ldots, n\}$. Choosing S of size k and then $s \in S$ gives $\sum_k k\binom{n}{k}$. Choosing s first (n ways) and then any subset of the other $n-1$ elements to complete S (2^{n-1} ways) gives $n2^{n-1}$. □

Here the counting proof is at least as easy, but generating functions are mechanical, and for problems such as the Fibonacci and Catalan numbers in the next two lectures they are often the most direct route.

Source. Peter Shor, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 5, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 6: More on Generating Functions

6.1 Sequences

Recall that disjoint unions add generating functions and Cartesian products, with additive size, multiply them. One more operation builds sequences.

Definition 6.1 [Sequence class]. For a class $\mathcal{A}$ with no objects of size 0, let $\mathrm{Seq}(\mathcal{A}) = \bigcup_{k\geq 0} \mathcal{A}^k$ be the class of finite sequences of objects of $\mathcal{A}$, where the size of a sequence is the sum of the sizes of its entries. The empty sequence has size 0.

Theorem 6.2 [Sequence rule]. *If $a_0 = 0$, then $\mathrm{Seq}(\mathcal{A})$ has generating function*

$$S(x) = \sum_{k\geq 0} A(x)^k = \frac{1}{1 - A(x)}.$$

Proof. The classes $\mathcal{A}^k$ are disjoint and $\mathcal{A}^k$ has generating function $A(x)^k$ by the product rule, so $S(x) = \sum_k A(x)^k$. Because $a_0 = 0$, every term of $A(x)^k$ has degree at least k, so each coefficient of the sum is a finite sum and $S(x)$ is a well-defined power series. Multiplying out, $(1 - A(x)) \sum_{k\geq 0} A(x)^k = 1$, since all terms except the constant term telescope. □

The hypothesis $a_0 = 0$ is needed: with an object of size 0 we could insert it arbitrarily often, giving infinitely many sequences of each size. For binary strings, $\mathcal{A} = \{0, 1\}$ has $A(x) = 2x$, and $S(x) = 1/(1 - 2x) = \sum_n 2^n x^n$, as expected.

6.2 Dots and dashes

Messages are sequences of dots, taking one unit of time, and dashes, taking two. Let f_n be the number of messages of duration n:

n	0	1	2	3	4	5
f_n	1	1	2	3	5	8

The empty message gives $f_0 = 1$. A message ends in a dot or a dash, and removing it leaves a message of duration $n - 1$ or $n - 2$, so

$$f_n = f_{n-1} + f_{n-2} \qquad (n \geq 2).$$

Note. These are the Fibonacci numbers shifted by one: $f_n = F_{n+1}$ in the common convention $F_0 = 0, F_1 = 1$.

Generating function from the recurrence. Let $F(x) = \sum_{n \geq 0} f_n x^n$. Multiplying by x and x^2 shifts the coefficients:

$$\begin{aligned} F(x) &= 1 + x + 2x^2 + 3x^3 + 5x^4 + \cdots \\ xF(x) &= \quad x + x^2 + 2x^3 + 3x^4 + \cdots \\ x^2F(x) &= \quad\quad x^2 + x^3 + 2x^4 + \cdots \end{aligned}$$

By the recurrence, $F(x) - xF(x) - x^2F(x)$ has every coefficient 0 except the constant term 1, so

$$F(x) = \frac{1}{1 - x - x^2}.$$

The sequence rule gives this at once: a message is a sequence from {dot, dash}, whose generating function is $x + x^2$, so $F(x) = 1/(1 - x - x^2)$.

Closed form. Factor

$$1 - x - x^2 = (1 - \varphi_+ x)(1 - \varphi_- x), \qquad \varphi_\pm = \frac{1 \pm \sqrt{5}}{2}.$$

Substituting $y = 1/x$ and multiplying by y^2 shows that $\varphi_\pm$ are the roots of $y^2 - y - 1$ (the reciprocals of the roots of $1 - x - x^2$). By partial fractions,

$$F(x) = \frac{a}{1 - \varphi_+ x} + \frac{b}{1 - \varphi_- x},$$

where clearing denominators gives $a + b = 1$ and $a\varphi_- + b\varphi_+ = 0$, so $a = \varphi_+/\sqrt{5}$ and $b = -\varphi_-/\sqrt{5}$. Expanding each term as a geometric series, $1/(1 - \alpha x) = \sum_n \alpha^n x^n$, and comparing coefficients of x^n,

$$f_n = \frac{1}{\sqrt{5}} \left[\left(\frac{1 + \sqrt{5}}{2} \right)^{n+1} - \left(\frac{1 - \sqrt{5}}{2} \right)^{n+1} \right].$$

Since $|\varphi_-| < 1$, the second term tends to 0 and $f_n \sim (\varphi_+/\sqrt{5})\, \varphi_+^n$. So a dot-dash channel can send about $\log_2 f_n \approx n \log_2 \varphi_+$ bits in time n, a rate of

$$\log_2 \frac{1 + \sqrt{5}}{2} \approx 0.694 \text{ bits per time unit.}$$

6.3 Diagonals of Pascal's triangle

The shallow diagonals of Pascal's triangle (Figure 5) sum to Fibonacci numbers.

Theorem 6.3. *For every $n \geq 0$,*

$$\sum_{b \geq 0} \binom{n-b}{b} = f_n.$$

In particular, $\sum_{j=0}^{m} \binom{m+j}{2j} = f_{2m}$.

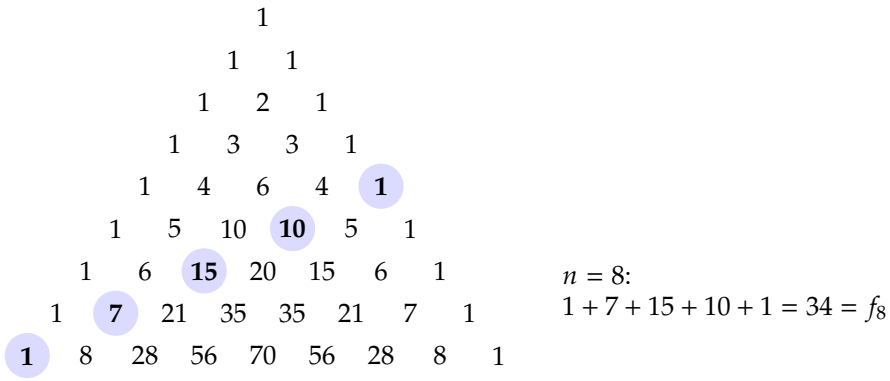


Figure 5: Rows 0 to 8 of Pascal's triangle. The shaded entries $\binom{8-b}{b}$, $b = 0, \ldots, 4$, lie on a shallow diagonal and sum to $f_8 = 34$.

Proof. Generating functions. Encode the whole triangle in two variables: the coefficient of $x^a y^b$ in

$$G(x, y) = \sum_{a,b \geq 0} \binom{a+b}{a} x^a y^b$$

is the entry in row $a + b$, position b. Summing row by row with the binomial theorem,

$$G(x, y) = \sum_{n \geq 0} \sum_{a+b=n} \binom{n}{a} x^a y^b = \sum_{n \geq 0} (x + y)^n = \frac{1}{1 - x - y}.$$

Substituting $y = x^2$ sends $x^a y^b$ to x^{a+2b}, so the coefficient of x^n in $G(x, x^2)$ is $\sum_{a+2b=n} \binom{a+b}{a} = \sum_b \binom{n-b}{b}$. On the other hand $G(x, x^2) = 1/(1 - x - x^2) = F(x)$, whose coefficient of x^n is f_n. The second statement is the case $n = 2m$ with $j = m - b$.

Counting in two ways. A message of duration n with b dashes has $n - 2b$ dots, hence $n - b$ symbols, and the dashes can occupy any b of these positions: $\binom{n-b}{b}$ messages. Summing over b counts all f_n messages. (Induction also works once the answer is known; generating functions found it without a guess.) □

Source. Peter Shor, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 6, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 7: Generating Functions for Catalan Numbers

7.1 A quadratic equation

The Catalan numbers $C_0, C_1, C_2, \ldots = 1, 1, 2, 5, 14, 42, \ldots$ count, among other families, binary trees with n internal nodes (every internal node has exactly two children, so there are $2n+1$ nodes in all) and Dyck paths of length $2n$. Earlier we found $C_n = \frac{1}{n+1}\binom{2n}{n}$ with bijections; here we derive it from a recurrence.

A binary tree with $n \geq 1$ internal nodes is a root with a left subtree having some j internal nodes and a right subtree having $n-1-j$. Hence

$$C_n = \sum_{j=0}^{n-1} C_j C_{n-1-j} \qquad (n \geq 1),$$

for example $C_4 = 5 \cdot 1 + 2 \cdot 1 + 1 \cdot 2 + 1 \cdot 5 = 14$. This recurrence is quadratic, unlike the linear recurrences of the previous lecture, but generating functions still handle it.

Let $C(x) = \sum_{n \geq 0} C_n x^n$. The coefficient of x^{n-1} in $C(x)^2$ is $\sum_{j=0}^{n-1} C_j C_{n-1-j} = C_n$, so $xC(x)^2$ has coefficient C_n at x^n for every $n \geq 1$. Only the constant term is missing, so

$$C(x) = 1 + xC(x)^2.$$

Solving. By the quadratic formula,

$$C(x) = \frac{1 \pm \sqrt{1-4x}}{2x}.$$

We need $C(0) = 1$. With the plus sign the numerator tends to 2 and the expression blows up as $x \to 0$. With the minus sign,

$$\frac{1-\sqrt{1-4x}}{2x} = \frac{2}{1+\sqrt{1-4x}} \to 1,$$

so $C(x) = \left(1 - \sqrt{1-4x}\right)/(2x)$.

Extracting coefficients. For any real α, the binomial theorem extends to

$$(1+u)^\alpha = \sum_{k \geq 0} \binom{\alpha}{k} u^k, \qquad \binom{\alpha}{k} = \frac{\alpha(\alpha-1)\cdots(\alpha-k+1)}{k!}.$$

For $\alpha = \frac{1}{2}$ and $k \geq 1$,

$$\binom{1/2}{k} = \frac{\frac{1}{2}\left(-\frac{1}{2}\right)\left(-\frac{3}{2}\right)\cdots\left(\frac{3}{2}-k\right)}{k!} = (-1)^{k-1}\frac{1\cdot 3\cdots(2k-3)}{2^k k!},$$

so with $u = -4x$ all signs combine to

$$\binom{1/2}{k}(-4x)^k = -\frac{2^k\,(2k-3)!!}{k!}\,x^k,$$

where $(2k-3)!! = 1\cdot 3\cdots(2k-3)$ (empty product 1 when $k = 1$). The constant terms cancel in $1-\sqrt{1-4x}$, and dividing by $2x$ shifts $k = n+1$ to x^n:

$$C_n = \frac{1}{2}\cdot\frac{2^{n+1}(2n-1)!!}{(n+1)!} = \frac{2^n\,(2n-1)!!}{(n+1)!}.$$

Every coefficient is positive, as it must be for a count. Finally, dividing $(2n)!$ by its even factors leaves the odd ones, so

$$(2n-1)!! = \frac{(2n)!}{2\cdot 4\cdots(2n)} = \frac{(2n)!}{2^n n!}, \qquad C_n = \frac{(2n)!}{n!\,(n+1)!} = \frac{1}{n+1}\binom{2n}{n}.$$

For example, $C_3 = 2^3\cdot 15/4! = 5$.

7.2 A second derivation from Dyck paths

A Dyck path is PRIMITIVE if it returns to height 0 only at its endpoints. Cutting a Dyck path at its returns to 0 splits it uniquely into a sequence of primitive paths (Figure 6).

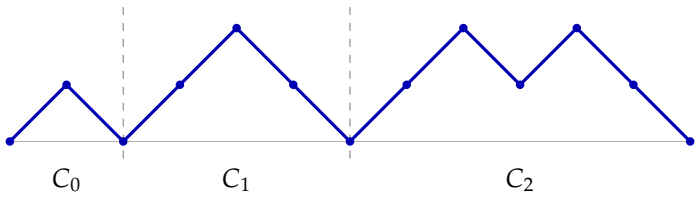


Figure 6: A Dyck path of length 12 cut at its returns to 0 into three primitive paths. Removing the first and last steps of a primitive path of length $2m$ leaves an arbitrary Dyck path of length $2m-2$, counted by C_{m-1} (here $m = 1, 2, 3$).

Measure a path by half its length. A primitive path of size $m \geq 1$ is an up step, a Dyck path of size $m-1$, and a down step, so the primitive paths have generating function $D(x) = xC(x)$, with no objects of size 0. By the sequence rule,

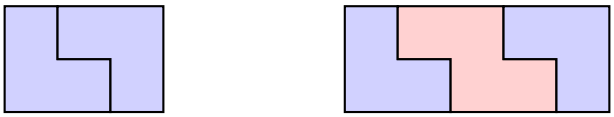
$$C(x) = \frac{1}{1-D(x)} = \frac{1}{1-xC(x)},$$

which rearranges to $C(x) = 1 + xC(x)^2$ again.

7.3 Tiling a strip

Count tilings of a $2 \times n$ strip by L-shaped trominoes and Z-shaped tetrominoes, each of which may be rotated and reflected. Let b_n be the number of tilings, and call a tiling INDECOMPOSABLE if no vertical line cuts the strip into a $2 \times k$ and a $2 \times (n-k)$ strip without crossing a tile; let a_n count these. Every tiling is, uniquely, a sequence of indecomposable tilings, so $B(x) = 1/(1 - A(x))$.

In two rows a Z can only lie horizontally. At the left end of the strip, a Z would cover one cell of the first column and isolate the other, so an L covers the first column and one cell of the second. The cell left open in the second column must then be covered by an L, which leaves a flat edge and ends the indecomposable piece, or by a Z, which leaves the same notch two columns further right. Hence the indecomposable tilings are an L, then $k - 1 \geq 0$ Z's, then an L, of length $2k + 1$, and the orientation of the first L determines the rest (Figure 7).



$n = 3$ $\qquad$ $n = 5$

Figure 7: Indecomposable tilings of lengths 3 and 5, with L trominoes in blue and a Z tetromino in red. Reflecting each in the horizontal midline gives the other indecomposable tiling of the same length.

So $a_{2k+1} = 2$ for $k \geq 1$ and $a_n = 0$ otherwise, and

$$A(x) = 2x^3 \sum_{j \geq 0} x^{2j} = \frac{2x^3}{1 - x^2}, \qquad B(x) = \frac{1}{1 - \frac{2x^3}{1-x^2}} = \frac{1 - x^2}{1 - x^2 - 2x^3}.$$

Multiplying out $B(x)(1 - x^2 - 2x^3) = 1 - x^2$ and comparing coefficients gives $b_0 = 1$, $b_1 = 0$, $b_2 - b_0 = -1$, and

$$b_n = b_{n-2} + 2b_{n-3} \qquad (n \geq 3),$$

so $(b_n)_{n \geq 0} = 1, 0, 0, 2, 0, 2, 4, 2, 8, \ldots$. For instance $b_6 = 4$ (two length-3 pieces) and $b_8 = 8$ (a length-3 and a length-5 piece in either order).

The recurrence also has a direct explanation. If the last indecomposable piece of a tiling of length n has length at least 5, deleting one Z from it leaves a nonempty tiling of length $n - 2$; if it has length 3, it is one of 2 pieces following a tiling of length $n - 3$. Both maps are reversible, and only the empty tiling cannot be extended by a Z, which is why $b_2 = 0$ rather than $b_0 = 1$.

7.4 Rational generating functions and linear recurrences

The two Fibonacci-type examples follow a general pattern, written here with three terms; k terms work the same way. Suppose

$$f_n = \alpha f_{n-1} + \beta f_{n-2} + \gamma f_{n-3} \qquad (n \geq 3).$$

Then $F(x)(1 - \alpha x - \beta x^2 - \gamma x^3) = p(x)$, where p has degree at most 2 and is fixed by f_0, f_1, f_2, so

$$F(x) = \frac{p(x)}{1 - \alpha x - \beta x^2 - \gamma x^3}.$$

Conversely, given a rational generating function, the denominator gives the recurrence and the numerator gives the initial conditions, as in the tiling example.

If $1 - \alpha x - \beta x^2 - \gamma x^3 = (1 - r_1 x)(1 - r_2 x)(1 - r_3 x)$ with distinct r_i, partial fractions and geometric series give

$$F(x) = \sum_{i=1}^{3} \frac{a_i}{1 - r_i x}, \qquad f_n = a_1 r_1^n + a_2 r_2^n + a_3 r_3^n.$$

Substituting $y = 1/x$ shows that the r_i are the roots of $y^3 - \alpha y^2 - \beta y - \gamma$. The constants a_i are easiest to find by setting $n = 0, 1, 2$ in this formula and solving the resulting linear system with the known f_0, f_1, f_2. A root r of multiplicity m contributes terms $n^j r^n$ for $0 \leq j < m$ instead.

Source. Peter Shor, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 7, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 8: Tail Bounds

A tail bound bounds the probability that a random variable is far from its expectation. The more we know about the random variable, the tighter the bound: Markov's inequality uses only the mean, Chebyshev's inequality also uses the variance, and the Chernoff bound of the next lecture uses mutual independence.

8.1 Markov's inequality

Theorem 8.1 [Markov's inequality]. *Let X be a nonnegative random variable. Then for every $c > 0$,*

$$\mathbb{P}(X \geq c) \leq \frac{\mathbb{E}[X]}{c}.$$

Proof. Let $A = \{X \geq c\}$ and write $p(s)$ for the probability of the outcome s in the sample space S. Then

$$\mathbb{E}[X] = \sum_{s \in A} p(s)X(s) + \sum_{s \notin A} p(s)X(s) \geq \sum_{s \in A} p(s)X(s) \geq c\,\mathbb{P}(A).$$

The first inequality drops a sum that is nonnegative because $X \geq 0$. □

Equivalently, conditioning on A, $\mathbb{E}[X] \geq \mathbb{E}[X \mid A]\mathbb{P}(A) \geq c\mathbb{P}(A)$. The intuition is that if $X \geq c$ were too likely, these values alone would push the expectation above $\mathbb{E}[X]$, and nonnegativity means nothing can cancel them. Setting $c = t\mathbb{E}[X]$ gives the form

$$\mathbb{P}\big(X \geq t\,\mathbb{E}[X]\big) \leq \frac{1}{t} \qquad (t > 0);$$

for instance, a nonnegative random variable is at least twice its mean with probability at most $\frac{1}{2}$.

Example:

- *Nonnegativity is essential.* If $X = \pm 1$ with probability $\frac{1}{2}$ each, then $\mathbb{E}[X] = 0$, yet $\mathbb{P}(X \geq 3\mathbb{E}[X]) = \mathbb{P}(X \geq 0) = \frac{1}{2}$.
- *Markov is weak with little information.* The average height of an adult US male is about 69 inches, so Markov's inequality says that the probability of a height of at least 138 inches (11 feet 6 inches) is at most $\frac{1}{2}$.

8.2 Chebyshev's inequality

Theorem 8.2 [Chebyshev's inequality]. *Let X be a random variable with finite variance. Then for every $c > 0$,*

$$\mathbb{P}\big(|X - \mathbb{E}[X]| \geq c\big) \leq \frac{\mathrm{Var}(X)}{c^2}.$$

Proof. The events $|X - \mathbb{E}[X]| \geq c$ and $(X - \mathbb{E}[X])^2 \geq c^2$ coincide. Apply Markov's inequality to the nonnegative random variable $Y = (X - \mathbb{E}[X])^2$ with threshold c^2:

$$\mathbb{P}\big(|X - \mathbb{E}[X]| \geq c\big) = \mathbb{P}(Y \geq c^2) \leq \frac{\mathbb{E}[Y]}{c^2} = \frac{\mathrm{Var}(X)}{c^2}. \qquad \square$$

No sign assumption on X is needed; the proof manufactures the nonnegative variable Y. With the standard deviation $\sigma(X) = \sqrt{\mathrm{Var}(X)}$ and $c = t\sigma(X)$,

$$\mathbb{P}\big(|X - \mathbb{E}[X]| \geq t\,\sigma(X)\big) \leq \frac{1}{t^2},$$

so a random variable is two standard deviations from its mean with probability at most $\frac{1}{4}$. For heights with standard deviation 2.7 inches, the probability of a height of at least $69 + 2 \cdot 2.7 = 74.4$ inches is at most $\frac{1}{4}$: still crude, but no longer absurd.

Note. Both inequalities are tight in general. If $X = c$ with probability p and $X = 0$ otherwise, then $\mathbb{P}(X \geq c) = p = \mathbb{E}[X]/c$. Applying this to $Y = (X - \mathbb{E}[X])^2$ shows that equality in Chebyshev's inequality holds when X equals $\mathbb{E}[X] \pm c$ with probability $p/2$ each and $\mathbb{E}[X]$ otherwise.

8.3 The weak law of large numbers

Theorem 8.3 [Weak law of large numbers]. *Let $X_1, \ldots, X_n$ be independent copies of a random variable X with $\mathbb{E}[X] = \mu$ and $\mathrm{Var}(X) < \infty$, and let $S_n = \frac{1}{n}(X_1 + \cdots + X_n)$. Then for every fixed $\epsilon > 0$,*

$$\mathbb{P}\big(|S_n - \mu| \geq \epsilon\big) \to 0 \qquad \text{as } n \to \infty.$$

Proof. By linearity of expectation, $\mathbb{E}[S_n] = \frac{1}{n}\sum_i \mathbb{E}[X_i] = \mu$. Scaling by $\frac{1}{n}$ scales the variance by $\frac{1}{n^2}$, and variances of independent random variables add (Lecture 3), so

$$\mathrm{Var}(S_n) = \frac{1}{n^2}\,\mathrm{Var}\Big(\sum_{i=1}^{n} X_i\Big) = \frac{1}{n^2} \cdot n\,\mathrm{Var}(X) = \frac{\mathrm{Var}(X)}{n}.$$

By Chebyshev's inequality,

$$\mathbb{P}\big(|S_n - \mu| \geq \epsilon\big) \leq \frac{\mathrm{Var}(X)}{n\epsilon^2} \to 0. \qquad \square$$

The order of quantifiers matters: ϵ is fixed first, and then n grows. The proof is also quantitative: to reach the same bound with an accuracy five times finer (say 0.01 instead of 0.05 in a poll), n must be 25 times larger.

Example: *Gambler's ruin.* A simplified craps bet wins \$1 with probability about 0.4929 and loses \$1 otherwise, for an expected value of about -0.014 per round. By the weak law with $\epsilon = 0.007$, with probability tending to 1 the average winnings after n rounds are below -0.007, so the total is below $-0.007n$. Hence a gambler with any fixed bankroll is ruined by round n with probability tending to 1: any game with negative expected value eventually breaks the player.

8.4 Sums of independent random variables

The same argument applies to sums of independent random variables that are not identically distributed.

Theorem 8.4. *Let $X_1, \ldots, X_n$ be independent with $\mathbb{E}[X_i] = \mu_i$ and $\mathrm{Var}(X_i) = \sigma_i^2$. Then for every $a > 0$,*

$$\mathbb{P}\left(\left|\sum_{i=1}^{n} X_i - \sum_{i=1}^{n} \mu_i\right| \geq a\right) \leq \frac{\sum_{i=1}^{n} \sigma_i^2}{a^2}.$$

Proof. Apply Chebyshev's inequality to $\sum_i X_i$, whose variance is $\sum_i \sigma_i^2$ by independence. $\square$

The only use of independence was $\mathrm{Var}(\sum_i X_i) = \sum_i \mathrm{Var}(X_i)$, which, as shown in Lecture 3, needs only pairwise independence; mutual independence (Lecture 2) is a stronger assumption.

For pairwise independent variables Theorem 8.4 cannot be improved in general. For example, if $X_1, \ldots, X_d$ are independent uniform ± 1 variables and $Y_S = \prod_{i \in S} X_i$ for $S \subseteq \{1, \ldots, d\}$, the Y_S are pairwise independent, and Chebyshev's inequality is asymptotically tight for $\sum_S Y_S$. Mutual independence allows exponentially small tails.

8.5 Boosting a randomized algorithm

Suppose a randomized algorithm, for instance a primality test, answers correctly with probability $\frac{2}{3}$ on every input. Run it n times independently and output the majority answer. Let X_i indicate that run i is correct and $S_n = \frac{1}{n}\sum_i X_i$, so $\mathbb{E}[S_n] = \frac{2}{3}$ and $\mathrm{Var}(X_i) = \frac{2}{3} - \frac{4}{9} = \frac{2}{9}$. The majority is wrong only

if $S_n \le \frac{1}{2}$, so by Chebyshev's inequality

$$\mathbb{P}(\text{majority wrong}) \le \mathbb{P}\big(|S_n - \tfrac{2}{3}| \ge \tfrac{1}{6}\big) \le \frac{2/9}{n/36} = \frac{8}{n}.$$

A failure probability of 10^{-10} then requires about $8 \cdot 10^{10}$ repetitions. Chebyshev's inequality uses only pairwise independence; the Chernoff bound, proved in the next lecture, exploits mutual independence to give a failure probability exponentially small in n, so $O(\log(1/\text{failure probability}))$ repetitions suffice.

8.6 The probabilistic method

To show that an object with some property exists, it suffices to show that a random object has the property with positive probability. The basic tool is the following lemma.

Lemma 8.5. *If X is a nonnegative integer-valued random variable with $\mathbb{E}[X] < 1$, then $\mathbb{P}(X = 0) > 0$.*

Proof. $\mathbb{E}[X] = \sum_{i \ge 0} i\,\mathbb{P}(X = i) \ge \sum_{i \ge 1} \mathbb{P}(X = i) = 1 - \mathbb{P}(X = 0)$, so $\mathbb{P}(X = 0) \ge 1 - \mathbb{E}[X] > 0$. □

Ramsey theory studies structure that every large enough object must contain. For example, among any six people there are three mutual friends or three mutual strangers: every graph on six vertices contains a triangle or three pairwise nonadjacent vertices.

Definition 8.6 [Arithmetic progression]. For integers a and $b \ge 1$, the *k-term arithmetic progression* with start a and step b is

$$Q_{a,b} = \{a,\ a + b,\ a + 2b,\ \ldots,\ a + (k-1)b\}.$$

Theorem 8.7 [van der Waerden]. *For every k there is an n such that every coloring of $\{1, \ldots, n\}$ with two colors contains a monochromatic k-term arithmetic progression.*

Let $W(k)$ be the least such n. A lower bound needs only one good coloring: coloring $1, 2, 5, 6$ red and $3, 4, 7, 8$ blue leaves no monochromatic 3-term progression in $\{1, \ldots, 8\}$, so $W(3) \ge 9$. Upper bounds are much harder, since they concern every coloring. The probabilistic method gives lower bounds that grow exponentially in k.

Theorem 8.8. *For every $k \ge 2$, $W(k) \ge \sqrt{(k-1)\,2^{k-1}}$.*

Proof. Let $n < \sqrt{(k-1)2^{k-1}}$ and color each integer in $\{1, \ldots, n\}$ red or blue independently and uniformly. Let X be the number of monochromatic k-term progressions $Q_{a,b} \subseteq \{1, \ldots, n\}$, and $I_{a,b}$ the indicator that $Q_{a,b}$ is monochromatic, so $X = \sum_{a,b} I_{a,b}$.
Once the color of a is chosen, the other $k - 1$ terms must match it, so $\mathbb{E}[I_{a,b}] = 2^{-(k-1)}$. A

progression inside $\{1, \ldots, n\}$ has $a \le n$ and $(k-1)b \le n$, so there are at most $n \cdot \frac{n}{k-1}$ of them. By linearity of expectation,

$$\mathbb{E}[X] \le \frac{n^2}{(k-1)\,2^{k-1}} < 1.$$

By Lemma 8.5, some coloring has $X = 0$, so $W(k) > n$. As this holds for every integer $n < \sqrt{(k-1)2^{k-1}}$, $W(k) \ge \sqrt{(k-1)2^{k-1}}$. □

The proof shows that a good coloring exists without constructing one.

Source. Ankur Moitra, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 8, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 9: Chernoff Bounds

Markov's inequality uses only nonnegativity and the mean, and Chebyshev's inequality adds the variance; for a sum of variables, Chebyshev needs only pairwise independence. The Chernoff bound assumes that X is a sum of *mutually* independent variables and gives tail probabilities that decay exponentially rather than polynomially. Without mutual independence it can fail completely.

9.1 Statement

Theorem 9.1 [Chernoff bounds]. *Let $X = \sum_{i=1}^{n} X_i$, where the X_i are mutually independent, $X_i = 1$ with probability p_i and $X_i = 0$ otherwise. Let $\mu = \mathbb{E}[X] = \sum_{i=1}^{n} p_i$. Then*

(i) (Upper tail) for all $\delta > 0$,

$$\mathbb{P}\big(X \geq (1+\delta)\mu\big) \leq e^{-\frac{\delta^2}{2+\delta}\mu};$$

(ii) (Lower tail) for all $0 < \delta < 1$,

$$\mathbb{P}\big(X \leq (1-\delta)\mu\big) \leq e^{-\mu\delta^2/2}.$$

The two tails differ only because different estimates of the logarithm are used in their proofs. The exponent grows with μ: for n flips of a fair coin, $\mu = n/2$, so the chance of 10% too many heads decays exponentially in n, whereas the weak law of large numbers gives only an inverse polynomial rate.

Corollary 9.2. *With X and μ as above,*

$$\mathbb{P}\big(|X - \mu| \geq \delta\mu\big) \leq 2e^{-\mu\delta^2/3} \qquad \text{for all } 0 < \delta < 1.$$

Proof. For $0 < \delta < 1$ we have $\frac{\delta^2}{2+\delta} \geq \frac{\delta^2}{3}$ and $\frac{\delta^2}{2} \geq \frac{\delta^2}{3}$, so each tail in Theorem 9.1 is at most $e^{-\mu\delta^2/3}$. Add the two bounds. □

Many other Chernoff bounds exist, for instance for variables that are not Bernoulli. They are proved by the same method, and in practice one looks up and cites the version that fits.

9.2 Example: boosting a randomized algorithm

Return to the primality test of Lecture 8: it is correct with probability $2/3$, we run it n times independently, let S_n count the answers "prime", and output "prime" if $S_n/n \geq 1/2$. Chebyshev bounded the failure probability by $8/n$, so a failure probability of 10^{-10} needed about $8 \cdot 10^{10}$

repetitions. If the input is prime, then $\mathbb{E}[S_n/n] = 2/3$, and the composite case is symmetric. With Corollary 9.2 and $\delta = 1/4$, so that $\delta \cdot 2n/3 = n/6$,

$$\mathbb{P}\big(|S_n/n - 2/3| \geq 1/6\big) \leq 2e^{-(2n/3)(1/16)/3} = 2e^{-n/72}.$$

Now $n \approx 72\ln(2 \cdot 10^{10}) < 1{,}800$ repetitions suffice: the number of repetitions grows like the logarithm of the inverse failure probability.

9.3 The moment generating function

Definition 9.3 [Moment generating function]. The moment generating function of a random variable X is $M_X(s) = \mathbb{E}\big[e^{sX}\big]$ for $s \in \mathbb{R}$.

Expanding e^{sX} and using linearity (when the series converges),

$$M_X(s) = \mathbb{E}\Big[1 + sX + \tfrac{1}{2!}s^2X^2 + \tfrac{1}{3!}s^3X^3 + \cdots\Big] = \sum_{i=0}^{\infty} \frac{s^i}{i!}\mathbb{E}\big[X^i\big].$$

The coefficients record the moments $\mathbb{E}[X^i]$, which can be read off by differentiation: $M_X'(0) = \mathbb{E}[X]$ and $M_X''(0) = \mathbb{E}[X^2]$. Just as a counting generating function stores a sequence in its coefficients, M_X stores all moments of X at once. The Chernoff bound uses all of them, while Markov and Chebyshev use only the first two.

9.4 Proof of the Chernoff bound

The proof follows the template of Chebyshev's inequality: apply Markov's inequality to a nonnegative function of X. Instead of squaring, exponentiate. For any $s > 0$ and $a \in \mathbb{R}$, the events $\{X \geq a\}$ and $\{e^{sX} \geq e^{sa}\}$ coincide because $t \mapsto e^{st}$ is increasing, and $e^{sX} \geq 0$, so

$$\mathbb{P}(X \geq a) = \mathbb{P}\big(e^{sX} \geq e^{sa}\big) \leq \frac{M_X(s)}{e^{sa}}. \tag{9.1}$$

Similarly, $\mathbb{P}(X \leq a) = \mathbb{P}\big(e^{-sX} \geq e^{-sa}\big) \leq M_X(-s)/e^{-sa}$. Three lemmas finish the argument.

Lemma 9.4 [Products]. *If $X = \sum_{i=1}^n X_i$ with the X_i mutually independent, then $M_X(s) = \prod_{i=1}^n M_{X_i}(s)$.*

Proof. $M_X(s) = \mathbb{E}\big[\prod_i e^{sX_i}\big] = \prod_i \mathbb{E}\big[e^{sX_i}\big]$. The second step needs the e^{sX_i} to be mutually independent, which holds because the X_i are; pairwise independence would not suffice. □

Lemma 9.5 [Bernoulli terms]. *If $X_i = 1$ with probability p_i and 0 otherwise, then $M_{X_i}(s) \leq e^{p_i(e^s - 1)}$ for all $s \in \mathbb{R}$.*

Proof. $M_{X_i}(s) = p_i e^s + (1-p_i) = 1 + p_i(e^s - 1) \le e^{p_i(e^s-1)}$, using $1 + y \le e^y$ for all real y. □

The weaker form is chosen so that the p_i sit in the exponent: by Lemmas 9.4 and 9.5,

$$M_X(s) \le \prod_{i=1}^{n} e^{p_i(e^s-1)} = e^{(e^s-1)\mu}. \tag{9.2}$$

This is the only step that uses the Bernoulli assumption. For other summands one computes or bounds their moment generating functions and reassembles them in the same way.

Lemma 9.6 [Logarithm estimates]**.**

(i) $\ln(1+x) \ge \dfrac{x}{1+x/2}$ *for* $x > 0$.

(ii) $(1-x)\ln(1-x) \ge -x + \dfrac{x^2}{2}$ *for* $0 < x < 1$.

Proof. (i) Let $g(x) = \ln(1+x) - \frac{2x}{2+x}$. Then $g(0) = 0$ and $g'(x) = \frac{1}{1+x} - \frac{4}{(2+x)^2} = \frac{x^2}{(1+x)(2+x)^2} \ge 0$.
(ii) Let $f(x) = (1-x)\ln(1-x) + x - \frac{x^2}{2}$. Then $f(0) = 0$ and $f'(x) = -\ln(1-x) - x \ge 0$, since $-\ln(1-x) \ge x$. □

Proof of Theorem 9.1. (i) Take $a = (1+\delta)\mu$ and $s = \ln(1+\delta) > 0$ in (9.1). This s minimizes the resulting exponent in s. By (9.2), since $e^s - 1 = \delta$,

$$\mathbb{P}\big(X \ge (1+\delta)\mu\big) \le \frac{e^{\delta\mu}}{(1+\delta)^{(1+\delta)\mu}} = \left(\frac{e^{\delta}}{(1+\delta)^{1+\delta}}\right)^{\mu}.$$

The logarithm of the right side is $\mu\big(\delta - (1+\delta)\ln(1+\delta)\big)$, and by Lemma 9.6(i),

$$\mu\big(\delta - (1+\delta)\ln(1+\delta)\big) \le \mu\left(\delta - \frac{2\delta(1+\delta)}{2+\delta}\right) = -\frac{\delta^2}{2+\delta}\,\mu.$$

(ii) Take $a = (1-\delta)\mu$ and $s = -\ln(1-\delta) > 0$, so $e^{-s} = 1 - \delta$. By the lower-tail form of (9.1) and (9.2) at $-s$,

$$\mathbb{P}\big(X \le (1-\delta)\mu\big) \le \frac{e^{(e^{-s}-1)\mu}}{e^{-s(1-\delta)\mu}} = \left(\frac{e^{-\delta}}{(1-\delta)^{1-\delta}}\right)^{\mu}.$$

Its logarithm is $\mu\big(-\delta - (1-\delta)\ln(1-\delta)\big) \le \mu\big(-\delta + \delta - \frac{\delta^2}{2}\big) = -\frac{\mu\delta^2}{2}$ by Lemma 9.6(ii). □

The proof has three parts. The exponential trick with Markov's inequality gives (9.1). Mutual independence gives the product in Lemma 9.4. Everything after that is calculus.

Note. The bound is only as good as the independence assumption. Before 2008, subprime mortgages were pooled into securities with a "waterfall" structure: the top tranche lost money only if, say, 40% of the mortgages defaulted, while each defaulted with probability about 10%. For a pool of 1,000 mortgages defaulting independently, $\mu = 100$, and the upper tail with $\delta = 3$ bounds the chance of 400 defaults by $e^{-(9/5)\cdot 100} = e^{-180}$, so the tranche looked essentially riskless. Defaults, however, are driven by common economic shocks and are strongly dependent. The Chernoff bound did not apply, and these assets failed together.

Source. Ankur Moitra, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 9, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 10: Modular Arithmetic

Modular arithmetic keeps track only of remainders. Even plus odd is odd no matter which even and odd numbers are added; this is arithmetic modulo 2. The same idea for a general modulus underlies cryptography, coding theory, and the algebra of the next lecture.

10.1 Congruences and residues

Definition 10.1 [Congruence]. Let n be a positive integer. Integers a and b are congruent modulo n, written $a \equiv b \pmod{n}$, if $a - b$ is a multiple of n, that is, $a - b = kn$ for some $k \in \mathbb{Z}$.

For example, $-2 \equiv 8 \pmod{5}$ and $12 \equiv 5 \pmod{7}$. Congruence modulo n is an equivalence relation (reflexive, symmetric, transitive), so it partitions $\mathbb{Z}$ into classes.

Theorem 10.2 [Division algorithm]. *For all integers a and $n > 0$ there is a unique pair of integers q, r with $a = qn + r$ and $0 \le r \le n - 1$.*

The quotient is q, and the remainder or residue r is written $a \bmod n$. The notation $a \bmod n$ is a number in $\{0, \dots, n-1\}$, whereas $a \equiv b \pmod{n}$ is a relation with no such restriction.

Lemma 10.3. *$a \equiv b \pmod{n}$ if and only if $a \bmod n = b \bmod n$.*

Proof. Write $a = qn + r$ and $b = q'n + r'$ with $0 \le r, r' \le n - 1$. If $r = r'$, then $a - b = (q - q')n$. Conversely, if $n \mid a - b = (q - q')n + (r - r')$, then $n \mid r - r'$. Since $-n < r - r' < n$, this forces $r = r'$. □

So each class contains exactly one element of $\mathbb{Z}_n := \{0, 1, \dots, n-1\}$. Picture $0, \dots, n-1$ evenly spaced around a clock: 12 and 5 land on the same position modulo 7 because 12 goes around once more. For $a, b \in \mathbb{Z}_n$, define $a \oplus b$ and $a \otimes b$ as the residues of $a + b$ and ab modulo n; later we simply write $+$ and $\cdot$. For example, in $\mathbb{Z}_5$, $3 \oplus 4 = 2$ and $3 \otimes 4 = 2$.

These operations are compatible with congruence. If $a \equiv a' \pmod{n}$ and $b \equiv b' \pmod{n}$, then $a + b \equiv a' + b'$ and $ab \equiv a'b'$, because $ab - a'b' = a(b - b') + b'(a - a')$. So one may add and multiply integers as usual and reduce modulo n only at the end: $10 \equiv 3$ and $12 \equiv 5 \pmod{7}$, and indeed $10 \cdot 12 = 120 \equiv 1 \equiv 15 \pmod{7}$. Associativity, commutativity, and the distributive law $a(b + c) \equiv ab + ac$ carry over from $\mathbb{Z}$. In $\mathbb{Z}_n$,

(i) $a \oplus b \in \mathbb{Z}_n$;

(ii) $a \oplus (b \oplus c) = (a \oplus b) \oplus c$;

(iii) $a \oplus 0 = a = 0 \oplus a$;

(iv) every a has some b with $a \oplus b = 0 = b \oplus a$.

These are the axioms of a group, studied in the next lecture.

Example: *A five-card protocol.* Two people want to learn whether both like some embarrassing movie, without anyone learning more than that. Use three red cards and two black cards. Each person gets one red and one black card, and one red card is placed face down in the middle. The first person places a pair to the left of the middle card, putting red next to the middle (order BR) for yes and black next to the middle (order RB) for no; the second person places a pair to the right, again red next to the middle (RB) for yes and black next to it (BR) for no. Both then cut the stacked deck, which shifts the positions cyclically by an unknown amount modulo 5. When the cards are laid out in a circle, three red cards are cyclically adjacent if and only if both said yes (BRRRB). The other three outcomes, BRRBR, RBRRB, and RBRBR, are cyclic shifts of RRBRB, so after the cuts they look identical, and a yes answer is hidden whenever the result is negative. This is a simple example of a zero-knowledge protocol.

10.2 Multiplicative inverses and the gcd

An element b is a MULTIPLICATIVE INVERSE of a modulo n if $ab \equiv 1 \pmod{n}$. Modulo 7, $2 \cdot 4 \equiv 1$ and $3 \cdot 5 \equiv 1$, so $2, 4$ and $3, 5$ are inverse pairs. Multiplying by an invertible element scrambles residues, and multiplying by its inverse undoes this, which is the basic mechanism of later cryptographic schemes. The element 0 is never invertible. Which nonzero elements are, and how can an inverse be found without writing out the whole multiplication table?

The GREATEST COMMON DIVISOR $\gcd(a, b)$ is the largest integer dividing both a and b, with $\gcd(a, 0) = a$. Integers with $\gcd(a, b) = 1$ are RELATIVELY PRIME. Trying every candidate divisor is hopeless for numbers with many digits. Euclid's algorithm is fast.

Lemma 10.4. *If $b \geq a$, then $\gcd(a, b) = \gcd(a, b - a)$.*

Proof. If $k \mid a$ and $k \mid b$, then $k \mid b - a$; conversely, if $k \mid a$ and $k \mid b - a$, then $k \mid (b - a) + a = b$. So the pairs (a, b) and $(a, b - a)$ have the same common divisors, hence the same greatest one. □

Lemma 10.5. *If $a > 0$, then $\gcd(a, b) = \gcd(a, b \bmod a)$.*

Proof. Write $b = qa + r$. Applying Lemma 10.4 q times, $\gcd(a, b) = \gcd(a, b - a) = \cdots = \gcd(a, b - qa) = \gcd(a, r)$. □

Algorithm 10.1: Euclid's algorithm

Input: integers $a \geq b \geq 0$, not both 0
Output: $\gcd(a, b)$

```
while b ≠ 0 do
    (a, b) ← (b, a mod b)
return a
```

Correctness follows from Lemma 10.5 and $\gcd(a, 0) = a$. It is also fast: if $b \leq a$, then $a \bmod b < a/2$ (if $b \leq a/2$ the remainder is below b, and otherwise it is $a - b < a/2$), so the larger number at least halves every two iterations and there are $O(\log a)$ iterations. For example,

$$\gcd(360, 46) = \gcd(46, 38) = \gcd(38, 8) = \gcd(8, 6) = \gcd(6, 2) = \gcd(2, 0) = 2,$$

and

$$\begin{aligned}\gcd(365, 211) &= \gcd(211, 154) = \gcd(154, 57) = \gcd(57, 40) = \gcd(40, 17) \\ &= \gcd(17, 6) = \gcd(6, 5) = \gcd(5, 1) = \gcd(1, 0) = 1.\end{aligned}$$

Extended Euclidean algorithm. Record each remainder as an integer combination of the two inputs. Starting from 360 and 46, the first remainder is $38 = 360 - 7 \cdot 46$, the next is $8 = 46 - 38 = 8 \cdot 46 - 360$, and so on; every intermediate number is $s \cdot 46 + t \cdot 360$ for some integers s, t. In general, maintain $a_i = x_i a + y_i b$; if $a_i = a_{i-2} - q a_{i-1}$, then $x_i = x_{i-2} - q x_{i-1}$ and $y_i = y_{i-2} - q y_{i-1}$ keep the invariant.

Algorithm 10.2: Extended Euclidean algorithm

Input: integers $a \geq b \geq 0$, not both 0
Output: $g = \gcd(a, b)$ and integers s, t with $sa + tb = g$

```
(r_0, x_0, y_0) ← (a, 1, 0);   (r_1, x_1, y_1) ← (b, 0, 1)
while r_1 ≠ 0 do
    q ← ⌊r_0 / r_1⌋
    (r_0, x_0, y_0, r_1, x_1, y_1) ← (r_1, x_1, y_1, r_0 − q r_1, x_0 − q x_1, y_0 − q y_1)
return (r_0, x_0, y_0)
```

For $a = 365$ and $b = 211$, the successive rows $(a_i, a_{i+1}; x_i, y_i)$ are

state	x	y
$\gcd(365, 211)$	1	0
$\gcd(211, 154)$	0	1
$\gcd(154, 57)$	1	-1
$\gcd(57, 40)$	-1	2
$\gcd(40, 17)$	3	-5
$\gcd(17, 6)$	-4	7
$\gcd(6, 5)$	11	-19
$\gcd(5, 1)$	-26	45
$\gcd(1, 0)$	37	-64

so $1 = 37 \cdot 365 - 64 \cdot 211$. For instance, $6 = 11 \cdot 365 - 19 \cdot 211$ comes from subtracting twice $17 = -4 \cdot 365 + 7 \cdot 211$ from $40 = 3 \cdot 365 - 5 \cdot 211$. Similarly, $2 = 47 \cdot 46 - 6 \cdot 360$.

Lemma 10.6 [Bézout]. *For all integers a, b, not both 0, there are integers s, t with $sa + tb = \gcd(a, b)$.*

The pair (s, t) is also a certificate. Anyone can check that a claimed g divides a and b, and if $sa + tb = g$, then every common divisor of a and b divides g, so no larger common divisor exists.

Theorem 10.7. *Let $n \geq 2$. An integer a has a multiplicative inverse modulo n if and only if $\gcd(a, n) = 1$. In particular, if n is prime, every $a \not\equiv 0 \pmod{n}$ is invertible.*

Proof. If $\gcd(a, n) = 1$, Lemma 10.6 gives $sa + tn = 1$, so $sa \equiv 1 \pmod{n}$ and $s \bmod n$ is an inverse. Conversely, let $g = \gcd(a, n)$ and suppose $ab \equiv 1$, that is, $ab = kn + 1$. Then g divides $ab - kn = 1$, so $g = 1$. □

To invert a modulo a large prime p, run the extended Euclidean algorithm on p and a; the gcd is known to be 1, and the coefficient of a is the inverse. For example, $\gcd(4, 7) = 1$ with $2 \cdot 4 - 1 \cdot 7 = 1$, so $4^{-1} \equiv 2 \pmod{7}$. The modulus need not be prime: 5 is invertible modulo 6 (indeed $5 \cdot 5 = 25 \equiv 1$), while 2 is not, since $\gcd(2, 6) = 2$.

10.3 The Chinese remainder theorem

Example: A treasure chest holds gold coins. Divided among 5 pirates, 4 coins are left over; divided among 7, 1 is left over. The smallest possible number n satisfies $n \equiv 4 \pmod{5}$ and $n \equiv 1 \pmod{7}$, and one finds $n = 29$.

This is a system of linear equations in different moduli.

Lemma 10.8. *If $a \mid N$, $b \mid N$, and $\gcd(a, b) = 1$, then $ab \mid N$.*

Proof. Write $1 = sa + tb$, so $N = saN + tbN$. Since $b \mid N$, $ab \mid saN$, and since $a \mid N$, $ab \mid tbN$. Hence $ab \mid N$. □

Without coprimality this fails: $6 \mid 18$ and $9 \mid 18$, but $54 \nmid 18$.

Theorem 10.9 [Chinese remainder theorem]. *Let $a, b \geq 1$ with $\gcd(a, b) = 1$. For every pair (r, s) with $0 \leq r < a$ and $0 \leq s < b$ there is exactly one $n \in \{0, \ldots, ab-1\}$ with $n \equiv r \pmod{a}$ and $n \equiv s \pmod{b}$. Moreover, the correspondence $n \mapsto (n \bmod a,\ n \bmod b)$ respects addition and multiplication: the pair of a sum or product is the componentwise sum or product of the pairs.*

Proof. Both $\{0, \ldots, ab-1\}$ and the set of pairs have ab elements, so it suffices to show that the map $n \mapsto (n \bmod a, n \bmod b)$ is injective. Suppose $0 \leq y < x < ab$ have the same pair. Then $a \mid x - y$ and $b \mid x - y$, so $ab \mid x - y$ by Lemma 10.8. But $0 < x - y < ab$, a contradiction. An injective map between finite sets of the same size is a bijection, so every pair has exactly one preimage. The last statement follows from the compatibility of congruences with $+$ and $\times$. □

For instance, with $a = 7$ and $b = 13$, the residue $73 \bmod 91$ corresponds to $(73 \bmod 7,\ 73 \bmod 13) = (3, 8)$, and no other residue modulo 91 does.

Finding the solution. Given (r, s), write the unknown as $n = kb + s$ with $0 \leq k < a$, which settles the condition modulo b. The condition modulo a becomes $kb \equiv r - s \pmod{a}$. Since $\gcd(a, b) = 1$, b has an inverse modulo a, computed by the extended Euclidean algorithm, and

$$k = b^{-1}(r - s) \bmod a, \qquad n = kb + s.$$

For the pirates, $a = 5$, $b = 7$, $r = 4$, $s = 1$: $7^{-1} \equiv 3 \pmod 5$, so $k \equiv 3 \cdot 3 \equiv 4 \pmod 5$ and $n = 4 \cdot 7 + 1 = 29$.

Square roots of 1. Pairs make some equations easy to solve. If $\gcd(a, b) = 1$ and $a, b \geq 3$, the equation $x^2 \equiv 1 \pmod{ab}$ has at least four solutions, namely the residues with pairs $(\pm 1, \pm 1)$, where -1 stands for $a - 1$ or $b - 1$. Each squares to the pair $(1, 1)$, which corresponds only to 1. Modulo 15 these are the residues $1, 4, 11, 14$, with pairs $(1, 1), (1, 4), (2, 1), (2, 4)$, and indeed $1, 16, 121, 196 \equiv 1 \pmod{15}$.

Several moduli and fountain codes. The theorem extends by induction to pairwise relatively prime $m_1, \ldots, m_k$: the residues modulo each m_i determine n modulo $m_1 \cdots m_k$. Suppose a large file is encoded as an integer N, and a server endlessly broadcasts pairs $(p_i,\ N \bmod p_i)$ for many distinct primes p_i. A receiver who joins at any time collects pairs until the product of the primes received exceeds N, then reconstructs N by the Chinese remainder theorem. It does not matter which pairs are received, only that enough of them are. This is a much simplified version of a

fountain code, used to send one file to many receivers who listen at different times.

Source. Ankur Moitra, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 10, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 11: Basic Group Theory

The previous lecture showed that a is invertible modulo n exactly when $\gcd(a, n) = 1$. Group theory turns such facts into general structure, and Lagrange's theorem then gives short proofs of Fermat's and Euler's theorems, the identities behind RSA encryption.

11.1 Groups

Definition 11.1 [Group]. A group $(G, *)$ is a set G, possibly infinite, with a binary operation $*$ such that

(i) (*closure*) $a * b \in G$ for all $a, b \in G$;

(ii) (*associativity*) $(a * b) * c = a * (b * c)$ for all $a, b, c \in G$;

(iii) (*identity*) there is $e \in G$ with $a * e = e * a = a$ for all $a \in G$;

(iv) (*inverses*) for every $a \in G$ there is $a^{-1} \in G$ with $a * a^{-1} = a^{-1} * a = e$.

If moreover $a * b = b * a$ for all a, b, the group is abelian.

Example:

- $(\mathbb{Z}, +)$ is an abelian group with identity 0 and inverse $-a$; so are $(\mathbb{Q}, +)$, $(\mathbb{R}, +)$, and $(\mathbb{C}, +)$.
- $(\mathbb{Z}, \times)$ is not a group: 0 and 2 have no inverse in $\mathbb{Z}$. Removing 0 repairs this for the rationals, reals, and complex numbers: $(\mathbb{Q} \setminus \{0\}, \times)$ is an abelian group.
- The invertible $n \times n$ real matrices form a group GL_n under matrix multiplication, which is not abelian for $n \geq 2$.
- $(\mathbb{Z}_n, +)$ is a finite abelian group with identity 0. For $n = 2$ it is the arithmetic of evens and odds.
- Fix a polynomial $q(x)$. Polynomials of degree less than $\deg q$, added modulo $q(x)$, form a group: two polynomials are equivalent when their difference is a multiple $k(x)q(x)$. The identity is the zero polynomial and the inverse of $p(x)$ is $-p(x)$. For $q(x) = x^3 + 2x^2 + 1$, the residues are the polynomials of degree at most 2.

11.2 Multiplicative groups modulo n

Corollary 11.2. *$(\mathbb{Z}_n \setminus \{0\}, \times)$ is a group if and only if n is prime.*

Proof. Multiplication modulo n is associative with identity 1, so only closure and inverses are in question. If n is prime, every nonzero residue is invertible by Theorem 10.7, and the product of invertible residues is invertible, hence nonzero. If n is composite, some a with $1 < a < n$ has $\gcd(a, n) \neq 1$, and this a has no inverse. □

For composite n a group still sits inside. Let

$$\mathbb{Z}_n^* := \{a \in \mathbb{Z}_n : \gcd(a, n) = 1\}.$$

Then $(\mathbb{Z}_n^*, \times)$ is a group. A product of numbers relatively prime to n is relatively prime to n, associativity is inherited, and $1 \in \mathbb{Z}_n^*$. Each $a \in \mathbb{Z}_n^*$ has an inverse s from $sa + tn = 1$, and $\gcd(s, n) = 1$ by the same equation, so $s \bmod n$ lies in $\mathbb{Z}_n^*$. For example, $\mathbb{Z}_{15}^* = \{1, 2, 4, 7, 8, 11, 13, 14\}$.

The ORDER of a finite group G is its number of elements $|G|$. For prime p, $|\mathbb{Z}_p^*| = p - 1$.

Proposition 11.3. *If $p \neq q$ are primes and $m = pq$, then $|\mathbb{Z}_m^*| = (p - 1)(q - 1)$.*

Proof. Count $a \in \{1, \ldots, m - 1\}$ with $\gcd(a, m) = 1$ by inclusion-exclusion. Remove the multiples of q, namely $q, 2q, \ldots, (p - 1)q$, and the multiples of p, namely $p, \ldots, (q - 1)p$. No number below pq is a multiple of both. So $|\mathbb{Z}_m^*| = (pq - 1) - (p - 1) - (q - 1) = (p - 1)(q - 1)$. □

For $m = 15$ this gives $2 \cdot 4 = 8$, matching the list above. The quantity $(p - 1)(q - 1)$ is the key to RSA.

11.3 Subgroups and Lagrange's theorem

Definition 11.4 [SUBGROUP]. If $(G, *)$ is a group and $H \subseteq G$ is a group under the same operation, then H is a SUBGROUP of G.

For a nonempty $H \subseteq G$ it suffices to check that H is closed under $*$ and under inverses: the identity is then $a * a^{-1} \in H$, and associativity is inherited. The point is that products and inverses of elements of H must stay in H, not merely in G.

Example: In $G = \mathbb{Z}_7^*$, of order 6, the set $H = \{1, 2, 4\}$ is a subgroup: $2 \cdot 4 \equiv 1$, so 2 and 4 are inverse to each other. In fact H is the set of powers of 2: $2^1 = 2$, $2^2 = 4$, $2^3 \equiv 1$. Products of powers of 2 add exponents, which explains closure.

Theorem 11.5 [LAGRANGE'S THEOREM]. *If G is a finite group and H is a subgroup of G, then $|H|$ divides $|G|$.*

So a group of order 6 has no subgroup of order 4. The proof partitions G into translates of H. For $x \in G$, the LEFT COSET of H by x is $xH := \{x * y : y \in H\}$; for $x \in H$ it is H itself. In the example, $3H = \{3, 6, 5\}$.

Proof. Choose cosets greedily: let $x_1 = e$, and while some element of G is not yet covered, let x_{j+1} be any uncovered element. Because G is finite, this ends with $G = x_1H \cup \cdots \cup x_kH$. It remains to prove two claims.
(1) $|xH| = |H|$ for every x. If $xh_1 = xh_2$, multiplying on the left by x^{-1} and using associativity, $x^{-1}x = e$, and $eh = h$ gives $h_1 = h_2$. So distinct elements of H give distinct elements of xH.
(2) The cosets are disjoint. Suppose $x_ih_1 = x_jh_2$ with $i < j$ and $h_1, h_2 \in H$. Multiplying on the right by h_2^{-1} gives $x_j = x_i(h_1h_2^{-1})$. Since $h_2^{-1} \in H$ and H is closed, $h_1h_2^{-1} \in H$, so $x_j \in x_iH$ was already covered when it was chosen, a contradiction.
Hence $|G| = k|H|$. □

Note. A subgroup H is NORMAL if $aH = Ha$ for all $a \in G$, where $Ha = \{h * a : h \in H\}$ is a right coset. In abelian groups every subgroup is normal. The cosets of a normal subgroup form a group under $(aH)(bH) = (ab)H$, the FACTOR GROUP G/H. For example, $n\mathbb{Z}$ is a subgroup of $(\mathbb{Z}, +)$, its cosets are the residue classes modulo n, and $\mathbb{Z}/n\mathbb{Z}$ is $(\mathbb{Z}_n, +)$.

11.4 Fermat's little theorem and Euler's theorem

Theorem 11.6. *Let G be a finite group and $a \in G$. Let k be the least positive integer with $a^k = e$, the* ORDER *of a. Such a k exists, $\{e, a, \ldots, a^{k-1}\}$ is a subgroup of order k, and k divides $|G|$. Consequently, $a^{|G|} = e$.*

Proof. The powers $a, a^2, a^3, \ldots$ lie in the finite set G, so by the pigeonhole principle $a^i = a^j$ for some $i < j$. Multiplying by $(a^{-1})^i$ gives $a^{j-i} = e$, so k exists. The powers $e, a, \ldots, a^{k-1}$ are distinct (if $a^i = a^j$ with $0 \le i < j < k$, then $a^{j-i} = e$ with $0 < j - i < k$), they are closed under multiplication since exponents add modulo k, and a^{k-i} is the inverse of a^i. So they form a subgroup of order k, and $k \mid |G|$ by Theorem 11.5. Writing $|G| = ck$, $a^{|G|} = (a^k)^c = e^c = e$. □

Theorem 11.7 [FERMAT'S LITTLE THEOREM]. *If p is prime and $p \nmid a$, then $a^{p-1} \equiv 1 \pmod p$.*

Proof. Apply Theorem 11.6 to a mod p in $\mathbb{Z}_p^*$, which has order $p - 1$. □

In particular $a \cdot a^{p-2} \equiv 1$, so $a^{-1} \equiv a^{p-2} \pmod p$: inverses modulo a prime can be computed by powering as well as by the extended Euclidean algorithm. Modulo 7, $2^5 = 32 \equiv 4$, confirming $2^{-1} \equiv 4$.

A combinatorial proof. Fermat's theorem is equivalent to $p \mid a^p - a$ for every integer a (for $p \nmid a$, cancel a; for $p \mid a$ both sides vanish modulo p). Consider the a^p strings of length p over an alphabet of a bead colors, and join the ends of each string to form a necklace. Rotating a string gives the same necklace. The a monochromatic strings are fixed by every rotation. Any other string has p

distinct rotations: if a rotation by r positions, with $0 < r < p$, fixed the string, then bead i would equal bead $i + r$, hence bead $i + jr$ for all j (indices modulo p). Since p is prime, the multiples of r cover every position, so the string would be monochromatic. Hence the $a^p - a$ non-monochromatic strings fall into classes of size exactly p, and $p \mid a^p - a$. For $a = 2$ and $p = 5$, the 30 two-colored strings form 6 necklaces of 5 strings each.

Theorem 11.8 [Euler]. *If $m = pq$ for distinct primes p, q and $\gcd(a, m) = 1$, then $a^{(p-1)(q-1)} \equiv 1 \pmod{m}$.*

Proof. The proof of Fermat's theorem goes through unchanged with the group $\mathbb{Z}_m^*$, which contains $a \bmod m$ and has order $(p-1)(q-1)$ by Proposition 11.3. □

The same argument gives $a^{|\mathbb{Z}_m^*|} \equiv 1 \pmod{m}$ for every m and every a with $\gcd(a, m) = 1$. The exponent changes from $p - 1$ to $(p-1)(q-1)$ because $\mathbb{Z}_m \setminus \{0\}$ is no longer a group, and what matters is the order of the group $\mathbb{Z}_m^*$ inside it.

11.5 Fields and polynomial roots

Definition 11.9 [Field]. A field $(\mathbb{F}, +, \cdot)$ is a set with two operations such that

1. $(\mathbb{F}, +)$ is an abelian group with identity 0;
2. $(\mathbb{F} \setminus \{0\}, \cdot)$ is an abelian group with identity $1 \neq 0$;
3. $a(b + c) = ab + ac$ for all $a, b, c \in \mathbb{F}$.

The rationals, reals, and complex numbers are fields, and by Corollary 11.2, $\mathbb{Z}_m$ with modular addition and multiplication is a field if and only if m is prime.

Lemma 11.10. *In a field, $ab = 0$ implies $a = 0$ or $b = 0$.*

Proof. If $a, b \neq 0$, then $(b^{-1}a^{-1})(ab) = 1$, so ab has an inverse and cannot be 0, since $0 \cdot c = 0$ for all c. □

Lemma 11.11. *If $p(x)$ has coefficients in a field and $p(a) = 0$, then $x - a$ divides $p(x)$.*

Proof. Polynomial division by the monic $x - a$ gives $p(x) = q(x)(x - a) + r$ with $r \in \mathbb{F}$. Evaluating at a gives $r = 0$. □

Theorem 11.12. *A polynomial of degree $d \geq 1$ with coefficients in a field $\mathbb{F}$ has at most d roots in $\mathbb{F}$.*

Proof. Induct on d. If $d = 1$, $ax + b = 0$ with $a \neq 0$ has the unique solution $x = -ba^{-1}$. For $d > 1$, if p has no root we are done. Otherwise let a be a root; by Lemma 11.11, $p(x) = (x - a)q(x)$ with $\deg q = d - 1$. By Lemma 11.10, every root of p is a or a root of q, and q has at most $d - 1$ roots by induction. So p has at most d roots. □

This familiar fact about real polynomials holds over every field, including finite fields such as $\mathbb{Z}_p$, where it underlies secret-sharing schemes and Reed-Solomon codes.

Source. Ankur Moitra, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 11, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 12: Introduction to Linear Programming

A linear program (LP) optimizes, that is, minimizes or maximizes, a linear objective function of decision variables subject to linear equality and inequality constraints. Linear programming drives much of practical optimization, underlies combinatorial optimization, and through duality is also a proof technique. To model a problem, choose variables, write the objective, and write the constraints; solving the LP is then left to software.

12.1 Formulations

The diet problem. Two grains supply starch, protein, and vitamins:

	Starch	Protein	Vitamins	Cost (\$/kg)
G_1	5	4	2	0.60
G_2	7	2	1	0.35

The daily requirements are 8, 15, and 3 units. With x_1, x_2 the amounts of G_1, G_2 consumed per day, the cheapest adequate diet solves

$$\begin{aligned} \text{minimize} \quad & z = 0.6x_1 + 0.35x_2 \\ \text{subject to} \quad & 5x_1 + 7x_2 \geq 8, \quad 4x_1 + 2x_2 \geq 15, \quad 2x_1 + x_2 \geq 3, \\ & x_1 \geq 0, \; x_2 \geq 0. \end{aligned}$$

The last two are nonnegativity constraints, present in most LPs and often handled separately. A point satisfying all constraints is feasible, the set of such points is the feasible region, and a feasible point with the best objective value is optimal.

The transportation problem. Factories F_1, F_2 supply 6 and 9 thousand widgets per month, and retail centers C_1, C_2, C_3 demand 8, 5, and 2 thousand. Shipping costs (in cents per widget) are

	C_1	C_2	C_3
F_1	5	5	3
F_2	6	4	1

With x_{ij} the amount shipped from F_i to C_j, minimize $5x_{11} + 5x_{12} + 3x_{13} + 6x_{21} + 4x_{22} + x_{23}$ subject to the demands $x_{11} + x_{21} = 8$, $x_{12} + x_{22} = 5$, $x_{13} + x_{23} = 2$, the supplies $x_{11} + x_{12} + x_{13} \leq 6$ and $x_{21} + x_{22} + x_{23} \leq 9$, and $x_{ij} \geq 0$. Total supply equals total demand, so the supply constraints can be taken as equalities. One of the five equalities is then redundant: the three demand equations

minus the first supply equation give the second supply equation.

A bake sale. A batch of cookies uses 1 egg, 2 cups of sugar, and 1.5 cups of flour; a batch of meringues uses 3 eggs and 1 cup of sugar. Each batch sells for \$20, and there are 9 eggs, 8 cups of sugar, and 15 cups of flour. With x_1, x_2 the numbers of batches, allowed to be fractional (integer constraints would make this a much harder integer program),

$$\begin{aligned} &\text{maximize} && z = 20x_1 + 20x_2 \\ &\text{subject to} && x_1 + 3x_2 \le 9, \quad 2x_1 + x_2 \le 8, \quad 1.5x_1 \le 15, \\ & && x_1, x_2 \ge 0. \end{aligned} \tag{12.1}$$

With two variables the feasible region can be drawn (Figure 8). The flour constraint $x_1 \le 10$ is implied by the sugar constraint and can be dropped.

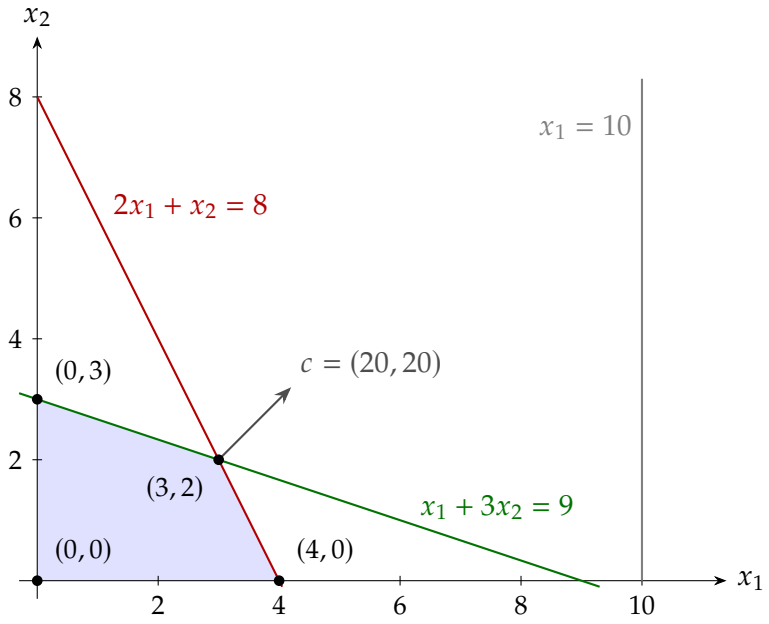


Figure 8: Feasible region of the bake-sale LP (12.1), shaded. Its corners are the basic feasible solutions, and the objective direction c is maximized at $(3, 2)$ with $z = 100$.

12.2 Standard and canonical forms

A general LP in variables $x_1, \ldots, x_n$ is

$$\begin{aligned} &\text{maximize or minimize} && z = c_0 + c_1x_1 + \cdots + c_nx_n \\ &\text{subject to} && a_{i1}x_1 + \cdots + a_{in}x_n \ \{\le, \ge, =\} \ b_i \quad (i = 1, \ldots, m), \end{aligned}$$

with each x_j either restricted to $x_j \ge 0$ or FREE (of either sign). The c_j are COST COEFFICIENTS, the b_i are RIGHT-HAND SIDES, and the constant c_0 does not affect which solutions are optimal. Two forms

are convenient. In matrix notation, with $A = [a_{ij}]$ an $m \times n$ matrix and c, b, x column vectors,

standard form: max $c^\mathsf{T} x$ s.t. $Ax = b,\ x \geq 0$; **canonical form:** max $c^\mathsf{T} x$ s.t. $Ax \leq b,\ x \geq 0$.

Theorem 12.1. *Every LP can be rewritten in standard form and in canonical form.*

Proof. Apply the following rewrites, each of which preserves the set of optimal solutions after renaming variables.

- Minimizing z is maximizing $-z$.
- A free variable x_j becomes $x_j^+ - x_j^-$ with $x_j^+, x_j^- \geq 0$; every real value of x_j arises this way.
- An inequality $a^\mathsf{T} x \leq b_i$ becomes $a^\mathsf{T} x + s = b_i$ with a SLACK VARIABLE $s \geq 0$, and $a^\mathsf{T} x \geq b_i$ becomes $a^\mathsf{T} x - s = b_i$ with a SURPLUS VARIABLE $s \geq 0$. Each constraint gets its own new variable. In matrix form, $Ax \leq b$ becomes $Ax + Is = b$, $s \geq 0$.
- An equality $a^\mathsf{T} x = b_i$ becomes the pair $a^\mathsf{T} x \leq b_i$ and $-a^\mathsf{T} x \leq -b_i$, and any $\geq$ constraint is negated into a $\leq$ constraint. In matrix form, $Ax = b$ becomes $\left[\begin{smallmatrix} A \\ -A \end{smallmatrix}\right] x \leq \left[\begin{smallmatrix} b \\ -b \end{smallmatrix}\right]$. □

For example, minimizing $2x_1 - x_2$ subject to $x_1 + x_2 \geq 2$, $3x_1 + 2x_2 \leq 4$, $x_1 + 2x_2 = 3$, x_1 free, $x_2 \geq 0$ is equivalent to the standard-form LP

$$\begin{aligned} \text{maximize} \quad & -2x_1^+ + 2x_1^- + x_2 \\ \text{subject to} \quad & x_1^+ - x_1^- + x_2 - x_3 = 2, \\ & 3x_1^+ - 3x_1^- + 2x_2 + x_4 = 4, \\ & x_1^+ - x_1^- + 2x_2 = 3, \\ & x_1^+, x_1^-, x_2, x_3, x_4 \geq 0. \end{aligned}$$

Standard form is used by the simplex method, and canonical form is convenient for duality.

12.3 Basic feasible solutions

Consider an LP in standard form with A of size $m \times n$. If $Ax = b$ has no solution, the LP is infeasible; the converse fails, since every solution may have a negative coordinate. Assume $Ax = b$ is solvable and that redundant equations have been removed, so $\operatorname{rank} A = m$. A set $B \subseteq \{1, \ldots, n\}$ with $|B| = m$ is a BASIS if the $m \times m$ submatrix A_B of the columns indexed by B is invertible; there are at most $\binom{n}{m}$ bases. With N the complementary indices, $Ax = b$ reads $A_B x_B + A_N x_N = b$, that is,

$$x_B = A_B^{-1} b - A_B^{-1} A_N x_N.$$

Setting $x_N = 0$ gives the BASIC SOLUTION $(A_B^{-1} b, 0)$, and it is a BASIC FEASIBLE SOLUTION (bfs) if moreover $A_B^{-1} b \geq 0$.

Theorem 12.2. *If an LP in standard form with* $\operatorname{rank} A = m$ *is feasible and bounded, then some optimal*

solution is a basic feasible solution.

Proof sketch. Let x^* be optimal and $J = \{j : x_j^* > 0\}$. If the columns A_J are linearly independent, extend J to a basis B using $\operatorname{rank} A = m$; then x^* is the bfs for B. Otherwise pick $y \neq 0$ with $A_J y_J = 0$ and $y_j = 0$ for $j \notin J$. Since $x_J^* > 0$, both $x^* \pm \epsilon y$ are feasible for small $\epsilon > 0$, so optimality forces $c^\mathsf{T} y = 0$. Replacing y by $-y$ if necessary, some $y_j < 0$. Increase λ from 0 until a coordinate of $x^* + \lambda y$ in J first reaches 0. The result is optimal with fewer positive coordinates, and repeating reaches the independent case. □

Geometrically, the basic feasible solutions are the corners of the feasible region. In the bake sale, standard form adds slacks: $x_1 + 3x_2 + s_1 = 9$, $2x_1 + x_2 + s_2 = 8$, $1.5x_1 + s_3 = 15$. The basis $\{x_1, x_2, s_3\}$ gives $(x_1, x_2) = (3, 2)$ with $s_3 = 10.5$, a corner. The basis $\{x_1, s_1, s_2\}$ sets $x_2 = s_3 = 0$, so $x_1 = 10$, $s_1 = -1$, $s_2 = -12$. This basic solution is the intersection of $x_1 = 10$ with $x_2 = 0$ and is infeasible.

Note. The SIMPLEX METHOD (Dantzig) starts at a basic feasible solution, such as the origin here, and repeatedly moves along an edge of the feasible region to an adjacent corner that improves the objective. Such an edge exists unless the current corner is optimal. It works very well in practice, but for essentially every rule for choosing the edge there are examples requiring exponentially many steps. Spielman and Teng's *smoothed analysis* explains its practical success: after small random perturbations of the input, the expected running time is polynomial.

12.4 Duality

How good is a proposed optimum for the bake sale (12.1)? Any feasible point gives a lower bound on the maximum: $(2, 2)$ is feasible with $z = 80$. Upper bounds come from combining constraints. Since $x_2 \geq 0$,

$$20x_1 + 20x_2 \leq 20(x_1 + 3x_2) \leq 180, \qquad 20x_1 + 20x_2 \leq 20(2x_1 + x_2) \leq 160.$$

Adding the two constraints gives $3x_1 + 4x_2 \leq 17$, and $20x_1 + 20x_2 \leq \frac{20}{3}(3x_1 + 4x_2) \leq \frac{340}{3} = 113\frac{1}{3}$. To find the best such bound, multiply the constraints by $y_1, y_2 \geq 0$ (negative multipliers would flip the inequalities) and add:

$$(y_1 + 2y_2)x_1 + (3y_1 + y_2)x_2 \leq 9y_1 + 8y_2.$$

If $y_1 + 2y_2 \geq 20$ and $3y_1 + y_2 \geq 20$, the left side is at least $20x_1 + 20x_2$ for $x \geq 0$, so $9y_1 + 8y_2$ is an upper bound. The best bound solves another LP, the DUAL:

$$\text{minimize } 9y_1 + 8y_2 \quad \text{subject to} \quad y_1 + 2y_2 \geq 20, \quad 3y_1 + y_2 \geq 20, \quad y_1, y_2 \geq 0.$$

The primal point $x = (3, 2)$ has $z = 100$, and the dual point $y = (4, 8)$ is feasible ($4 + 16 = 20$, $12 + 8 = 20$) with $9 \cdot 4 + 8 \cdot 8 = 100$. The two bounds meet, so 100 is the optimum, and the pair of points certifies it.

In general, for a PRIMAL LP in canonical form, the dual transposes A and exchanges the objective and the right-hand sides:

$$\textbf{primal: } \max\ z = c^{\mathsf{T}}x \ \text{ s.t. } Ax \le b,\ x \ge 0; \qquad \textbf{dual: } \min\ w = b^{\mathsf{T}}y \ \text{ s.t. } A^{\mathsf{T}}y \ge c,\ y \ge 0.$$

In the example $A = \left[\begin{smallmatrix} 1 & 3 \\ 2 & 1 \end{smallmatrix}\right]$ and the dual matrix is $A^{\mathsf{T}} = \left[\begin{smallmatrix} 1 & 2 \\ 3 & 1 \end{smallmatrix}\right]$.

Theorem 12.3 [WEAK DUALITY]. *If x is feasible for the primal and y is feasible for the dual, then $c^{\mathsf{T}}x \le b^{\mathsf{T}}y$.*

Proof. Using $c \le A^{\mathsf{T}}y$ with $x \ge 0$, then $Ax \le b$ with $y \ge 0$,

$$c^{\mathsf{T}}x \le (A^{\mathsf{T}}y)^{\mathsf{T}}x = y^{\mathsf{T}}Ax \le y^{\mathsf{T}}b = b^{\mathsf{T}}y. \qquad \square$$

Theorem 12.4 [STRONG DUALITY]. *If the primal optimum z^* is finite, then so is the dual optimum w^*, and $z^* = w^*$.*

Strong duality, one of the central theorems of combinatorial optimization, is proved in the next lecture.

Proposition 12.5. *The dual of the dual is the primal.*

Proof. In canonical form the dual reads: maximize $-b^{\mathsf{T}}y$ subject to $-A^{\mathsf{T}}y \le -c$, $y \ge 0$. Its dual is: minimize $-c^{\mathsf{T}}x$ subject to $(-A^{\mathsf{T}})^{\mathsf{T}}x \ge -b$, $x \ge 0$. Negating the objective and the constraints gives: maximize $c^{\mathsf{T}}x$ subject to $Ax \le b$, $x \ge 0$. $\square$

A useful habit is to write down the dual of a new LP and look for an intuitive reason why it bounds the primal. Sometimes it is obvious, sometimes it takes thought, and sometimes none is found. Zero-sum games and maximum flow give examples where both programs have clear meanings.

Source. Peter Shor, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 12, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 13: Duality in Linear Programming

13.1 Weak duality and its consequences

Recall the canonical form of a linear program and its dual:

$$\begin{aligned} &\text{(P)} \quad \max\ z = c^\mathsf{T} x \quad \text{subject to } Ax \le b,\ x \ge 0, \\ &\text{(D)} \quad \min\ w = b^\mathsf{T} y \quad \text{subject to } A^\mathsf{T} y \ge c,\ y \ge 0. \end{aligned}$$

The dual takes the right-hand sides of the primal as its objective and the primal objective as its right-hand sides. A point is feasible if it satisfies the constraints.

Theorem 13.1 [Weak duality]. *If x is feasible for (P) and y is feasible for (D), then $c^\mathsf{T} x \le b^\mathsf{T} y$.*

Proof. Since $x \ge 0$ and $A^\mathsf{T} y \ge c$, and since $y \ge 0$ and $Ax \le b$,

$$c^\mathsf{T} x \le (A^\mathsf{T} y)^\mathsf{T} x = y^\mathsf{T} A x \le y^\mathsf{T} b. \qquad \square$$

A linear program either has a finite optimum, is infeasible (for example $x_1 \ge 5$ and $x_1 \le 3$), or is unbounded, meaning feasible points have arbitrarily good objective value (for example, maximize x subject to $x \ge 3$). Weak duality immediately gives:

- if the primal is unbounded, the dual is infeasible, since a feasible y would bound every primal value by $b^\mathsf{T} y$;
- if the dual is unbounded, the primal is infeasible.

It is also possible for both the primal and the dual to be infeasible. Strong duality states that if the primal has a finite optimum, so does the dual, and the two optimal values are equal.

13.2 Taking the dual of a general linear program

The dual has one variable for each primal constraint and one constraint for each primal variable. It is obtained by multiplying the primal constraints by dual variables and adding them so that every

inequality points the same way. Consider

$$\begin{aligned} \max \quad & 3x_1 + 5x_2 - 2x_3 \\ \text{s.t.} \quad & x_1 + x_2 \le 8 \\ & x_1 - x_3 \ge 1 \\ & x_2 + x_3 = 4 \\ & x_1, x_3 \ge 0, \quad x_2 \text{ unrestricted.} \end{aligned}$$

Multiply the three constraints by y_1, y_2, y_3. To add them into a single upper bound, the $\le$ constraint needs $y_1 \ge 0$, the $\ge$ constraint needs $y_2 \le 0$ (which flips it), and the equation allows y_3 of either sign. So, in a maximization, a $\le$ constraint gives a nonnegative dual variable, a $\ge$ constraint a nonpositive one, and an equation an unrestricted one.

Negative variables are error prone, so it is simpler to first rewrite every inequality in the direction of the canonical form. For a maximization, turn $x_1 - x_3 \ge 1$ into $-x_1 + x_3 \le -1$. Then read off the dual using the rules in Table 1.

Table 1: Dual of a maximization whose inequalities are all written as $\le$.

Primal (max)	Dual (min)
constraint i: $\le b_i$	variable $y_i \ge 0$
constraint i: $= b_i$	variable y_i unrestricted
variable $x_j \ge 0$	constraint j: $(A^{\mathsf{T}}y)_j \ge c_j$
variable x_j unrestricted	constraint j: $(A^{\mathsf{T}}y)_j = c_j$

For the example, the transpose of the constraint matrix gives

$$\begin{aligned} \min \quad & 8y_1 - y_2 + 4y_3 \\ \text{s.t.} \quad & y_1 - y_2 \ge 3 \\ & y_1 + y_3 = 5 \\ & y_2 + y_3 \ge -2 \\ & y_1, y_2 \ge 0, \quad y_3 \text{ unrestricted.} \end{aligned}$$

The equation comes from the unrestricted variable x_2, and y_3 is unrestricted because the third primal constraint is an equation.

13.3 Complementary slackness

Theorem 13.2 [Complementary slackness]. *Let x be feasible for (P) and y feasible for (D). Then x and y are both optimal if and only if*

1. *for every i, either $y_i = 0$ or $(Ax)_i = b_i$, and*

2. *for every j, either $x_j = 0$ or $(A^\mathsf{T} y)_j = c_j$.*

In words, a dual variable can be nonzero only if its primal constraint is tight, and a primal variable can be nonzero only if its dual constraint is tight. The theorem holds for every linear program; it suffices to prove it in canonical form, since every program can be brought to that form.

Proof. The proof of Theorem 13.1 gives $c^\mathsf{T} x \le y^\mathsf{T} A x \le y^\mathsf{T} b$, and these two inequalities can be rewritten as

$$(y^\mathsf{T} A - c^\mathsf{T})\, x \ge 0, \qquad y^\mathsf{T}(b - Ax) \ge 0.$$

In each product, both vectors have nonnegative entries, so the product is zero exactly when, in every coordinate, one of the two factors is zero. These are conditions (2) and (1).
If (1) and (2) hold, both products vanish, so $c^\mathsf{T} x = b^\mathsf{T} y$. By weak duality no primal value exceeds $b^\mathsf{T} y$ and no dual value is below $c^\mathsf{T} x$, so x and y are optimal. Conversely, if x and y are optimal, strong duality gives $c^\mathsf{T} x = b^\mathsf{T} y$, so both inequalities are equalities, both products vanish, and (1) and (2) hold. □

Example: Complementary slackness recovers a dual optimum from a primal one. In the example above, $x = (4, 4, 0)$ is feasible: $4+4 \le 8$, $-4+0 \le -1$, and $4+0 = 4$. Its value is $3 \cdot 4 + 5 \cdot 4 = 32$. Since $x_1, x_2 \neq 0$, the first two dual constraints are tight: $y_1 - y_2 = 3$ and $y_1 + y_3 = 5$. The second primal constraint is slack ($-4 < -1$), so $y_2 = 0$. Hence $y = (3, 0, 2)$, which is feasible ($y_2 + y_3 = 2 \ge -2$) and has value $8 \cdot 3 - 0 + 4 \cdot 2 = 32$. Equal values certify that both are optimal.

13.4 A mechanical justification of strong duality

Strong duality can be proved by running the simplex method and reading the dual optimum off the final tableau, or by Farkas' lemma, which is essentially strong duality restated. A third route, sketched here, assumes a basic fact from mechanics: a ball at rest is in equilibrium, so the forces on it sum to zero.

Take the program

$$\text{(P')} \quad \min\ c^\mathsf{T} x \ \text{ s.t. } Ax \ge b,\ x \text{ unrestricted}, \qquad \text{(D')} \quad \max\ b^\mathsf{T} y \ \text{ s.t. } A^\mathsf{T} y = c,\ y \ge 0.$$

The dual has equations because every x_j is unrestricted. Each constraint $a_i^\mathsf{T} x = \sum_j a_{ij} x_j \ge b_i$ says the point lies on one side of a hyperplane. Treat each hyperplane as a solid wall, and let a ball fall under a gravity that pulls in the direction of decreasing $c^\mathsf{T} x$. Assume the optimum is finite. The ball rolls along the walls and comes to rest at an optimal point x.

At rest, the force of gravity is balanced by the walls. A wall can only push perpendicular to itself and toward the feasible side, so wall i exerts a force $y_i a_i$ with $y_i \ge 0$. Balance of forces gives $\sum_i y_i a_i = c$,

that is, $A^{\mathsf{T}} y = c$, so y is feasible for (D′). Moreover, wall i pushes only if the ball touches it:

$$y_i > 0 \implies a_i^{\mathsf{T}} x = b_i.$$

Therefore $y^{\mathsf{T}}(Ax - b) = \sum_i y_i(a_i^{\mathsf{T}} x - b_i) = 0$, and

$$c^{\mathsf{T}} x = (A^{\mathsf{T}} y)^{\mathsf{T}} x = y^{\mathsf{T}} A x = y^{\mathsf{T}} b.$$

The primal and dual values agree, so by weak duality both are optimal. The two physical conditions are exactly complementary slackness.

13.5 König's theorem

Let G be a bipartite graph. A MATCHING is a set of edges no two of which share an endpoint, and a VERTEX COVER is a set of vertices meeting every edge.

Theorem 13.3 [KÖNIG]. *In a bipartite graph, the maximum size of a matching equals the minimum size of a vertex cover.*

One inequality is easy: a vertex cover must contain an endpoint of each edge of a matching, and these endpoints are distinct, so every vertex cover is at least as large as every matching.

For equality, relax both problems to linear programs. With a variable x_e for each edge,

$$\text{(M)} \quad \max \sum_e x_e \ \text{ s.t. } \sum_{e \ni v} x_e \le 1 \ \ \forall v, \quad x \ge 0,$$

and, with a variable y_v for each vertex, its dual is

$$\text{(C)} \quad \min \sum_v y_v \ \text{ s.t. } y_u + y_v \ge 1 \ \ \forall\, e = uv, \quad y \ge 0.$$

A 0/1 solution of (M) is exactly a matching, and a 0/1 solution of (C) is exactly a vertex cover. It remains to show that both programs have integral optima; strong duality then gives the theorem.

Integrality of (M). Let x be optimal with the fewest fractional edges, and suppose the set F of edges with $0 < x_e < 1$ is nonempty. Changing the values on F by $\pm\varepsilon$ with alternating signs along a path or cycle keeps every vertex sum unchanged except at the ends of a path.

- If F contains a cycle, the cycle is even because G is bipartite. Alternating $+\varepsilon, -\varepsilon$ around it keeps every constraint and the objective unchanged.
- Otherwise, take a maximal path in F. Each endpoint v has no other fractional edge, and its integral edges are all 0: an edge with value 1 would give $\sum_{e \ni v} x_e > 1$. So the constraint at v has slack, and alternating signs along the path stays feasible for small ε. On an odd

path, adding ε to both end edges would increase the objective, contradicting optimality; so the path is even and the objective does not change.

Increase ε until some edge reaches 0 or 1. This gives an optimal solution with fewer fractional edges, a contradiction. □

A similar argument shows that (C) has an integral optimum, which completes the proof of König's theorem. In the same way, linear programs and their duals will be used to analyze zero-sum games and maximum flows.

Source. Peter Shor, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 13, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 14: Zero-Sum Games

14.1 Two-player zero-sum games

Definition 14.1 [Two-player zero-sum game]. A two-player zero-sum game consists of a row player (Alice) with m strategies, a column player (Bob) with n strategies, and an $m \times n$ payoff matrix C. If Alice plays i and Bob plays j, Bob pays Alice C_{ij} (a negative entry means Alice pays Bob).

The gains of the two players always sum to zero, hence the name.

Example: *Matching pennies.* Each player chooses H or T simultaneously. Alice wins 1 if the coins match and loses 1 otherwise:

$$C = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix},$$

with rows and columns indexed by H, T. By symmetry, neither player has an advantage. For the 3×3 game

$$C = \begin{pmatrix} 2 & -1 & 3 \\ 2 & 2 & -3 \\ -1 & 2 & -3 \end{pmatrix}$$

it is not obvious who is favored; this is answered at the end of the lecture.

14.2 Pure and mixed Nash equilibria

Definition 14.2 [Pure Nash equilibrium]. A pair of strategies (i, j) is a pure Nash equilibrium if no player has an incentive to deviate:

$$C_{i'j} \le C_{ij} \text{ for all } i', \qquad C_{ij'} \ge C_{ij} \text{ for all } j'.$$

The inequalities point in opposite directions because Bob's payoff is $-C_{ij}$. Matching pennies has no pure Nash equilibrium: for any pair of announced choices, the player who is losing gains by switching. This suggests letting players randomize.

Write $\Delta_m = \{x \in \mathbb{R}^m : x \ge 0,\ \sum_i x_i = 1\}$ for the set of probability distributions on m strategies, a set defined by linear constraints. If Alice samples i from $x \in \Delta_m$ and Bob independently samples j from $y \in \Delta_n$, Alice's expected payoff is

$$\sum_{i=1}^{m} \sum_{j=1}^{n} C_{ij} x_i y_j = x^{\mathsf{T}} C y.$$

Definition 14.3 [Mixed Nash equilibrium]. A pair $(x, y) \in \Delta_m \times \Delta_n$ is a mixed Nash equilibrium if no player gains by deviating to a pure strategy:

$$\sum_{j=1}^{n} C_{i'j} y_j \le x^\mathsf{T} C y \ \text{ for all } i', \qquad \sum_{i=1}^{m} C_{ij'} x_i \ge x^\mathsf{T} C y \ \text{ for all } j'.$$

Allowing deviations to mixed strategies would give the same notion: against a fixed y, the payoff of a mixed strategy x' is an average of the payoffs of pure strategies, so some pure strategy does at least as well. The following lemma makes this precise. Let e_j denote the j-th standard basis vector, the pure strategy j.

Lemma 14.4. *For $x \in \Delta_m$ and $\lambda \in \mathbb{R}$,*

$$x^\mathsf{T} C y \ge \lambda \ \textit{for all } y \in \Delta_n \iff x^\mathsf{T} C e_j \ge \lambda \ \textit{for all } j \in \{1, \dots, n\}.$$

Proof. The forward direction takes $y = e_j$. Conversely, write $y = \sum_j y_j e_j$. Then

$$x^\mathsf{T} C y = \sum_{j=1}^{n} y_j \, x^\mathsf{T} C e_j \ge \sum_{j=1}^{n} y_j \, \lambda = \lambda,$$

a nonnegative combination of the finitely many inequalities on the right. □

Deriving every constraint as a nonnegative combination of a few others is exactly the reasoning behind weak duality.

14.3 The minimax theorem

Theorem 14.5 [von Neumann's minimax theorem]. *For every payoff matrix C,*

$$\max_{x \in \Delta_m} \min_{y \in \Delta_n} x^\mathsf{T} C y = \min_{y \in \Delta_n} \max_{x \in \Delta_m} x^\mathsf{T} C y.$$

This common value λ^* is the value *of the game.*

The left side is the game in which Alice must commit to a mixed strategy first and Bob then best-responds; the right side is the game in which Bob commits first. For pure strategies, the order matters a great deal: in matching pennies, whoever moves second always wins. The theorem says that once players randomize, it does not matter who commits first.

Corollary 14.6. *Every two-player zero-sum game has a mixed Nash equilibrium.*

Proof. Let x^* attain the maximum on the left and y^* the minimum on the right of Theorem 14.5. Then

$$x^{*\mathsf{T}} C y \geq \lambda^* \ \text{ for all } y \in \Delta_n, \qquad x^{\mathsf{T}} C y^* \leq \lambda^* \ \text{ for all } x \in \Delta_m.$$

Taking $y = y^*$ and $x = x^*$ gives $x^{*\mathsf{T}} C y^* = \lambda^*$. The second family of inequalities with $x = e_{i'}$ says Alice cannot gain by deviating, and the first with $y = e_{j'}$ says the same for Bob. □

Proof of Theorem 14.5. By Lemma 14.4, the game in which Alice commits first is the linear program

$$\text{(P)} \quad \max \ \lambda \ \text{ s.t. } \ x^{\mathsf{T}} C e_j \geq \lambda \ \ \forall j, \quad \sum_{i=1}^{m} x_i = 1, \quad x \geq 0,$$

whose value is $\max_x \min_y x^{\mathsf{T}} C y$: Alice seeks the largest guarantee λ against every reply of Bob. Symmetrically, the game in which Bob commits first is

$$\text{(D)} \quad \min \ \mu \ \text{ s.t. } \ e_i^{\mathsf{T}} C y \leq \mu \ \ \forall i, \quad \sum_{j=1}^{n} y_j = 1, \quad y \geq 0,$$

with value $\min_y \max_x x^{\mathsf{T}} C y$. These programs are dual: assign the dual variable $y_j \geq 0$ to the constraint $\lambda - (C^{\mathsf{T}} x)_j \leq 0$ and the unrestricted variable μ to $\sum_i x_i = 1$. The column of $x_i \geq 0$ gives $\mu - (Cy)_i \geq 0$, and the column of the unrestricted λ gives $\sum_j y_j = 1$. Both programs are feasible, since any distribution together with a small enough λ (or large enough μ) works. By weak duality both optima are then finite, and strong duality gives equal optimal values. □

The duality has a game-theoretic reading. For feasible solutions of (P) and (D), $\lambda \leq x^{\mathsf{T}} C y \leq \mu$: a strategy for Bob certifies that Alice cannot guarantee more than μ, and vice versa. This is weak duality. Strong duality says the best such certificates meet.

14.4 Solving the 3×3 example

Solving (P) and (D) for the 3×3 game gives $\lambda^* = \frac{1}{3}$, with

$$x^* = \left(\tfrac{5}{9}, 0, \tfrac{4}{9}\right), \qquad y^* = \left(0, \tfrac{2}{3}, \tfrac{1}{3}\right).$$

The row player is favored. Indeed,

$$x^{*\mathsf{T}} C = \left(\tfrac{2}{3}, \tfrac{1}{3}, \tfrac{1}{3}\right), \qquad C y^* = \left(\tfrac{1}{3}, \tfrac{1}{3}, \tfrac{1}{3}\right)^{\mathsf{T}},$$

so Alice guarantees at least $\frac{1}{3}$ against every column and Bob guarantees losing at most $\frac{1}{3}$ against every row. Alice never plays her second row, and Bob never plays his first column.

The zero in y^* is forced by complementary slackness: the first entry of $x^{*\mathsf{T}} C$ exceeds λ^*, so Bob's

constraint for column 1 is slack, and the corresponding variable y_1^* must vanish. If Bob put positive weight on that column, Alice's strategy would earn more than $\frac{1}{3}$ and λ^* would not be the value.

Note. The same argument applies to enormous games that could never be written down, which makes the minimax theorem a tool for proving that combinatorial objects exist: it suffices to show that no strategy of an opponent can rule them out.

Source. Ankur Moitra, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 14, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 15: Max-Flow Min-Cut Theorem

15.1 Flows and cuts

A network is a directed graph with vertex set V and arc set A, a capacity $u(i,j) \geq 0$ on each arc $(i,j) \in A$, a source s, and a sink t. The flow could be water in pipes, traffic on roads, or data on a communication network.

Definition 15.1 [Flow]. A flow assigns a value $x_{ij} \geq 0$ to each arc so that

1. (capacity constraints) $x_{ij} \leq u(i,j)$ for every arc, and
2. (flow conservation) for every $i \neq s, t$, the flow into i equals the flow out of i.

Its value z is the net flow out of s, which equals the net flow into t.

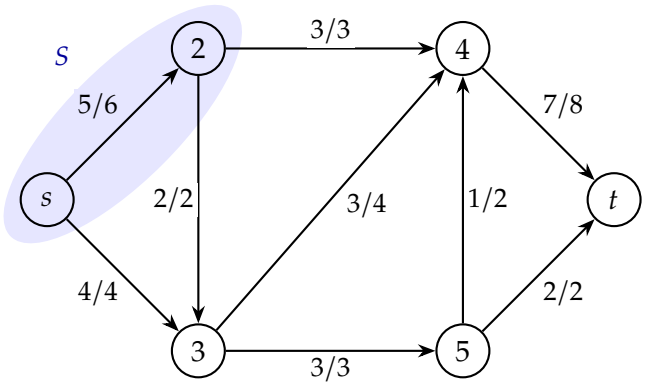


Figure 9: A flow of value 9; each arc is labeled flow/capacity. The three arcs leaving $S = \{s, 2\}$ have total capacity $4 + 2 + 3 = 9$, so no flow is larger.

Definition 15.2 [Cut]. An s-t cut is a set $S \subseteq V$ with $s \in S$ and $t \notin S$. Its capacity is

$$\text{cap}(S) = \sum_{(i,j) \in A:\ i \in S,\ j \notin S} u(i,j).$$

Only arcs leaving S count; arcs entering S cannot help flow escape.

Lemma 15.3. *If x is a flow of value z and S is an s-t cut, then $z \leq \text{cap}(S)$.*

Proof. Summing the net outflow over $i \in S$, arcs with both ends in S cancel, and only s

contributes, so

$$z = \sum_{(i,j)\in A:\ i\in S,\ j\notin S} x_{ij} \;-\; \sum_{(j,i)\in A:\ j\notin S,\ i\in S} x_{ji}. \tag{15.1}$$

Bound the first sum using $x_{ij} \le u(i,j)$ and drop the second using $x_{ji} \ge 0$. □

So max flow ≤ min cut. The goal is equality.

Theorem 15.4 [MAX-FLOW MIN-CUT]. *The maximum value of a flow equals the minimum capacity of an s-t cut.*

15.2 Proof by linear programming duality

With a variable x_{ij} for each arc and z for the value, the maximum flow is the linear program

$$\begin{aligned} \max\quad & z \\ \text{s.t.}\quad & \sum_{j:(i,j)\in A} x_{ij} - \sum_{j:(j,i)\in A} x_{ji} - (\mathbf{1}_{i=s} - \mathbf{1}_{i=t})\, z = 0 && \forall i \in V, \\ & x_{ij} \le u(i,j) && \forall (i,j) \in A, \\ & x, z \ge 0. \end{aligned}$$

Give the conservation constraint at i the unrestricted dual variable β_i and the capacity constraint on (i,j) the variable $\alpha_{ij} \ge 0$. The variable x_{ij} appears in the constraints of i, j, and (i,j), and z appears in those of s and t, so the rules of Table 1 give the dual

$$\begin{aligned} \min\quad & \sum_{(i,j)\in A} u(i,j)\, \alpha_{ij} \\ \text{s.t.}\quad & \alpha_{ij} + \beta_i - \beta_j \ge 0 && \forall (i,j) \in A, \\ & \beta_t - \beta_s \ge 1, \\ & \alpha \ge 0, \quad \beta \text{ unrestricted.} \end{aligned}$$

For fixed β, the best choice is $\alpha_{ij} = (\beta_j - \beta_i)^+$, where $(a)^+ = \max(a, 0)$. Shifting all β_i by a constant changes nothing, so take $\beta_s = 0$. Think of placing vertex i at position β_i on a line: an arc pointing right contributes its capacity times its length, and an arc pointing left contributes nothing.

Cuts give dual solutions. For a cut S, setting $\beta_i = 0$ for $i \in S$, $\beta_i = 1$ otherwise, and $\alpha_{ij} = 1$ exactly on arcs leaving S is feasible with objective $\mathrm{cap}(S)$. So the dual optimum is at most the minimum cut.

Dual solutions give cuts. Let (α, β) be dual optimal with $\beta_s = 0$. Replacing each β_i by $\min(\max(\beta_i, 0), 1)$ keeps $\beta_s = 0$ and $\beta_t = 1$ and does not increase any $(\beta_j - \beta_i)^+$, so assume all $\beta_i \in [0,1]$. Let $0 = b_0 < b_1 < \cdots < b_r = 1$ be the distinct values of β, and let

$$C_k = \{i \in V : \beta_i \le b_k\}, \qquad k = 0, \dots, r-1,$$

the cut obtained by splitting the line between b_k and b_{k+1}. Each is an s-t cut. An arc (i, j) with $\beta_i < \beta_j$ leaves exactly the cuts C_k with $\beta_i \le b_k < \beta_j$, and the gaps $b_{k+1} - b_k$ for these k add up to $\beta_j - \beta_i$. Hence, writing $\lambda_k = b_{k+1} - b_k$,

$$\sum_{(i,j)\in A} u(i,j)\,\alpha_{ij} \;\ge\; \sum_{(i,j)\in A} u(i,j)\,(\beta_j - \beta_i)^+ \;=\; \sum_{k=0}^{r-1} \lambda_k \,\mathrm{cap}(C_k), \qquad \sum_{k=0}^{r-1} \lambda_k = 1.$$

The right side is a weighted average of cut capacities, hence at least the minimum cut. Equivalently, C_k is the cut $\{i : \beta_i \le q\}$ for a threshold q chosen uniformly in $[0, 1)$, and the dual value bounds its expected capacity.

Proof of Theorem 15.4. The two observations show that the dual optimum equals the minimum cut. The primal optimum is the maximum flow, and the primal is feasible (the zero flow) and bounded (Lemma 15.3). By strong duality the two optima are equal. □

Analyzing the dual of a linear program and interpreting it for the original problem is a basic principle of combinatorial optimization.

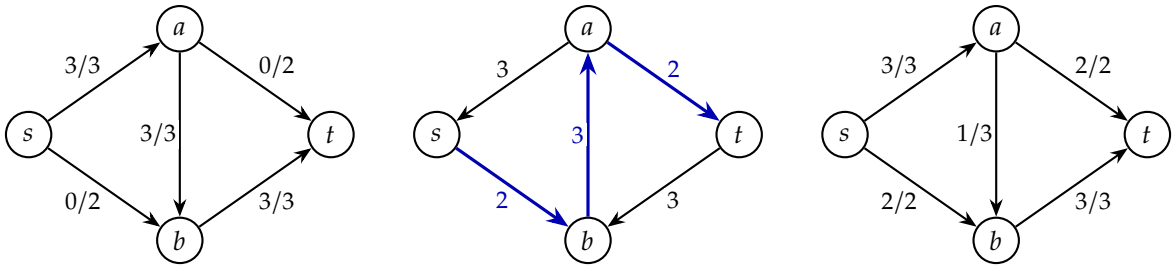
15.3 Augmenting paths

With integer capacities, the maximum flow value is an integer, since it equals the capacity of a cut. Whether some maximum flow is integral on every arc is less clear from the linear program. The augmenting path algorithm answers this.

Greedily pushing flow along paths can get stuck. In Figure 10, pushing 3 units along $s \to a \to b \to t$ leaves no s-t path of unsaturated arcs, yet a flow of value 5 exists. The fix is to allow pushing flow backward along an arc that already carries flow, which cancels part of that flow.

Definition 15.5 [Residual network]. Given a flow x, the residual network G_x has an arc (i, j) with residual capacity $u_x(i, j) = u(i, j) - x_{ij}$ for each $(i, j) \in A$ with $x_{ij} < u(i, j)$, and an arc (j, i) with $u_x(j, i) = x_{ij}$ for each $(i, j) \in A$ with $x_{ij} > 0$. An augmenting path is a directed s-t path in G_x.

Pushing $\varepsilon(P)$, the smallest residual capacity on an augmenting path P, increases the flow on forward arcs and decreases it on reversed arcs. This keeps conservation and capacity constraints and raises the value by $\varepsilon(P)$.



(a) after the path s, a, b, t (b) residual network (c) after augmenting by 2

Figure 10: Augmenting along $s \to b \to a \to t$ (bold) sends 2 more units and cancels 2 units on (a, b). The final flow has value 5, matching the cut $\{s, a, b\}$ of capacity $2 + 3$.

Algorithm 15.1: Augmenting path algorithm

Input: network (V, A), capacities u, source s, sink t
Output: a maximum flow x

$x \leftarrow 0$
while G_x contains an s-t path P **do**
  $\varepsilon \leftarrow \min_{(i,j)\in P} u_x(i, j)$
  for each arc $(i, j) \in P$ **do**
    if $(i, j) \in A$ and $x_{ij} < u(i, j)$ **then** $x_{ij} \leftarrow x_{ij} + \varepsilon$
    else $x_{ji} \leftarrow x_{ji} - \varepsilon$
return x

Theorem 15.6. *With integer capacities, the augmenting path algorithm terminates with an integral flow x^*, and there is an s-t cut whose capacity equals the value of x^*. In particular, x^* is a maximum flow.*

Proof. By induction, the flow stays integral: it starts at 0, and if x is integral, all residual capacities and hence $\varepsilon(P)$ are integers. Each augmentation raises the value by at least 1, and the value is bounded by $\mathrm{cap}(\{s\})$, so the algorithm terminates.
At termination, let S be the set of vertices reachable from s in G_{x^*}. Then $s \in S$ and $t \notin S$. No residual arc leaves S, so $x^*_{ij} = u(i, j)$ on every arc leaving S and $x^*_{ji} = 0$ on every arc entering S. By (15.1), the value of x^* is $\mathrm{cap}(S)$, and Lemma 15.3 shows both are optimal. □

For integer capacities, this gives a second proof of Theorem 15.4, and shows that integer capacities admit an integral maximum flow. With irrational capacities, a poor choice of augmenting paths may never terminate.

15.4 Bipartite matching

Let G be a bipartite graph with parts P and Q. Build a network H: add a source s with an arc (s, p) of capacity 1 for each $p \in P$, a sink t with an arc (q, t) of capacity 1 for each $q \in Q$, and direct each edge of G from P to Q with capacity 1 (or $+\infty$). Integral flows of value k correspond to matchings

of size k: the arcs from P to Q carrying flow 1 form the matching, since each vertex receives or sends at most one unit. By Theorem 15.6, a maximum matching is found by a maximum flow.

Proof of König's theorem (Theorem 13.3) by flows. Give the arcs from P to Q capacity $+\infty$, and let S be a minimum cut, which has finite capacity because $\{s\}$ does. No arc can go from $S \cap P$ to $Q \setminus S$, since it would have infinite capacity. So $C = (P \setminus S) \cup (Q \cap S)$ is a vertex cover. Its size is the number of cut arcs (s, p) with $p \notin S$ plus the number of cut arcs (q, t) with $q \in S$, which is $\mathrm{cap}(S)$, the size of a maximum matching. Since every vertex cover is at least as large as every matching, both are optimal. □

The same construction gives a criterion for matching all of P. For $R \subseteq P$ let $\Gamma(R) = \{q \in Q : (p, q) \in E \text{ for some } p \in R\}$ be its neighborhood.

Theorem 15.7 [Hall]. *A bipartite graph has a matching covering every vertex of P if and only if $|\Gamma(R)| \geq |R|$ for every $R \subseteq P$.*

Proof. If such a matching exists, the partners of R are $|R|$ distinct vertices of $\Gamma(R)$. Conversely, let S be any cut of finite capacity in H with infinite middle capacities. As above, $\Gamma(S \cap P) \subseteq S \cap Q$, so Hall's condition gives $|S \cap P| \leq |\Gamma(S \cap P)| \leq |S \cap Q|$ and

$$\mathrm{cap}(S) = |P \setminus S| + |S \cap Q| = |P| - |S \cap P| + |S \cap Q| \geq |P|.$$

The cut $\{s\}$ has capacity $|P|$, so the minimum cut, and hence the maximum matching, has size $|P|$. □

Source. Peter Shor, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 15, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 16: Data Compression and Shannon's Noiseless Coding Theorem

How much can data be compressed? Telegraph code books, in which a single paid word stood for a whole phrase, were an early answer. In 1948 Shannon founded information theory and determined exactly how far a random source can be compressed.

16.1 Sources and coding functions

Definition 16.1 [First-order source]. A first-order source consists of an alphabet $\mathcal{A} = \{a_1, \dots, a_k\}$ and a distribution $p = (p_1, \dots, p_k)$ on it. It emits letters $X_1, X_2, \dots$ that are i.i.d. with $\mathbb{P}(X_t = a_i) = p_i$.

A first-order source is a loaded k-sided die. Richer models, such as Markov chains in which a letter depends on the previous ones, are better models of English, and the theory extends to them.

Definition 16.2 [Coding function]. A coding function for messages of length n is an injective map $\phi : \mathcal{A}^n \to \{0,1\}^*$ into binary strings of any length.

Injectivity is exactly what decoding requires; the message is sent once, and its end is marked. The expected length is $\mathbb{E}[L]$, where $L = |\phi(X_1, \dots, X_n)|$. Frequent messages should get short codewords.

Example: Let $k = 2$, $p = (\frac{7}{8}, \frac{1}{8})$, and $n = 2$. The messages $a_1a_1, a_1a_2, a_2a_1, a_2a_2$ have probabilities $\frac{49}{64}, \frac{7}{64}, \frac{7}{64}, \frac{1}{64}$. Coding letter by letter ($a_1 \mapsto 0$, $a_2 \mapsto 1$) has $\mathbb{E}[L] = 2$. The code $0, 10, 110, 111$ has

$$\mathbb{E}[L] = \frac{49 \cdot 1 + 7 \cdot 2 + 7 \cdot 3 + 1 \cdot 3}{64} = \frac{87}{64} < 2.$$

16.2 Entropy and Shannon's theorem

Definition 16.3 [Entropy]. The entropy of a distribution $p = (p_1, \dots, p_k)$ is

$$H(p) = -\sum_{i=1}^{k} p_i \log_2 p_i,$$

with $0 \log_2 0 = 0$.

Each term is nonnegative, so $H(p) \geq 0$. By concavity of the logarithm, $H(p) \leq \log_2 k$, with equality for the uniform distribution $p_i = \frac{1}{k}$. The uniform source is the hardest to compress: when k is a power of 2, the naive code with $\log_2 k$ bits per letter cannot be beaten.

Theorem 16.4 [Shannon's noiseless coding theorem]. *Let a first-order source have entropy H.*

1. *Every coding function for messages of length n satisfies $\mathbb{E}[L] \geq Hn - o(n)$.*
2. *Some coding function satisfies $\mathbb{E}[L] \leq Hn + o(n)$.*

Here $o(n)$ denotes a quantity $f(n)$ with $f(n)/n \to 0$. So the source carries H bits of information per letter. The proof combines a bound for uniform distributions, counting, Stirling's formula, and Chebyshev's inequality.

16.3 Coding a uniform distribution

Lemma 16.5. *Let ϕ be a coding function for a set of N messages. Some codeword has length at least $\lfloor \log_2 N \rfloor$, and if the messages are equally likely, then $\mathbb{E}[L] \geq \log_2 N - 2$.*

Proof. Codewords are distinct nodes of the infinite binary tree whose root is the empty string; a node at depth ℓ is a string of length ℓ. Only $2^\ell - 1$ strings have length less than ℓ, so with $\ell = \lfloor \log_2 N \rfloor$ some codeword has length at least ℓ.
For equally likely messages, $\mathbb{E}[L]$ is the average depth, which is smallest when the levels of the tree are filled in order. Write $N = r\,2^\ell$ with $r \in [1, 2)$. Levels $0, \dots, \ell - 1$ hold $2^\ell - 1$ codewords with total depth $\sum_{d<\ell} d\,2^d = (\ell - 2)2^\ell + 2$, and the remaining $N - 2^\ell + 1$ codewords have depth at least ℓ. Thus

$$\mathbb{E}[L] \geq \frac{\ell N - 2^{\ell+1} + \ell + 2}{N} > \ell - \frac{2}{r} = \log_2 N - \left(\log_2 r + \frac{2}{r}\right) \geq \log_2 N - 2,$$

since $\log_2 r + \frac{2}{r}$ is convex on $[1, 2]$ and equals 2 at both endpoints. □

The lemma also bounds compression of structured objects. Writing each entry of a permutation of $\{1, \dots, n\}$ in $\lceil \log_2(n+1) \rceil$ bits uses $n \log_2 n + O(n)$ bits, and $\log_2 n! = n \log_2 n - O(n)$ by Stirling's formula, so this simple code is asymptotically optimal. Likewise, a Dyck path of length $2n$ is coded by $2n$ bits (up = 1, down = 0), and since there are $C_n = \frac{1}{n+1}\binom{2n}{n}$ of them, $\log_2 C_n = 2n - O(\log n)$; this code is also asymptotically optimal.

16.4 The lower bound

Suppose we are told the type of the message: the number n_i of occurrences of each a_i. Every message of that type has the same probability $p_1^{n_1} \cdots p_k^{n_k}$, and there are

$$M = \binom{n}{n_1, \dots, n_k} = \frac{n!}{n_1! \cdots n_k!}$$

of them. Conditioned on the type, the message is uniform on these M strings, so Lemma 16.5 gives

$$\mathbb{E}[L \mid n_1, \dots, n_k] \geq \log_2 M - 2. \tag{16.1}$$

Stirling's formula $n! \sim \sqrt{2\pi n}\,(n/e)^n$ gives $\log_2 n! = n \log_2 n - n \log_2 e + O(\log n)$. The terms $n \log_2 e$ cancel because $\sum_i n_i = n$, and writing $n \log_2 n = \sum_i n_i \log_2 n$,

$$\log_2 M = n \log_2 n - \sum_{i=1}^{k} n_i \log_2 n_i + O(k \log n) = n\, H\!\left(\frac{n_1}{n}, \dots, \frac{n_k}{n}\right) + O(k \log n). \tag{16.2}$$

So knowing the type does not help beat the entropy of the empirical frequencies. It remains to show that the type is close to np with high probability.

Definition 16.6 [Typical message]. For $\varepsilon > 0$, a message is ε-typical if $|n_i - n p_i| \leq \varepsilon n$ for every i.

Each n_i is a sum of n independent indicators with mean np_i and variance $np_i(1-p_i)$. By Chebyshev's inequality and a union bound,

$$\mathbb{P}(\text{not } \varepsilon\text{-typical}) \leq \sum_{i=1}^{k} \frac{p_i(1-p_i)}{\varepsilon^2 n} \leq \frac{1}{\varepsilon^2 n}.$$

By continuity of H, there is $\delta(\varepsilon) \to 0$ as $\varepsilon \to 0$ such that $|H(q) - H(p)| \leq \delta(\varepsilon)$ whenever $|q_i - p_i| \leq \varepsilon$ for all i.

Proof of Theorem 16.4, part 1. By the law of total expectation, dropping the atypical term,

$$\mathbb{E}[L] \geq \mathbb{P}(\text{typical}) \cdot \mathbb{E}[L \mid \text{typical}] \geq \left(1 - \frac{1}{\varepsilon^2 n}\right)\bigl(nH - n\,\delta(\varepsilon) - O(k \log n)\bigr),$$

using (16.1) and (16.2) for each typical type. Choose $\varepsilon = \varepsilon_n \to 0$ slowly enough that $\varepsilon_n^2 n \to \infty$. Then $\mathbb{E}[L] \geq nH - o(n)$. □

16.5 An optimal code

Proof of Theorem 16.4, part 2. Encode a message as follows.

1. One bit says whether the message is ε-typical.
2. If it is typical, send its type $(n_1, \dots, n_k)$, using $k\lceil \log_2(n+1) \rceil$ bits, and then its index among the M messages of that type, using $\lceil \log_2 M \rceil$ bits. By (16.2),

$$\log_2 M \leq nH + n\,\delta(\varepsilon) + O(k \log n).$$

3. If it is not typical, send each letter naively with $\lceil \log_2 k \rceil$ bits.

Beyond $\log_2 M$, the flag, the type, and rounding cost $O(k \log n)$ bits, which is not linear in n. The atypical branch is expensive but rare, so

$$\mathbb{E}[L] \leq 1 + k\lceil \log_2(n+1) \rceil + nH + n\,\delta(\varepsilon) + O(k \log n) + \frac{1}{\varepsilon^2 n} \cdot n\lceil \log_2 k \rceil.$$

The last term is $\lceil \log_2 k \rceil / \varepsilon^2$. With $\varepsilon = \varepsilon_n \to 0$ and $\varepsilon_n^2 n \to \infty$, every term except nH is $o(n)$. □

This code is not practical: its codebook is exponentially large in n, and encoding and decoding require enumerating messages. Huffman codes, in the next lecture, give efficient encoding and decoding.

Source. Ankur Moitra, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 16, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 17: Huffman Coding

Shannon's entropy theorem says that a first-order source over an alphabet A, with letter probabilities $(p_a)_{a \in A}$, needs at least $nH(p) - o(n)$ bits on average to encode n letters, and that $nH(p) + o(n)$ bits suffice, where

$$H(p) = -\sum_{a \in A} p_a \log_2 p_a .$$

The scheme achieving the upper bound is impractical: it lists a codeword for every typical sequence, and the $o(n)$ term decays slowly. Huffman coding gives an explicit code described by $O(|A|)$ data that spends at most $H(p) + 1$ bits per letter, and coding blocks of letters brings the total down to $nH(p) + 1$.

17.1 Prefix codes and binary trees

Definition 17.1 [Prefix code]. A prefix code assigns to each letter $a \in A$ a binary string $\phi(a)$ such that for distinct letters $a \neq a'$, the string $\phi(a)$ is not a prefix of $\phi(a')$.

A prefix code is a binary tree whose leaves are the letters: $\phi(a)$ is read along the path from the root to a, with 0 for a left step and 1 for a right step. Prefix-freeness says exactly that no letter sits at an ancestor of another letter. For example, the tree in Figure 11 gives

$$a \mapsto 10, \quad b \mapsto 000, \quad c \mapsto 110, \quad d \mapsto 01, \quad e \mapsto 001, \quad f \mapsto 111.$$

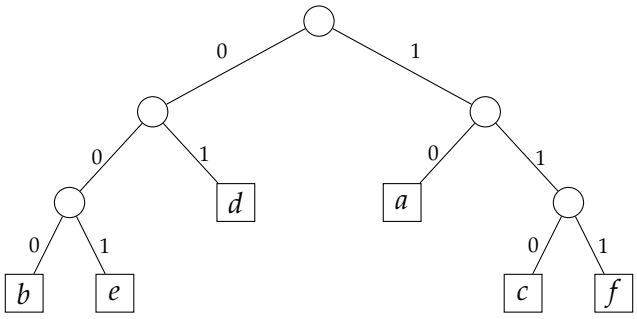


Figure 11: A prefix code as a binary tree. Letters are leaves; the bits on the edges from the root spell each codeword.

Encoding a message concatenates the codewords. Decoding walks down the tree from the root and outputs a letter each time it reaches a leaf, then restarts at the root. For instance, 1111001001 splits uniquely as $111 \mid 10 \mid 01 \mid 001$ and decodes to $fade$. No other split is possible: if a different first codeword were read, one codeword would be a prefix of another.

If $\ell_a = |\phi(a)|$, the EXPECTED LENGTH per letter is

$$L = \sum_{a \in A} p_a \ell_a,$$

and by linearity of expectation n letters cost nL bits on average. The goal is a prefix code minimizing L.

17.2 Huffman's algorithm

Running example: $A = \{a, b, c, d, e, f\}$ with

x	a	b	c	d	e	f
p_x	0.40	0.05	0.18	0.07	0.20	0.10

The argument assumes an optimal tree T and derives enough structure to build it. An optimal prefix code exists: contracting an internal node with a single child shortens codewords, so it suffices to minimize over trees in which every internal node has two children, and there are only finitely many such trees with leaf set A.

Lemma 17.2 [TWO LEAST LIKELY LETTERS ARE SIBLINGS]. *Let x and y be letters with the two smallest probabilities. Some optimal tree has x and y as sibling leaves at maximum depth.*

Proof. Let T be optimal. Swapping the letters at two leaves keeps the code prefix-free. If $p_u \le p_v$ but $\ell_u < \ell_v$, swapping u and v changes L by

$$p_u \ell_v + p_v \ell_u - p_u \ell_u - p_v \ell_v = -(p_v - p_u)(\ell_v - \ell_u) \le 0.$$

Hence we may swap x into a leaf of maximum depth without increasing L. Its parent has a second child: otherwise contracting the edge to x would shorten $\phi(x)$ by one bit and strictly decrease L. Since x is at maximum depth, this sibling is a leaf, holding some letter z. Swapping y and z does not increase L, because $p_y \le p_z$ and $\ell_y \le \ell_z$, and makes x and y siblings. □

Merge x and y into a new letter α with $p_\alpha = p_x + p_y$, and let $A' = (A \setminus \{x, y\}) \cup \{\alpha\}$.

Lemma 17.3 [MERGING PRESERVES OPTIMALITY]. *Let T be an optimal tree for A in which x and y are siblings, and let T' be the tree for A' obtained by replacing their parent and the two leaves with a single leaf α. Then T' is optimal for A', and $L(T) = L(T') + p_x + p_y$.*

Proof. In T, $\ell_x = \ell_y = \ell_\alpha + 1$ and all other lengths agree, so $L(T) = L(T') + p_x + p_y$. Suppose some tree S' for A' has $L(S') < L(T')$. Replacing the leaf α in S' by an internal node with children x

and y gives a prefix code S for A with

$$L(S) = L(S') + p_x + p_y < L(T') + p_x + p_y = L(T),$$

contradicting the optimality of T. $\square$

Conversely, expanding α in an optimal tree for A' yields a tree for A of cost $L' + p_x + p_y$, which by Lemmas 17.2 and 17.3 is the optimal cost for A. Induction on $|A|$ therefore proves that the following greedy procedure is optimal; the base case $|A| = 2$ is the tree with one root and two leaves.

Algorithm 17.1: Huffman coding

Input: an alphabet A with probabilities p_a, $|A| \geq 2$
Output: an optimal prefix-code tree

```
make each a ∈ A a one-node tree of weight p_a
while more than one tree remains do
    remove the two trees T_x, T_y of smallest weights
    join them as the two children of a new root of weight p_x + p_y
return the remaining tree
```

Theorem 17.4 [Huffman]. *Huffman's algorithm outputs a prefix code with the minimum expected length among all prefix codes for A.*

Example: For the running example, the merges are

$$\alpha = \{b, d\}: \quad p_\alpha = 0.12, \qquad \beta = \{f, \alpha\}: \quad p_\beta = 0.22,$$
$$\gamma = \{c, e\}: \quad p_\gamma = 0.38, \qquad \delta = \{\beta, \gamma\}: \quad p_\delta = 0.60,$$

and finally a joins δ. Unwinding the merges from the two-letter alphabet $\{a, \delta\}$ gives the tree of Figure 12 and the code

$$a \mapsto 0, \quad f \mapsto 100, \quad b \mapsto 1010, \quad d \mapsto 1011, \quad c \mapsto 110, \quad e \mapsto 111.$$

Its expected length is $0.40 \cdot 1 + 0.05 \cdot 4 + 0.18 \cdot 3 + 0.07 \cdot 4 + 0.20 \cdot 3 + 0.10 \cdot 3 = 2.32$, compared with the entropy $H(p) \approx 2.255$ and the $\log_2 6 \approx 2.585$ bits of a fixed-length code. Ties may be broken arbitrarily; different choices give different optimal trees with the same expected length.

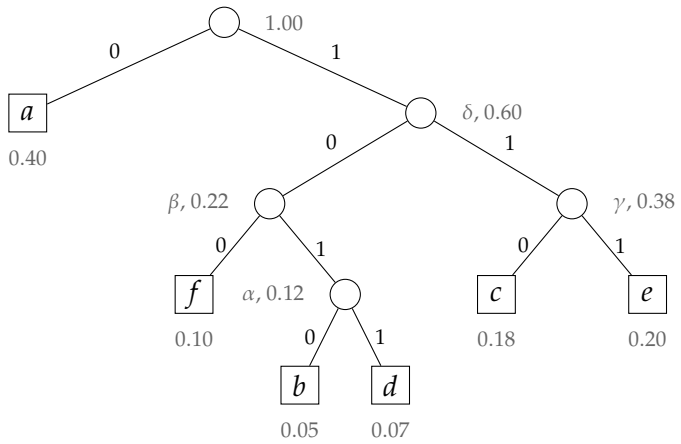


Figure 12: The Huffman tree for the running example, with the weight of each node in gray.

17.3 Bounding the expected length

Lemma 17.5 [Kraft's inequality]. *Let $\ell_1, \ldots, \ell_k \in \mathbb{N} \cup \{0\}$. There is a binary tree with k leaves at depths $\ell_1, \ldots, \ell_k$ (equivalently, a prefix code with these codeword lengths) if and only if*

$$\sum_{i=1}^{k} 2^{-\ell_i} \leq 1.$$

Proof. Let $D = \max_i \ell_i$. A node at depth ℓ has $2^{D-\ell}$ descendants at depth D in the complete tree of depth D, and since no leaf is an ancestor of another, these sets are disjoint for distinct leaves. So a tree gives $\sum_i 2^{D-\ell_i} \leq 2^D$.
Conversely, sort $\ell_1 \leq \cdots \leq \ell_k$ and number the 2^D nodes at depth D from left to right. Assign to leaf i the next $2^{D-\ell_i}$ consecutive depth-D nodes. The inequality guarantees there is room. Each earlier block size $2^{D-\ell_j}$ with $j < i$ is a multiple of $2^{D-\ell_i}$, so block i starts at a multiple of its own size and is exactly the set of descendants of one node at depth ℓ_i. Place leaf i at that node. The blocks are disjoint, so no chosen node is an ancestor of another. □

Theorem 17.6. *The Huffman code has expected length $H(p) \leq L \leq H(p) + 1$.*

Proof. Upper bound. Let $\ell_a = \lceil -\log_2 p_a \rceil$. Then

$$\sum_{a \in A} 2^{-\ell_a} \leq \sum_{a \in A} 2^{\log_2 p_a} = \sum_{a \in A} p_a = 1,$$

so by Lemma 17.5 some prefix code has these lengths. Huffman's code is optimal, so

$$L \le \sum_{a \in A} p_a \lceil -\log_2 p_a \rceil \le \sum_{a \in A} p_a(-\log_2 p_a + 1) = H(p) + 1.$$

Lower bound. Encoding n letters one at a time with any prefix code gives a coding function that is one-to-one on sequences, with expected length nL. Shannon's theorem gives $nL \ge nH(p) - o(n)$ for every n; dividing by n and letting $n \to \infty$ gives $L \ge H(p)$. □

17.4 Coding blocks of letters

A pair of independent letters is a single letter of the source on $A \times A$ with probabilities $p_a p_{a'}$. Its entropy is

$$-\sum_{a,a'} p_a p_{a'} \log_2(p_a p_{a'}) = -\sum_{a,a'} p_a p_{a'} \log_2 p_a - \sum_{a,a'} p_a p_{a'} \log_2 p_{a'} = 2H(p),$$

summing out the index absent from each logarithm. Likewise blocks of k letters form a source of entropy $kH(p)$, so Huffman coding the blocks costs between $kH(p)$ and $kH(p) + 1$ bits per block: the additive loss is 1 per block, not 1 per letter. The price is a code table of size $|A|^k$. Taking a single block of all n letters gives expected length between $nH(p)$ and $nH(p) + 1$, which recovers Shannon's theorem with the $o(n)$ term replaced by 1.

Source. Ankur Moitra, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 17, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 18: Transmitting Information Reliably over a Noisy Channel & Shannon's Noisy Coding Theorem

So far the goal was compression: removing redundancy. Over a noisy channel the goal reverses: add redundancy so that the receiver can undo the errors. Shannon's noisy coding theorem says exactly how much redundancy is necessary and sufficient.

18.1 The binary symmetric channel and repetition codes

Definition 18.1 [Binary symmetric channel]**.** The binary symmetric channel BSC(p) transmits one bit at a time; each bit is flipped with probability p and passes intact with probability $1-p$, independently of all other bits (Figure 13).

The receiver does not know where the flips occurred. On n bits, the law of large numbers gives about pn flips. Assume $p < \frac{1}{2}$; for $p > \frac{1}{2}$ the receiver can flip every received bit, and for $p = \frac{1}{2}$ the output is independent of the input, so nothing can be communicated.

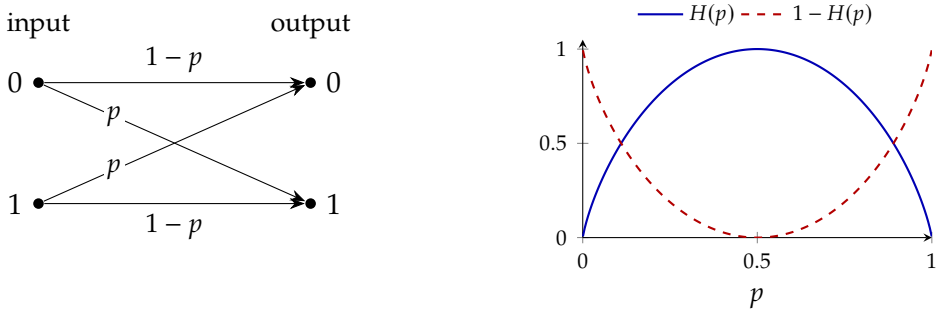


Figure 13: Left: the binary symmetric channel. Right: the binary entropy $H(p)$ and the capacity $1 - H(p)$.

The obvious defense is the repetition code: send each bit t times and decode each block by majority vote. With t odd, decoding a block fails only if at least $\lceil t/2 \rceil$ of its copies flip; for $t = 5$ the bit error probability falls from p to

$$\binom{5}{3}p^3(1-p)^2 + \binom{5}{4}p^4(1-p) + \binom{5}{5}p^5.$$

To send k bits with small total error, take $t = C \log k$. A block fails only if the number of flips exceeds its mean pt by a constant factor, so by the Chernoff bound it fails with probability $e^{-\Omega(t)} \le 1/k^2$ for a suitable constant C, and a union bound over the k blocks gives total failure probability at most $1/k$. Markov's inequality alone would force t to grow like k. Either way the rate, the message length divided by the transmitted length, is $1/t \to 0$ as $k \to \infty$. Until 1948 this was widely believed

unavoidable.

18.2 Encoding schemes and the capacity

Definition 18.2 [Encoding scheme]. A (k, n)-encoding function is a map $\mathrm{Enc} : \{0,1\}^k \to \{0,1\}^n$, and a (k, n)-decoding function is a map $\mathrm{Dec} : \{0,1\}^n \to \{0,1\}^k$. Together they form an encoding scheme of rate $R = k/n$.

A message m is encoded as the codeword $c = \mathrm{Enc}(m)$, sent through $\mathrm{BSC}(p)$, received as $\tilde{c}$, and decoded as $\tilde{m} = \mathrm{Dec}(\tilde{c})$. The best worst-case error at rate R and blocklength n is

$$\lambda^*(R, n) = \min_{(Rn,n)\text{-schemes}} \max_{m \in \{0,1\}^{Rn}} \mathbb{P}(\tilde{m} \neq m)$$

(rounding Rn to an integer).

Theorem 18.3 [Shannon's noisy coding theorem]. *Let $0 \le p < \frac{1}{2}$ and $H(p) = -p \log_2 p - (1-p)\log_2(1-p)$.*

1. *If $R > 1 - H(p)$, then $\lambda^*(R, n) \to 1$ as $n \to \infty$.*
2. *If $R < 1 - H(p)$, then $\lambda^*(R, n) \to 0$ as $n \to \infty$.*

The threshold $C = 1 - H(p)$ is the capacity of the channel: the number of message bits each channel use can carry. It equals 1 at $p = 0$, decreases in p, and equals 0 at $p = \frac{1}{2}$. The theorem is striking because a constant rate suffices for vanishing error. More generally, every memoryless channel has a capacity, though for many channels its numerical value is hard to determine.

18.3 Counting: Hamming balls and entropy

The Hamming distance $d_H(s, t)$ of two strings of equal length is the number of positions where they differ; for example $d_H(10110100, 00111100) = 2$. Entropy enters through binomial coefficients. By Stirling's formula $n! \sim \sqrt{2\pi n}\,(n/e)^n$,

$$\binom{n}{pn} \sim \frac{1}{\sqrt{2\pi n p(1-p)}}\, p^{-pn}(1-p)^{-(1-p)n}, \qquad \text{so} \qquad \frac{1}{n}\log_2 \binom{n}{pn} \to H(p).$$

Thus about $2^{H(p)n}$ strings lie at distance pn from a given string. The following bound is rigorous.

Lemma 18.4 [Volume of a Hamming ball]. *For $0 \le \lambda \le \frac{1}{2}$, $\displaystyle\sum_{j \le \lambda n} \binom{n}{j} \le 2^{H(\lambda)n}$.*

Proof. For $j \le \lambda n$ we have $\lambda^j(1-\lambda)^{n-j} \ge \lambda^{\lambda n}(1-\lambda)^{(1-\lambda)n} = 2^{-H(\lambda)n}$, because $\lambda/(1-\lambda) \le 1$. Hence

$$1 = (\lambda + 1 - \lambda)^n \ge \sum_{j \le \lambda n} \binom{n}{j} \lambda^j (1-\lambda)^{n-j} \ge 2^{-H(\lambda)n} \sum_{j \le \lambda n} \binom{n}{j}. \qquad \square$$

Definition 18.5 [Ring]**.** For $c \in \{0,1\}^n$ and $\gamma > 0$, let

$$\mathrm{Ring}_\gamma(c) = \left\{ \tilde{c} \in \{0,1\}^n : |d_H(c,\tilde{c}) - pn| \le \gamma n \right\}.$$

Two facts drive the proof. If $p + \gamma \le \frac{1}{2}$, then $|\mathrm{Ring}_\gamma(c)| \le 2^{H(p+\gamma)n}$ by Lemma 18.4. And $d_H(c,\tilde{c})$ is a sum of n independent Bernoulli(p) flips, so by Chebyshev's inequality $\mathbb{P}\big(\tilde{c} \notin \mathrm{Ring}_\gamma(c)\big) \le \frac{p(1-p)}{\gamma^2 n} \to 0$ for every fixed $\gamma > 0$.

18.4 Rates below capacity: random codes

The idea is to encode with a random function. Fix $R < 1 - H(p)$. Since H is continuous, choose $\gamma > 0$ with $p + \gamma \le \frac{1}{2}$ and $R < 1 - H(p+\gamma)$. Let $M = 2 \cdot 2^{Rn}$, twice the number of messages, and choose a codebook $C = \{c_1, \dots, c_M\}$ of independent uniform strings in $\{0,1\}^n$. Decode $\tilde{c}$ to c_i if $\tilde{c}$ lies in $\mathrm{Ring}_\gamma(c_i)$ and in no other ring; otherwise give up. Let $E_i(C)$ be the event that, when c_i is sent, the received $\tilde{c}_i$ lies in $\mathrm{Ring}_\gamma(c_i)$ and in no $\mathrm{Ring}_\gamma(c_j)$ with $j \ne i$, and let $\lambda_i(C) = 1 - \mathbb{P}(E_i(C))$ be the decoding failure probability for codeword i.

Lemma 18.6. *For every $\epsilon > 0$ and all large n, choosing C at random and i uniformly from $\{1, \dots, M\}$, $\mathbb{E}[\lambda_i(C)] \le \epsilon$.*

Proof. Fix i. The received word leaves its own ring with probability at most $\epsilon/2$ for large n, by Chebyshev. For $j \ne i$, the codeword c_j is uniform and independent of c_i and of the channel noise, so we may pick it after $\tilde{c}_i$ is known. Since the ring condition is symmetric,

$$\mathbb{P}\big(\tilde{c}_i \in \mathrm{Ring}_\gamma(c_j)\big) = \mathbb{P}\big(c_j \in \mathrm{Ring}_\gamma(\tilde{c}_i)\big) \le \frac{2^{H(p+\gamma)n}}{2^n}.$$

A union bound over the $M - 1$ other codewords bounds the probability of landing in some other ring by

$$M \cdot 2^{-(1-H(p+\gamma))n} = 2 \cdot 2^{-(1-H(p+\gamma)-R)n} \to 0.$$

So $\mathbb{P}(\text{not } E_i) \le \epsilon$ for large n, uniformly in i. $\square$

The lemma bounds the failure probability averaged over codebooks and messages, but the theorem needs one fixed code that works for every message. Two averaging steps remove the randomness.

Proof of Theorem 18.3, part 2. Write the lemma as an average over codebooks:

$$\mathbb{E}_C\left[\frac{1}{M}\sum_{i=1}^{M}\lambda_i(C)\right] \le \epsilon.$$

Some codebook C^* is at least as good as the average: $\frac{1}{M}\sum_i \lambda_i(C^*) \le \epsilon$ (the probabilistic method). By Markov's inequality, at most half of the indices have $\lambda_i(C^*) > 2\epsilon$; otherwise the average would exceed $\frac{1}{M}\cdot\frac{M}{2}\cdot 2\epsilon = \epsilon$. Let C' keep the $M/2 = 2^{Rn}$ codewords with $\lambda_i(C^*) \le 2\epsilon$. Removing codewords removes rings, so each failure probability can only decrease. Thus C' is an (Rn, n) scheme with $\max_m \mathbb{P}(\tilde{m} \neq m) \le 2\epsilon$, and ϵ was arbitrary. □

A finer analysis shows that the error probability decays exponentially in n, far better than the repetition code. The random code is still impractical: the codebook has exponentially many entries, and decoding searches all of them. Linear codes describe the codebook by a matrix.

18.5 Rates above capacity (sketch)

Suppose an (Rn, n) scheme decodes every message correctly with probability at least ϵ. The received word lies in $\mathrm{Ring}_\gamma(c_i)$ with probability close to 1, and each word in the ring has probability about $2^{-H(p)n}$. So the set of received words in the ring that decode to c_i must contain at least roughly $\frac{\epsilon}{2}\cdot 2^{H(p)n}$ words. These sets are disjoint for different messages, since Dec is a function, so

$$2^{Rn}\cdot\tfrac{\epsilon}{2}\,2^{(H(p)-o(1))n} \le 2^n, \qquad \text{that is,} \qquad R \le 1 - H(p) + o(1).$$

Making this precise shows that for $R > 1 - H(p)$ the success probability tends to 0.

Source. Ankur Moitra, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 18, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 19: Error-Correcting Codes: Hamming Codes

Shannon's random codes are impractical: the codebook is exponentially large, and finding the codeword nearest to a received word is NP-hard for general codes. Practical codes need structure that makes encoding and decoding efficient. Linear codes provide it. Hamming found the first family of them in 1950, two years after Shannon's theorem.

From now on errors are *adversarial*: a code CORRECTS t ERRORS if every codeword with any t or fewer symbols changed is still decoded correctly. This is stronger than handling random flips.

19.1 The $(7, 4)$ Hamming code

The $(7, 4)$ Hamming code encodes four message bits a, b, c, d into seven bits by appending three parity bits. Place the bits in a Venn diagram of three circles (Figure 14) and choose each parity bit so that the sum of the bits in its circle is 0 modulo 2:

$$p_1 = a + c + d, \qquad p_2 = a + b + d, \qquad p_3 = b + c + d.$$

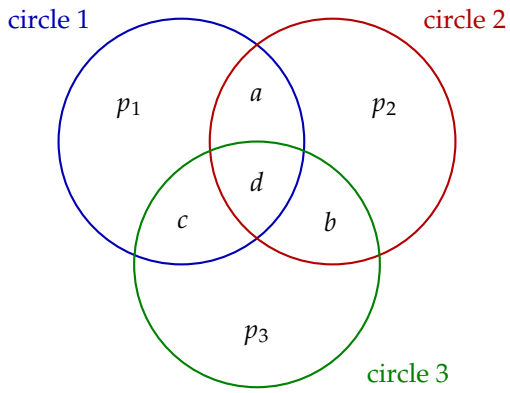


Figure 14: The $(7, 4)$ Hamming code. Each circle contains an even number of 1s; the message bits a, b, c, d sit in the overlaps.

To decode, recompute the three circle sums. A single flipped bit lies in a unique set of circles, so the pattern of failing circles locates it: a failure in circles 2 and 3 only means b was flipped, a failure in all three means d, a failure in circle 1 only means p_1, and no failure means no error.

Example: The message 1011 has $p_1 = 1$, $p_2 = 0$, $p_3 = 0$, so the codeword is 1011100. If the second bit flips, the received word is 1111100. Circle 1 still sums to $a + c + d + p_1 = 0$, but circles 2 and 3 sum to 1. The error is in circles 2 and 3 and not in 1, so it is b; correcting it gives 1011100, and the first four bits are the message.

19.2 Linear codes

Let $\mathbb{F}$ be a finite field. Binary codes use $\mathbb{F} = \mathbb{Z}_2$; Reed–Solomon codes will need a larger field.

Definition 19.1 [Linear code]. A linear code over $\mathbb{F}$ is a subspace $C \subseteq \mathbb{F}^n$: sums and scalar multiples of codewords are codewords. Equivalently, the encoding map $\mathbb{F}^k \to \mathbb{F}^n$ is linear. A linear code always contains 0.

The $(7, 4)$ Hamming code is linear, since the sum of two words satisfying the three parity checks satisfies them as well. A linear encoding map is determined by the images $c(e_i)$ of the standard basis vectors, so $c(m) = \sum_i m_i c(e_i) = mG$, where the $k \times n$ generator matrix G has rows $c(e_i)$. No codebook of $|\mathbb{F}|^k$ entries is needed, and the code is $C = \{mG : m \in \mathbb{F}^k\}$, the row space of G.

Lemma 19.2. *If the rows of $G \in \mathbb{F}^{k\times n}$ are linearly independent, then $m_1 G \neq m_2 G$ whenever $m_1 \neq m_2$.*

Proof. If $m_1 G = m_2 G$, then $(m_1 - m_2)G = 0$ is a linear combination of the rows of G with coefficients $m_1 - m_2$, which are not all zero. This contradicts independence. □

Row operations do not change the row space, and permuting coordinates does not change the code's error-correcting properties. Gaussian elimination therefore puts G in systematic form

$$G = \begin{bmatrix} I_k & S \end{bmatrix}, \qquad S \in \mathbb{F}^{k\times(n-k)},$$

so a codeword is the message followed by $n - k$ parity check symbols.

Definition 19.3 [Weight and distance]. The Hamming weight $w(v)$ of $v \in \mathbb{F}^n$ is its number of nonzero entries, and the Hamming distance is $d_H(u, v) = w(u - v)$, the number of positions where u and v differ. The minimum distance of a code C is $d^* = \min\{d_H(c_1, c_2) : c_1, c_2 \in C,\ c_1 \neq c_2\}$.

For example, $w(0, 1, 3, 2, 0, 4, 1) = 5$, and over $\mathbb{Z}_3$, $d_H\big((1, 0, 1, 2, 0, 1), (1, 0, 2, 2, 2, 0)\big) = 3$.

Theorem 19.4. *A code with minimum distance d^* corrects $t = \lfloor (d^* - 1)/2 \rfloor$ errors by nearest-codeword decoding. For a linear code, $d^* = \min\{w(c) : c \in C,\ c \neq 0\}$.*

Proof. If decoding could fail, some word $\tilde{c}$ would be within distance t of two codewords $c_1 \neq c_2$. The triangle inequality gives $d_H(c_1, c_2) \leq d_H(c_1, \tilde{c}) + d_H(\tilde{c}, c_2) \leq 2t < d^*$, a contradiction. For a linear code, $d_H(c_1, c_2) = w(c_1 - c_2)$ and $c_1 - c_2$ ranges over the nonzero codewords. □

So a single-error-correcting code needs $d^* \geq 3$.

19.3 Hamming codes

Let $G = [I_k \; S]$ be binary. A codeword is a sum of rows of G, and its identity part has one 1 for each row used. Therefore every nonzero codeword has weight at least 3 if and only if

- every row of S has weight at least 2 (one row gives weight 1 from I), and
- the rows of S are distinct (two rows give weight 2 from I and need a nonzero sum in S).

Sums of three or more rows already have weight at least 3 in the identity part. With $m = n - k$ parity bits, k is largest when the rows of S are all $2^m - m - 1$ strings in $\{0,1\}^m$ of weight at least 2. This is the Hamming code with

$$(n, k) = (2^m - 1,\ 2^m - 1 - m).$$

For $m = 3$ it is the $(7, 4)$ code above, with

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix},$$

and for $m = 4$ it is the $(15, 11)$ code. The rate $k/n = 1 - \frac{m}{2^m - 1}$ tends to 1, whereas the triple repetition code corrects one error at rate $\frac{1}{3}$. The weakness is that only one error per block is corrected: splitting a long message into 4-bit blocks at a bit error rate of 1% almost surely leaves some block with two errors.

19.4 Parity-check matrices and syndrome decoding

For $G = [I_k \; S]$ define the $n \times (n - k)$ parity-check matrix

$$H = \begin{bmatrix} -S \\ I_{n-k} \end{bmatrix}.$$

Over $\mathbb{Z}_2$, $-S = S$. Then $GH = -S + S = 0$, so every codeword satisfies $cH = mGH = 0$.

Theorem 19.5. $C = \{mG : m \in \mathbb{F}^k\} = \{c \in \mathbb{F}^n : cH = 0\}$.

Proof. The inclusion $\subseteq$ was just shown. If $cH = 0$, write $c = [m \; q]$ with $m \in \mathbb{F}^k$ and $q \in \mathbb{F}^{n-k}$. Then $cH = -mS + q = 0$, so $q = mS$ and $c = [m \; mS] = mG$. □

Note. Over a finite field, a subspace and its orthogonal complement can intersect. The last row 0001111 of the $(7, 4)$ generator matrix has even weight, so it is orthogonal to itself and to the other rows; it is a codeword and also lies in the column space of H. The dimensions still add to n.

If c is sent and $\tilde{c} = c + e$ is received, the SYNDROME is

$$\tilde{c}H = cH + eH = eH.$$

It depends only on the error, not on the message. For a general linear code, recovering e from eH is hard and code-specific. For a Hamming code it is immediate: a single error $e = e_i$ has syndrome e_iH, the ith row of H. The rows of H are distinct and nonzero, so the syndrome identifies the flipped position, and a zero syndrome means no error.

Example: For the $(7, 4)$ code,

$$H = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The received word $\tilde{c} = 1110111$ has syndrome $\tilde{c}H = 111$, the fourth row of H. So bit 4 is wrong, the codeword is 1111111, and the message is 1111. Similarly, 1110 encodes to 1110000; an error in position 6 gives 1110010, whose syndrome is 010, the sixth row of H.

The rows of H are all $2^{n-k} - 1$ nonzero strings of length $n - k$. Permuting coordinates so that row i of H is the binary representation of i makes the syndrome itself the binary address of the error.

19.5 Perfect codes

A binary code of length n correcting t errors has disjoint Hamming balls of radius t around its codewords, and there are 2^n words in total. For a linear (n, k) code this means

$$2^k \sum_{\ell=0}^{t} \binom{n}{\ell} \le 2^n, \qquad \text{equivalently} \qquad \sum_{\ell=0}^{t} \binom{n}{\ell} \le 2^{n-k}:$$

every error pattern of weight at most t needs its own syndrome. A code is PERFECT if equality holds, so that every word lies within distance t of exactly one codeword. For $t = 1$ the condition is $n + 1 = 2^{n-k}$, which Hamming codes meet: every Hamming code is perfect, and every nonzero syndrome is a single-bit error.

Perfect codes correcting more than one error are rare. The other binary solution is $n = 23$, $t = 3$,

where $1 + 23 + 253 + 1771 = 2048 = 2^{11}$; it is realized by the binary Golay code, a $(23, 12)$ code with minimum distance 7. Over $\mathbb{Z}_3$ there is also the ternary Golay code of length 11 and dimension 6, which corrects two errors.

Source. Peter Shor, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 19, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.

Lecture 20: Reed-Solomon Codes

Hamming codes correct only one error per block. Reed–Solomon codes (1960) correct many errors in each block. Combined with concatenated codes (1966) and the efficient Berlekamp–Massey decoder (1969), they made error correction practical: they were used on the Voyager probes (launched 1977) and on CDs. Turbo codes (1993), LDPC codes, and polar codes (2009) later came much closer to Shannon's capacity and replaced them in many new systems.

20.1 Reed–Solomon codes

The construction rests on one fact from algebra.

Theorem 20.1. *A nonzero polynomial of degree at most d over a field $\mathbb{F}$ has at most d roots.*

Definition 20.2 [Reed–Solomon code]. Let $\mathbb{F}$ be a finite field with $|\mathbb{F}| \geq n$ and choose distinct points $b_0, \dots, b_{n-1} \in \mathbb{F}$. The message $m = (m_0, \dots, m_{k-1}) \in \mathbb{F}^k$ defines $p(x) = \sum_{i=0}^{k-1} m_i x^i$, and its Reed–Solomon codeword is

$$c = \big(p(b_0), p(b_1), \dots, p(b_{n-1})\big).$$

We take $\mathbb{F} = \mathbb{Z}_p$ with $p \geq n$ prime and $b_i = i$. Finite fields exist exactly for prime-power sizes, and practical codes use $|\mathbb{F}| = 2^8$, so that each symbol is a byte; the theory is the same. The encoding is linear, $c = mG$ with

$$G = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 0 & 1 & 2 & \cdots & n-1 \\ 0 & 1^2 & 2^2 & \cdots & (n-1)^2 \\ \vdots & \vdots & \vdots & & \vdots \\ 0 & 1^{k-1} & 2^{k-1} & \cdots & (n-1)^{k-1} \end{bmatrix},$$

entries taken modulo p; column j evaluates p at j.

Theorem 20.3 [Minimum distance]. *For $k \leq n$, every nonzero Reed–Solomon codeword has weight at least $n - k + 1$. Hence $d^* = n - k + 1$, the rows of G are independent, and with $n = k + 2e$ the code corrects e errors.*

Proof. A nonzero message gives a nonzero p of degree at most $k - 1$, which has at most $k - 1$ roots by Theorem 20.1. So at most $k - 1$ entries of c are 0, and at least $n - k + 1$ are nonzero. By linearity this is the minimum distance. In particular no nonzero message encodes to 0, so the rows of G are independent. Finally, $n - k + 1 = 2e + 1$ gives $\lfloor (d^* - 1)/2 \rfloor = e$. □

20.2 Decoding by linear algebra

Suppose $c = (p(0), \dots, p(n-1))$ is sent with $n = k + 2e$, and $\tilde{c} = (r_0, \dots, r_{n-1})$ is received with at most e errors. Nearest codeword search is too slow. The trick is to introduce an unknown polynomial vanishing at the error positions.

Theorem 20.4. *If $d_H(c, \tilde{c}) \le e$, there are polynomials $f \neq 0$ of degree at most e and q of degree at most $k - 1 + e$ with*

$$q(i) = r_i f(i) \qquad \text{for } i = 0, 1, \dots, n-1.$$

Proof. Let the errors be at positions $i_1, \dots, i_\ell$, $\ell \le e$, and set $f(x) = \prod_{j=1}^{\ell} (x - i_j)$, the ERROR-LOCATOR POLYNOMIAL, and $q = pf$. At a correct position $r_i = p(i)$, so $q(i) = p(i)f(i) = r_i f(i)$. At an error position $f(i) = 0$, so $q(i) = 0 = r_i f(i)$. □

The conditions $q(i) = r_i f(i)$ are n homogeneous linear equations in the $(e+1) + (k+e) = n+1$ coefficients of f and q. Since there are more unknowns than equations, Gaussian elimination over $\mathbb{Z}_p$ finds a nonzero solution $(\hat{f}, \hat{q})$. Any such solution recovers p:

Why any solution works. At each of the at least $n - e = k + e$ correct positions, $r_i = p(i)$, so $\hat{q}(i) = p(i)\hat{f}(i)$. Thus $\hat{q} - p\hat{f}$ has degree at most $k + e - 1$ and at least $k + e$ roots, so $\hat{q} = p\hat{f}$. If $\hat{f} = 0$, then $\hat{q} = 0$ as well, contradicting that the solution is nonzero. So $\hat{f} \neq 0$ and $p = \hat{q}/\hat{f}$, an exact polynomial division. This also covers the case of fewer than e errors. □

Algorithm 20.1: Reed–Solomon decoding

Input: received word $(r_0, \dots, r_{n-1}) \in \mathbb{Z}_p^n$, with $n = k + 2e$
Output: the message polynomial p

1 solve $q(i) = r_i f(i)$, $0 \le i \le n-1$, for a nonzero pair with $\deg f \le e$, $\deg q \le k - 1 + e$
2 **return** q/f

Example: Over $\mathbb{Z}_7$ with $n = 7$ and $k = 3$, the code corrects $e = 2$ errors. The message $m = (2, 0, 5)$ gives $p(x) = 2 + 5x^2$ and codeword $c = (2, 0, 1, 5, 5, 1, 0)$. Suppose $r = (2, 2, 1, 0, 5, 1, 0)$ is received, with errors in positions 1 and 3. With $f = f_0 + f_1 x + f_2 x^2$ and $q = q_0 + \cdots + q_4 x^4$, the equations $q(i) = r_i f(i)$ are

$$\begin{aligned}
q_0 &= 2f_0, \\
q_0 + q_1 + q_2 + q_3 + q_4 &= 2(f_0 + f_1 + f_2), \\
q_0 + 2q_1 + 4q_2 + q_3 + 2q_4 &= f_0 + 2f_1 + 4f_2, \\
q_0 + 3q_1 + 2q_2 + 6q_3 + 4q_4 &= 0, \\
q_0 + 4q_1 + 2q_2 + q_3 + 4q_4 &= 5(f_0 + 4f_1 + 2f_2), \\
q_0 + 5q_1 + 4q_2 + 6q_3 + 2q_4 &= f_0 + 5f_1 + 4f_2, \\
q_0 + 6q_1 + q_2 + 6q_3 + q_4 &= 0.
\end{aligned}$$

One solution is $f = 1 + x + 5x^2 = 5(x-1)(x-3)$ and $q = 2 + 2x + x^2 + 5x^3 + 4x^4$; any nonzero multiple also works. Dividing,

$$p(x) = \frac{4x^4 + 5x^3 + x^2 + 2x + 2}{5x^2 + x + 1} = 5x^2 + 2,$$

so $m = (2, 0, 5)$. The roots of f are the error positions.

20.3 Faster decoding with finite differences

Solving an $n \times (n+1)$ system is the bottleneck. With the evaluation points $0, 1, \ldots, n-1$, it can be reduced to an $e \times (e+1)$ system.

For a sequence $a = (a_0, a_1, \ldots)$ define the FIRST DIFFERENCE $(\Delta a)_i = a_{i+1} - a_i$ and $\Delta^j a = \Delta(\Delta^{j-1} a)$; a finite sequence loses one term with each difference.

Lemma 20.5. *If $a_i = g(i)$ for a polynomial g of degree at most d, then $(\Delta a)_i = g_1(i)$ with $g_1(x) = g(x+1) - g(x)$ of degree at most $d-1$. Hence $\Delta^{d+1} a = 0$.*

Proof. In $g(x+1) - g(x)$ the terms of degree d cancel. Applying this d times leaves a constant sequence, whose difference is 0. □

For example, $g(x) = 2x^3 + x$ over $\mathbb{R}$ gives

a	0	3	18	57	132	255	438
Δa	3	15	39	75	123	183	
$\Delta^2 a$	12	24	36	48	60		
$\Delta^3 a$	12	12	12	12			
$\Delta^4 a$	0	0	0				

Theorem 20.6 [NEWTON'S FORWARD-DIFFERENCE FORMULA]. *Let $C_0(x) = 1$ and $C_j(x) = \frac{x(x-1)\cdots(x-j+1)}{j!}$. If $a_i = g(i)$ for a polynomial g of degree at most d, then*

$$g(x) = \sum_{j=0}^{d} (\Delta^j a)_0 \, C_j(x).$$

Over $\mathbb{Z}_p$ this requires $d < p$, so that each $j!$ is invertible.

Proof. Pascal's rule gives $C_j(x+1) - C_j(x) = C_{j-1}(x)$, and $C_j(0) = 0$ for $j \geq 1$. So the sequence $(C_j(i))_i$ has ℓth difference $C_{j-\ell}$ for $\ell \leq j$ and 0 for $\ell > j$, and its value at 0 is 1 if $\ell = j$ and 0 otherwise. The right side is therefore a polynomial of degree at most d whose differences at 0 match those of a up to order d. Since $a_i = \sum_{j \leq i} \binom{i}{j} (\Delta^j a)_0$, these differences determine $a_0, \ldots, a_d$,

and a polynomial of degree at most d is determined by $d+1$ values. $\square$

In the example, $(a_0, (\Delta a)_0, (\Delta^2 a)_0, (\Delta^3 a)_0) = (0, 3, 12, 12)$, and indeed $3x + 12 \cdot \frac{x(x-1)}{2} + 12 \cdot \frac{x(x-1)(x-2)}{6} = 2x^3 + x$.

Now apply this to decoding. Write the unknown error locator as $f(x) = \sum_{j=0}^{e} v_j x^j$, and let $a_i = r_i f(i)$. Then $a = \sum_j v_j b^{(j)}$, where $b_i^{(j)} = r_i i^j$ are known sequences. By Theorem 20.4 and Lemma 20.5, a comes from the polynomial q of degree at most $k + e - 1$, so its $(k+e)$th differences vanish:

$$0 = (\Delta^{k+e} a)_i = \sum_{j=0}^{e} v_j (\Delta^{k+e} b^{(j)})_i, \qquad 0 \le i \le n - 1 - (k+e) = e - 1.$$

These are e linear equations in the $e+1$ unknowns v_j, so a nonzero solution exists. Conversely, any nonzero solution works: the sequence a then has vanishing $(k+e)$th differences, so by Theorem 20.6 it agrees with $q(x) = \sum_{\ell < k+e} (\Delta^\ell a)_0 C_\ell(x)$, which has degree at most $k + e - 1$, and (f, q) solves the system of the previous section. The differences $(\Delta^\ell a)_0 = \sum_j v_j (\Delta^\ell b^{(j)})_0$ give q directly, and $p = q/f$. When e is small compared with k, this system is much smaller: for $e = 0.05k$ it has size about $e = n/22$ instead of n.

Example: Continue the $\mathbb{Z}_7$ example with $r = (2, 2, 1, 0, 5, 1, 0)$, $k = 3$, $e = 2$. The sequences $b_i^{(j)} = r_i i^j$ and their differences are

$b^{(0)}$	2	2	1	0	5	1	0
Δ	0	6	6	5	3	6	
Δ^2	6	0	6	5	3		
Δ^3	1	6	6	5			
Δ^4	5	0	6				
Δ^5	2	6					

$b^{(1)}$	0	2	2	0	6	5	0
Δ	2	0	5	6	6	2	
Δ^2	5	5	1	0	3		
Δ^3	0	3	6	3			
Δ^4	3	3	4				
Δ^5	0	1					

$b^{(2)}$	0	2	4	0	3	4	0
Δ	2	2	3	3	1	3	
Δ^2	0	1	0	5	2		
Δ^3	1	6	5	4			
Δ^4	5	6	6				
Δ^5	1	0					

The rows Δ^5 give

$$2v_0 + v_2 = 0, \qquad 6v_0 + v_1 = 0.$$

Taking $v_0 = 1$ gives $v_1 = 1$, $v_2 = 5$, so $f(x) = 1 + x + 5x^2 = 5(x-1)(x-3)$, locating the errors at positions 1 and 3. The leading differences of $a = b^{(0)} + b^{(1)} + 5b^{(2)}$ are $((\Delta^\ell a)_0)_{\ell=0}^{4} = (2, 5, 4, 6, 5)$. Over $\mathbb{Z}_7$,

$$C_1 = x, \quad C_2 = 4x^2 + 3x, \quad C_3 = 6x^3 + 3x^2 + 5x, \quad C_4 = 5x^4 + 5x^3 + 6x^2 + 5x,$$

so $q = 2C_0 + 5C_1 + 4C_2 + 6C_3 + 5C_4 = 4x^4 + 5x^3 + x^2 + 2x + 2$, and again $p = q/f = 5x^2 + 2$.

20.4 Concatenated codes

A Reed–Solomon code with $n = k + 2e$ corrects fewer than $n/2$ symbol errors. Over $\mathbb{F}_{2^8}$, a symbol is a byte. At a bit error rate of 10%, a byte is wrong with probability $1 - 0.9^8 \approx 0.57 > \frac{1}{2}$, so a Reed–Solomon code alone cannot cope, whatever its parameters.

A CONCATENATED CODE protects each symbol of an outer code with an inner code. Take the outer code to be Reed–Solomon over $\mathbb{F}_{2^8}$ with $n = 256$, $k = 150$, so $e = 53$, and the inner code to be a binary $(19, 8)$ code of minimum distance 7, which corrects 3 bit errors. To encode, split the message into 150 bytes, apply the outer code to get 256 bytes, and encode each byte into 19 bits with the inner code. To decode, decode each 19-bit block, then run the Reed–Solomon decoder on the resulting 256 bytes.

An inner block fails only with at least 4 flips among 19 bits, which has probability about 0.115. The outer decoder fails only if at least 54 of the 256 independent inner blocks fail, while about 29 fail on average; the binomial tail is about 8×10^{-6}. This code sends 1200 message bits as $256 \cdot 19 = 4864$ bits, rate about $\frac{1}{4}$. The capacity at $p = 0.1$ is $1 - H(0.1) \approx 0.53$, so an ideal code would need about 2300 bits: the concatenated code is within a factor of about 2. The inner code has only 2^8 codewords, so it can be decoded by a lookup table over all $2^{19} \approx 5 \times 10^5$ received words.

Source. Peter Shor, MIT 18.200, *Principles of Discrete Applied Mathematics*, Lecture 20, Spring 2024. Adapted from MIT OpenCourseWare, CC BY-NC-SA 4.0.